

Victor Patrick Sena Barbosa Lima

Comparação entre Modelos de Redes Neurais U-Net e U-Net Fuzzy para Mensurar Regiões de Imagens de Ultrassonografia Bovina



UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE MATEMÁTICA

2026

Victor Patrick Sena Barbosa Lima

Comparação entre Modelos de Redes Neurais U-Net e U-Net Fuzzy para Mensurar Regiões de Imagens de Ultrassonografia Bovina

Dissertação apresentada ao Programa de Pós-Graduação em Matemática da Universidade Federal de Uberlândia, como requisito parcial para obtenção do título de **Mestre em Matemática**.

Área de Concentração: Matemática.

Linha de Pesquisa: Matemática Aplicada.

Orientadora: Profa. Dra. Rosana Sueli da Motta Jafelice.

Coorientador: Prof. Dr. Caio Augusto Rodrigues dos Santos.

UBERLÂNDIA - MG

2026

Dedicatória

Dedico este trabalho à minha família, que sempre me apoiou e incentivou em minha jornada acadêmica. Sem o amor e o suporte de vocês, esta conquista não teria sido possível.

Agradecimentos

Gostaria de agradecer nominalmente a todos que, de alguma forma, contribuíram para a realização deste trabalho. Parafraseando Sir. Isaac Newton, só estou aqui porque me apoiei nos ombros de gigantes. Na impossibilidade de agradecer a todos, por serem muitos, quero mencionar, ao menos, quatro grupos a quem fiquei especialmente grato.

À minha família, meus pais, pelo apoio, incentivo e motivação ao longo de toda essa caminhada e minhas irmãs, que sempre estiveram oferecendo suporte e carinho.

Aos meus professores, especialmente à minha orientadora, professora Dra. Rosana S. M. Jafelice, e ao meu coorientador, professor Dr. Caio A. R. Santos, pela orientação, pelo apoio e por me ajudarem a superar os desafios encontrados durante o desenvolvimento deste trabalho.

Agradeço aos colaboradores, que disponibilizaram o banco de dados, professora Dra. Amanda M. Maiorano e as pesquisadoras do Centro Avançado de Pesquisa e Desenvolvimento de Bovinos de Corte, Maria E. Z. Mercadante e Karine D. S. Pinto.

Por fim, agradeço a Jéssica Plífinar e Cecília Plífinar, meus amores, que estiveram comigo durante essa jornada, pelo companheirismo e pelos ensinamentos. A todos que de alguma forma contribuíram para a realização deste trabalho, meu muito obrigado.

Resumo

A pecuária bovina é um dos pilares fundamentais da economia brasileira. A ultrassonografia de carcaça de bovinos permite mensurar, *in vivo*, a Área de Olho de Lombo (AOL) e, as Espessuras de Gordura subcutânea do Lombo (EGL) e da Garupa (EGG), relacionados à musculosidade e desenvolvimento precoce. Entretanto, a delimitação manual destes indicadores em imagens ultrassonográficas é um processo subjetivo, demorado e dependente da experiência do especialista. O objetivo deste trabalho é desenvolver e comparar dois modelos para delimitar e mensurar automaticamente esses indicadores em imagens de ultrassom de duas raças bovinas. O primeiro modelo é uma rede neural convolucional U-Net e o segundo, U-Net Fuzzy, é uma U-Net integrada ao Sistema de Inferência Neuro-Fuzzy Adaptativo, em inglês *Adaptive Neuro-Fuzzy Inference System* (ANFIS). As imagens foram obtidas de animais vivos do Centro Avançado de Pesquisa Tecnológica do Agronegócio (CAPTA) em Sertãozinho-SP, totalizando 135 imagens, das quais 108 são Nelore e 27 Caracu, que foram ampliadas por técnicas de expansão de dados, como a inclusão das mesmas imagens com redução e aumento de brilho. As máscaras de segmentação utilizadas como referência, conhecidas como *Ground Truth*, foram criadas manualmente com o auxílio de uma especialista, consistindo em imagens binárias utilizadas para isolar regiões específicas dentro da imagem, servindo como referência para o treinamento e avaliação dos modelos. O desempenho dos modelos é avaliado por meio de métricas como acurácia, erro absoluto médio, em inglês *Mean Absolute Error* (MAE) e o coeficiente de Dice, que quantifica a similaridade entre a segmentação produzida pelo modelo e as máscaras de segmentação desenvolvidas pela especialista. Os resultados indicam que o modelo U-Net Fuzzy é superior ao modelo U-Net para mensurar os indicadores AOL e EGG em todas as métricas de avaliação, com destaque para a acurácia de 94,17% na AOL e 98,6% na EGG. Em contrapartida, o U-Net obteve melhor resultado em todas as métricas para o indicador EGL, com foco especial para a métrica coeficiente de Dice de 70,25%. Por fim, foi desenvolvido um aplicativo computacional capaz de detectar regiões específicas e medir automaticamente os indicadores de interesse em imagens de ultrassom de bovinos utilizando ambos os modelos, podendo auxiliar nos processos de seleção e manejo dos rebanhos.

Palavras-chave: Ultrassonografia bovina; Delimitação de imagens; Redes neurais; U-Net; ANFIS.

Abstract

Cattle farming is one of the fundamental pillars of the Brazilian economy. Bovine carcass ultrasonography makes it possible to measure, *in vivo*, the Ribeye Area (REA), Backfat Thickness (BFT), and Rump Fat Thickness (RFT), which are related to muscularity, early fat development in the carcass, and meat quality. However, the manual delineation of these indicators in ultrasound images is a subjective, time-consuming process that depends on the specialist's experience. The objective of this work is to develop and compare two models to automatically delimit and measure these indicators in ultrasound images from two cattle breeds. The first model is a U-Net convolutional neural network, and the second, U-Net Fuzzy, is a U-Net integrated with the Adaptive Neuro-Fuzzy Inference System (ANFIS). The images were obtained from live animals at the Advanced Center for Technological Research in Agribusiness (CAPTA) in Sertãozinho-SP, totaling 135 images, of which 108 are Nelore and 27 Caracu. These images were expanded using data augmentation techniques, such as adding versions with reduced and increased brightness. The segmentation masks used as reference, known as *Ground Truth*, were manually created with the assistance of a specialist, consisting of binary images used to isolate specific regions within the image, serving as a reference for the training and evaluation of the models. The performance of the models was evaluated using metrics such as accuracy, Mean Absolute Error (MAE), and the Dice coefficient, which quantifies the similarity between the segmentation produced by the model and the segmentation masks created by the specialist. The results indicate that the U-Net Fuzzy model outperformed the U-Net model in measuring the REA and RFT indicators across all evaluation metrics, especially achieving accuracies of 94.17% for REA and 98.6% for RFT. In contrast, the U-Net model achieved better results for all metrics regarding the BFT indicator, with particular emphasis on the Dice coefficient metric of 70.25%. Finally, a computational application was developed capable of detecting specific regions and automatically measuring the indicators of interest in bovine ultrasound images using both models, potentially assisting in herd selection and management processes.

Keywords: Bovine ultrasonography; Image delineation; Neural networks; U-Net; ANFIS.

Lista de Figuras

1	Ilustração da localização das regiões de interesse.	15
2	Imagens de ultrassom das regiões de interesse.	16
3	Exemplo de <i>ground truth</i>	17
4	Fluxo de treinamento dos modelos.	20
1.1	Estrutura de um neurônio típico.	22
1.2	Modelo de uma unidade de McCulloch e Pitts.	23
1.3	Gráficos das Funções de Ativação.	25
1.4	Representação da taxa de aprendizagem.	28
1.5	Estrutura visual básica de uma unidade e uma rede neural com duas camadas ocultas.	31
1.6	Ilustração do processo de <i>backpropagation</i> em uma rede neural.	33
1.7	Comparação entre os métodos de descida do gradiente.	34
1.8	Ilustração do <i>overfitting</i>	35
1.9	Ilustração do <i>Dropout</i> removendo 60% das unidades aleatoriamente durante uma época de treinamento.	37
1.10	Ilustração do <i>Early Stopping</i>	37
1.11	Gráfico da derivada da função sigmoid logística.	39
1.12	Arquiteturas de Redes Neurais.	41
1.13	Fluxo de dados durante o processo de treinamento e avaliação da rede neural.	42
1.14	Exemplos de dígitos manuscritos do conjunto de dados MNIST.	42
1.15	Arquitetura da rede neural para classificação de dígitos manuscritos.	43
1.16	Paraboloide hiperbólico.	44
1.17	Gráfico da superfície e aproximação da superfície obtida pela MLP e a projeção do E no plano xy	45
1.18	Exemplo de uma imagem/matriz $10 \times 10 \times 1$ em escala de cinza, em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco).	47
1.19	Ilustração da operação de convolução discreta. O kernel w é deslizado sobre o vetor x , e em cada posição, a soma ponderada dos valores de x cobertos pelo kernel é calculada para gerar o valor correspondente na saída.	50

1.20	Exemplo de convolução bidimensional. A matriz de entrada X (matriz 7×7) é convoluída com o <i>kernel</i> W (matriz 2×2) para produzir a saída S (matriz 6×6). O <i>kernel</i> é “deslizado” sobre a matriz de entrada, formando sub-matrizes, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo <i>kernel</i> é calculada e o resultado é aplicado em uma função de ativação (σ) para gerar o valor correspondente ($\hat{y}_{11}$) na matriz de saída. b sendo o viés adicionado à soma ponderada.	53
1.21	Aplicações de diferentes <i>kernels</i> em uma imagem. A imagem original é representada em (a), juntamente com o <i>kernel</i> identidade, que não faz modificações. Em (b) a imagem passou por um <i>kernel</i> que aumenta a nitidez reforçando o pixel central e aumentando a diferença dele em relação à sua vizinhança. Em (c) foi usado um <i>kernel</i> de desfoque, que substitui um pixel pela média dele com seus vizinhos. A imagem em (d) é o resultado de aplicar na imagem em escala de cinza um <i>kernel</i> que reforça suas linhas verticais, enquanto (e) reforça suas linhas horizontais. O <i>kernel</i> em (f) reforça os contornos (bordas) da imagem. . .	53
1.22	Aplicação de <i>padding</i> em uma imagem. Em (a) a imagem original passou por zero <i>padding</i> . Em (b) a imagem original passou por <i>reflection padding</i> . A imagem em (c) passou por um <i>padding</i> constante, semelhante em (a), mas com valor diferente de zero.	55
1.23	Exemplo de <i>Max pooling</i> com dimensões 2×2 . A imagem original é reduzida para a imagem menor após a aplicação do <i>Max pooling</i> . Cada bloco de 2×2 pixels na imagem original é substituído pelo valor máximo dentro desse bloco na imagem reduzida.	56
1.24	Arquitetura típica de uma Rede Neural Convolucional com uma camada de convolução e uma camada de <i>pooling</i> , em que é enviado para rede a imagem de um cachorro e é feita a classificação do objeto na imagem.	58
1.25	Exemplo da operação de convolução transposta.	59
2.1	Função de pertinência $\mu_A(x)$ para o conjunto fuzzy “temperatura alta”.	61
2.2	Operações entre conjuntos fuzzy.	63
2.3	Função de pertinência de um número fuzzy gaussiano.	66
2.4	Arquitetura de um Sistema Baseado em Regras Fuzzy (SBRF).	68
2.5	Exemplo de inferência fuzzy utilizando o método de Takagi-Sugeno.	69
2.6	Arquitetura do ANFIS	74
3.1	Imagem de entrada X e <i>ground truth</i>	76
3.2	Aplicação de um <i>kernel</i> de convolução na imagem de entrada X	77
3.3	Aplicação de <i>padding</i> na imagem de entrada X	77
3.4	Aplicação da função de ativação ReLU no mapa de características.	78
3.5	Aplicação de uma camada de <i>Max pooling</i> no mapa de características resultante após a ReLU.	78

3.6	Aplicação da convolução transposta no mapa de <i>pooling</i>	79
3.7	Estrutura simplificada em forma de U da arquitetura U-Net.	80
3.8	Exemplo de conexões de atalho (<i>skip connections</i>) na arquitetura U-Net.	81
3.9	Exemplo da compatibilização de dimensões para a concatenação.	81
3.10	Exemplo de concatenação de dois mapas de características ao longo do eixo dos canais.	82
3.11	Aplicação da última camada de convolução para gerar a saída da rede.	82
3.12	Aplicação de um limiar (<i>threshold</i>) para gerar a máscara de segmentação.	83
3.13	Arquitetura completa da U-Net.	85
3.14	Arquitetura da U-Net multitarefa utilizada e as dimensões das imagens ao longo da rede.	88
3.15	Arquitetura da U-Net com a adição de uma camada Multitarefa e uma camada de inferência fuzzy.	91
4.1	Região do músculo <i>Longuissimus Dorsi</i> (em amarelo) e da garupa (em azul). . .	96
4.2	Aquisição da imagem de ultrassom da região do olho de lombo.	97
4.3	Demarcação manual da Área de Olho de Lombo (AOL) utilizando o aplicativo <i>ImageJ</i>	98
4.4	Demarcação manual da Espessura de Gordura subcutânea no Lombo (EGL) utilizando o aplicativo <i>ImageJ</i>	99
4.5	Demarcação manual da Espessura de Gordura subcutânea na Garupa.	99
4.6	Exemplo do processo de pré-processamento das imagens de ultrassom.	101
4.7	Exemplo das máscaras de segmentação criadas para as imagens de ultrassom. . .	102
4.8	Reta de regressão linear e R^2 entre as medidas preditas e as medidas reais da AOL e gráficos de custo, <i>Dice</i> e acurácia ao longo das épocas.	106
4.9	R^2 entre as medidas preditas e as medidas reais da AOL e gráficos de custo, <i>Dice</i> e acurácia ao longo das épocas para o modelo de U-Net Fuzzy da AOL.	108
4.10	Comparação entre as máscaras de segmentação.	109
4.11	Reta de regressão linear e R^2 entre as medidas preditas e as medidas reais da EGL e gráficos de custo, <i>Dice</i> e acurácia ao longo das épocas.	111
4.12	Reta de regressão linear e R^2 entre as medidas preditas e as medidas reais da EGL e gráficos de custo, <i>Dice</i> e acurácia ao longo das épocas.	112
4.13	Comparação entre as máscaras de segmentação.	114
4.14	Reta de regressão linear e R^2 entre as medidas preditas e as medidas reais da EGG para o modelo de U-Net multitarefa.	115
4.15	Reta de regressão linear e R^2 entre as medidas preditas e as medidas reais da EGG para o modelo de U-Net Fuzzy.	116
4.16	Comparação entre as máscaras de segmentação para a EGG.	117
4.17	Processo de extração de características e geração do mapa de características na primeira etapa de convolução da U-Net.	118
4.18	Mapa de características na última etapa de convolução do <i>baseline</i> da U-Net. . .	119

4.19	Mapa de características gerado com o módulo de inferência fuzzy e mapa de características gerado sem o módulo de inferência fuzzy.	120
4.20	Mapa de características gerado no terceiro bloco do <i>decoder</i> da U-Net Fuzzy e da U-Net multitarefa.	121
4.21	Tela inicial do aplicativo para seleção do indicador e modelo.	122
4.22	Tela de seleção das imagens de ultrassom a serem processadas.	123
4.23	Tela de visualização dos resultados, com as máscaras de segmentação sobrepostas à imagem original e os valores numéricos do indicador.	124
4.24	Exemplo de uma imagem de ultrassom com baixa qualidade, em que a rede gerou uma segmentação errônea, e o aplicativo calculou apenas a região maior, eliminando as regiões menores.	125

Lista de Tabelas

1.1	Banco de dados com as avaliações dos colegas e a decisão final de compra.	26
4.1	Quantidade de modelos desenvolvidos para cada medida.	103
4.2	Resumo da configuração de treinamento adotada nos experimentos.	104
4.3	Resumo dos resultados obtidos para a AOL, comparando o modelo de U-Net multitarefa com o modelo de U-Net Fuzzy.	109
4.4	Resumo dos resultados obtidos para a EGL, comparando o modelo de U-Net multitarefa com o modelo de U-Net Fuzzy.	113
4.5	Resumo dos resultados obtidos para a EGG, comparando o modelo de U-Net multitarefa com o modelo de U-Net Fuzzy.	116

Sumário

Dedicatória	3
Agradecimentos	4
Resumo	5
Abstract	6
Lista de Figuras	10
Lista de Tabelas	11
Introdução	15
1 Redes Neurais	21
1.1 Neurônio Biológico e Artificial	21
1.1.1 Neurônio Biológico	22
1.1.2 Unidade	23
1.1.3 Tipos de Função de Ativação	24
1.1.4 Perceptron e Algoritmo de Aprendizagem	25
1.2 Redes Neurais	30
1.2.1 <i>Backpropagation</i>	32
1.2.2 Gradiente Descendente Estocástico	33
1.2.3 <i>Overfitting</i>	34
1.2.4 Hiperparâmetros	35
1.2.5 Regularização	36
1.2.6 <i>Momentum</i> e Adam	37
1.2.7 <i>Vanishing Gradient</i>	38
1.2.8 Função de Custo <i>Binary Cross-Entropy</i>	39
1.2.9 Arquiteturas de Redes Neurais	40
1.2.10 Aplicação da Arquitetura MLP para Classificação de Dígitos Manuscritos	41
1.2.11 Aplicação da Arquitetura MLP para Reconstrução de Figura Geométrica	44
1.3 Rede Neural para Análise de Imagens	45
1.3.1 Córtex Visual Humano e Visão Computacional	45

1.3.2	Redes Neurais Convolucionais	46
1.3.3	Convolução	47
1.3.4	Correlação Cruzada	54
1.3.5	<i>Kernel</i> Multicamadas	54
1.3.6	<i>Padding</i>	55
1.3.7	<i>Stride</i>	55
1.3.8	<i>Pooling</i>	56
1.3.9	Convolução Transposta	57
2	Teoria dos Conjuntos Fuzzy	60
2.1	Conjuntos Fuzzy	60
2.1.1	Operações entre Conjuntos Fuzzy	61
2.1.2	Normas Triangulares	63
2.1.3	Níveis de um Conjunto Fuzzy	64
2.1.4	Números Fuzzy	65
2.2	Sistema Baseado em Regras Fuzzy	66
2.2.1	Regras e Inferência Fuzzy	67
2.2.2	Componentes de um SBRF	67
2.3	ANFIS	72
3	Estrutura das Redes Neurais U-Net e U-Net Fuzzy	75
3.1	Noções Básicas da Arquitetura U-Net	75
3.1.1	Arquitetura Original da U-Net	83
3.2	Modelos Propostos	85
3.2.1	Visão Geral das Modificações Propostas	85
3.3	Critérios de Avaliação	92
3.3.1	Coefficiente de Dice	92
3.3.2	Acurácia	92
3.3.3	Função de Custo	93
3.3.4	Erro Médio Absoluto	93
3.3.5	Coefficiente de Determinação e Reta de Regressão Linear	93
4	Aplicação da U-Net e U-Net Fuzzy em Ultrassonografia Bovina	95
4.1	Base de Dados e Planejamento Experimental	95
4.1.1	Contexto do Problema e Medidas Avaliadas	95
4.1.2	Conjunto de Dados	100
4.1.3	Pré-processamento e Máscaras de Segmentação	100
4.2	Configuração dos Modelos	102
4.2.1	Modelos Comparados	102
4.2.2	Configuração de Treinamento	103
4.2.3	Métricas de Avaliação	104
4.2.4	Ambiente Computacional	105

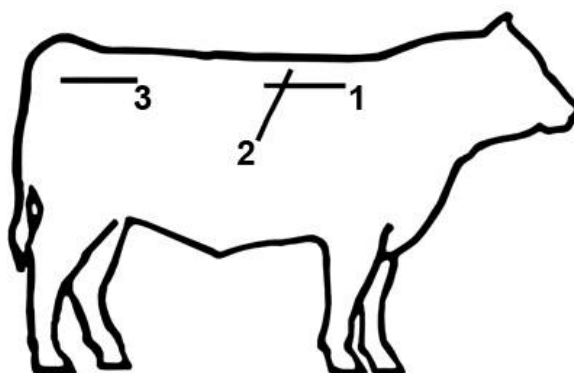
4.3	Resultados e Análise	105
4.3.1	AOL	105
4.3.2	EGL	110
4.3.3	EGG	114
4.4	Análise Geral dos Resultados	118
4.5	Aplicativo Computacional	122
5	Considerações Finais	126
	Referências Bibliográficas	131

Introdução

O desenvolvimento econômico e social do Brasil esteve profundamente enraizado em atividades agrícolas e pecuárias (GARCAO, 2015). De acordo com a Associação Brasileira das Indústrias Exportadoras de Carnes (ABIEC), o complexo agroindustrial voltado à produção de carne bovina representa cerca de 8,4% do Produto Interno Bruto (PIB) nacional (ABIEC, 2021). Além disso, o Brasil possui o maior rebanho bovino comercial do mundo, em 2021 correspondia a 11,6% do total mundial, e ocupava a primeira posição no comércio exterior de carne bovina, sendo responsável por aproximadamente 21% das exportações mundiais (GUARALDO, 2021). Atualmente, ainda em primeira lugar, é responsável por aproximadamente 30% das exportações mundiais em abril de 2026 (UNITED STATES DEPARTMENT OF AGRICULTURE, 2026).

Nesse cenário, a avaliação precisa das características da carcaça dos animais desempenha papel crucial para a cadeia produtiva, pois permite determinar indicadores relacionados ao desenvolvimento dos músculos, acúmulo de gordura na carcaça e qualidade da carne (WILSON, 1998). Entre os métodos disponíveis para esta avaliação, a ultrassonografia se destaca por possibilitar o exame *in vivo*, de forma rápida, com boa precisão e custos relativamente baixos (SUGUISAWA; MATOS; SUGUISAWA, 2013). Por meio de imagens de ultrassom, é possível mensurar características como a Área de Olho de Lombo (AOL) e a Espessura de Gordura subcutânea no Lombo (EGL), localizadas no músculo *longissimus dorsi* (Contra-filé) entre a 12ª e a 13ª e a Espessura de Gordura subcutânea na Garupa (EGG), medida no começo do músculo *Biceps Femolis* (ponta da picanha). Na Figura 1 é apresentada uma ilustração da localização das medidas no animal.

Figura 1: Ilustração da localização das regiões de interesse.

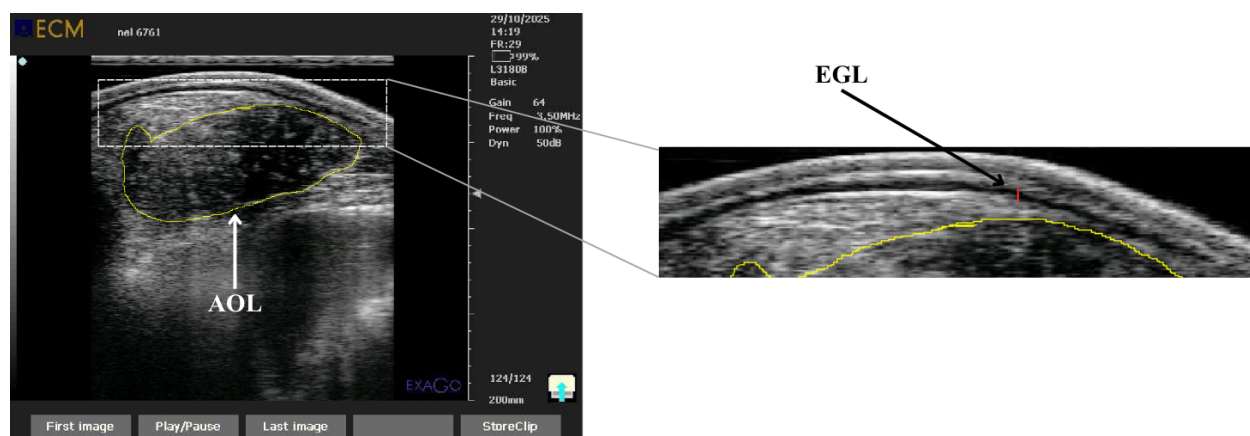


Fonte: Adaptação de (WILSON, 1998).

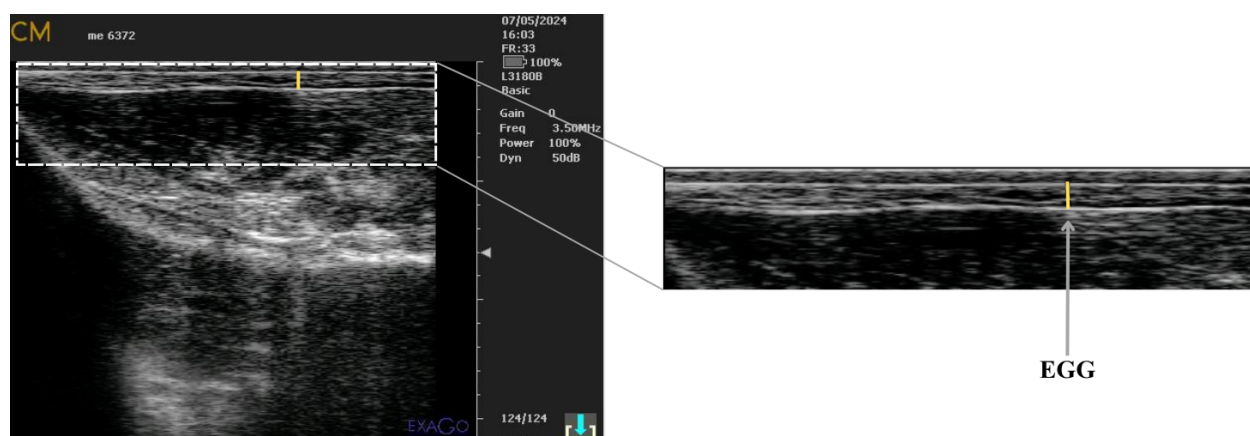
Na Figura 1, a linha horizontal (1) indica a localização da espessura de gordura subcutânea no lombo onde é possível medir a EGL, a linha inclinada (2) indica a localização do Olho do Lombo onde é possível medir a AOL e a linha horizontal (3) indica a região de medição da EGG.

As imagens de ultrassom correspondentes a essas regiões são apresentadas na Figura 2, as quais foram fornecidas pelo Centro Avançado de Pesquisa e Desenvolvimento de Bovinos de Corte, Instituto de Zootecnia, SAA-SA, localizado em Sertãozinho - SP.

Figura 2: Imagens de ultrassom das regiões de interesse.



(a) AOL e EGL.



(b) EGG.

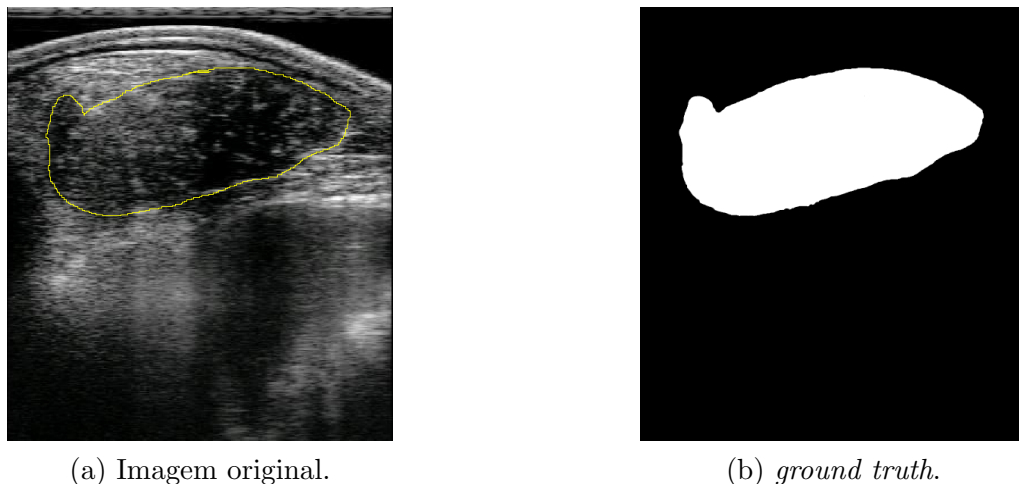
Fonte: Elaboração do Autor.

A delimitação do contorno da região em amarelo na Figura 2(a) corresponde à AOL, na mesma imagem encontra-se a EGL, demarcada em vermelho. A linha amarela na Figura 2(b) corresponde à EGG. A variabilidade nessas características influencia a comercialização e os resultados econômicos dos produtores (FELICIO, 2010), tornando a mensuração precisa uma demanda constante do setor.

Tradicionalmente, a delimitação e mensuração das regiões são realizadas manualmente por especialistas, um processo sujeito à subjetividade e que demanda tempo considerável. Diante disso, a aplicação de técnicas de visão computacional e inteligência artificial (SZELISKI, 2010) apresenta-se como uma alternativa promissora para automatizar e padronizar esse pro-

cesso. Em particular, arquiteturas de redes neurais convolucionais, em inglês *Convolutional Neural Networks* (CNNs) (O'SHEA; NASH, 2015), uma subárea da inteligência artificial dedicada à análise e interpretação de imagens, têm demonstrado resultados expressivos em diversas aplicações biomédicas, tal como, a análise de estruturas neuronais (RONNEBERGER; FISCHER; BROX, 2015). Uma das principais técnicas empregadas nestas redes é a segmentação de imagens, que consiste na classificação de cada pixel em uma categoria específica, possibilitando a identificação precisa das regiões de interesse. Para o treinamento e a avaliação desses modelos, utiliza-se uma máscara de segmentação, comumente denominada *ground truth*, criada manualmente por especialistas. Essa máscara corresponde a uma imagem binária na qual os pixels pertencentes à região de interesse são destacados em branco, enquanto os demais são representados em preto. Na Figura 3, é apresentada uma imagem de ultrassom e o *ground truth* criado para mensurar a AOL.

Figura 3: Exemplo de *ground truth*.



Fonte: Elaboração do Autor.

Na imagem original, Figura 3(a), a região de interesse não está demarcada, enquanto na Figura 3(b), pode-se visualizar o *ground truth* correspondente da Figura 3(a).

Entre as arquiteturas de redes neurais utilizadas para segmentação de imagens, destaca-se a U-Net. Seu nome deriva do formato de sua estrutura, que se assemelha à letra “U” (RONNEBERGER; FISCHER; BROX, 2015). Este modelo reduz a resolução da imagem recebida extraindo características, desde detalhes locais até padrões gerais, etapa conhecida como *encoder*, e, em seguida, a imagem é reconstruída a partir dessas características extraídas, etapa conhecida como *decoder*. Este processo permite a geração de uma máscara de segmentação predita pelo modelo. Essa máscara é posteriormente comparada ao *ground truth*, utilizando métricas de similaridade, como o coeficiente Dice, cuja descrição será apresentada adiante.

A U-Net foi originalmente proposta para segmentação de imagens biomédicas, sendo reconhecida pela capacidade de operar com conjuntos de dados reduzidos e pela eficiência na captura de informações em imagens. Quando é necessário que a arquitetura faça algo além da máscara de segmentação, como por exemplo, classificação, utiliza-se uma abordagem denomi-

nada Multitarefa, do inglês *Multitask*. Essa abordagem permite que a rede neural aprenda a realizar múltiplas tarefas simultaneamente.

Paralelamente, a teoria dos conjuntos fuzzy (ZADEH, 1965) oferece ferramentas para o tratamento de incertezas e imprecisões inerentes a imagens de ultrassom, onde as fronteiras entre as estruturas anatômicas nem sempre são claramente definidas. A integração de redes neurais com a teoria dos conjuntos fuzzy (PURWONO *et al.*, 2025), por meio do Sistema de Inferência Neuro-Fuzzy Adaptativo, do inglês *Adaptive Neuro-Fuzzy Inference System* (ANFIS), possibilita combinar a capacidade de aprendizado das redes neurais com a flexibilidade de um Sistema Baseado em Regras Fuzzy (SBRF) no tratamento de informações imprecisas. Alguns estudos têm explorado a aplicação da U-Net em tarefas de segmentação de imagens de ultrassom da região de olho de lombo (MELO *et al.*, 2022), como também a junção da U-Net com integração ao ANFIS para reconhecimento de imagens microscópicas de células (KIRICHEV; SLAVOV; MOMCHEVA, 2021).

Neste contexto, este trabalho tem como objetivos gerais:

- Desenvolver e comparar dois modelos para a delimitação automática das regiões de interesse e mensuração dos indicadores AOL, EGL e EGG em imagens de ultrassom de carcaça de bovinos das raças Nelore e Caracu.
- Comparar as medidas dos indicadores obtidas pelo modelo U-Net com abordagem multitarefa e o modelo U-Net Fuzzy, que é uma U-Net com abordagem multitarefa integrada ao ANFIS.

Para alcançar os objetivos gerais, foram estabelecidos os seguintes objetivos específicos:

- Compreender os conceitos fundamentais de redes neurais, teoria dos conjuntos fuzzy, sistemas baseados em regras fuzzy e apresentar a arquitetura U-Net e sua integração com o ANFIS para segmentação de imagens;
- Padronizar o banco de dados das imagens de ultrassom e desenvolver as respectivas máscaras de segmentação;
- Aplicar técnicas de pré-processamento e aumento de dados para adequar o banco de dados ao treinamento dos modelos;
- Desenvolver e treinar os modelos propostos utilizando as bibliotecas de aprendizagem de máquina disponíveis na linguagem de programação Python versão 3.11.15 (PYTHON SOFTWARE FOUNDATION, 1990), TensorFlow versão 2.21.0 com suporte a GPU (GOOGLE LLC, 2015) e Keras versão 3.13.2 (KERAS TEAM, 2015);
- Produzir um aplicativo computacional utilizando a biblioteca *Tkinter* (PYTHON SOFTWARE FOUNDATION, 1991) para automatizar a delimitação das regiões de interesse e a mensuração dos indicadores AOL, EGL e EGG em imagens de ultrassom.

A comparação entre os modelos visa avaliar a eficácia de cada um na predição das máscaras de segmentação e com isso na obtenção das medidas de AOL, EGL e EGG, contribuindo para a automação do processo de avaliação de características da carcaça em bovinos.

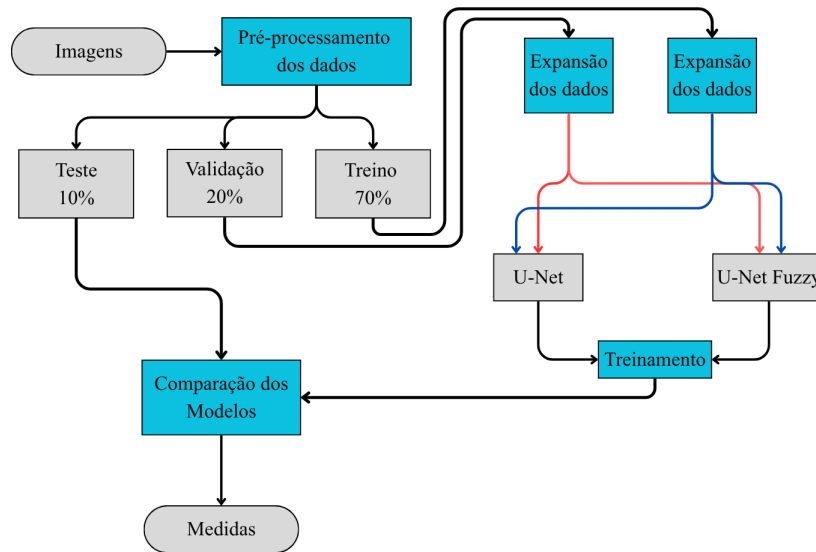
No trabalho foram utilizadas 135 imagens de ultrassom bovinas, das quais 108 são Nelore e 27 Caracu, todos machos com idade média de 12 meses. As imagens foram obtidas por meio do aparelho de ultrassom Aloka SSD-500, aparelho equipado com uma sonda linear de 3,5 MHz (Aloka Co. Ltd., 1990), utilizado por um especialista. Por conta do número limitado de imagens disponíveis, foi necessário realizar um processo de aumento de dados, do inglês *data augmentation*, para ampliar a quantidade de imagens. O processo de aumento de dados incluiu técnicas de rotação e adição de ruído às imagens originais por meio do aumento e da diminuição do brilho, simulando variações que podem ocorrer durante a aquisição das imagens de ultrassom. Para avaliar o desempenho dos modelos propostos, utilizam-se quatro métricas:

- O coeficiente de Dice, que quantifica a similaridade entre a máscara de segmentação desenvolvida pelo especialista e a imagem segmentada gerada pelo modelo (DICE, 1945).
- Erro Médio Absoluto, do inglês *Mean Absolute Error* (MAE), compara as medidas obtidas pelo especialista com as medidas extraídas da imagem segmentada gerada pelo modelo (MELO *et al.*, 2022).
- A Acurácia, que avalia a capacidade do modelo em classificar corretamente as regiões de interesse (MELO *et al.*, 2022).
- Regressão linear e Coeficiente de Determinação (R^2), usados para avaliar a relação entre as medidas obtidas pelo especialista e as medidas extraídas da imagem segmentada pelo modelo (MELO *et al.*, 2022).

Para cada um dos indicadores mensurados, ou seja, AOL, EGL e EGG, são desenvolvidos dois modelos, uma U-Net e outra U-Net Fuzzy. Para avaliar a consistência dos resultados, cada modelo foi treinado quatro vezes. Na Figura 4 é apresentado o fluxo de treinamento dos modelos desenvolvidos, assim como a divisão do banco de dados em conjuntos de treinamento(70%), validação(20%) e teste(10%). Neste processo de treinamento dos modelos, o banco de imagens é pré-processado e dividido em conjuntos de treinamento, validação e teste. Após a divisão, os conjuntos de treinamento e validação são expandidos, ou seja, *data augmentation*, e utilizados para o treinamento dos modelos, enquanto o conjunto de teste é reservado para a avaliação final do desempenho dos modelos.

Por fim, foi desenvolvido um aplicativo computacional utilizando a biblioteca disponível na linguagem de programação Python, *Tkinter*, que faz a implementação de uma interface gráfica para os modelos desenvolvidos, automatizando a delimitação da região e a mensuração dos indicadores AOL, EGL e EGG nas imagens de ultrassom, que poderá ser disponibilizado para o Centro Avançado de Pesquisa e Desenvolvimento de Bovinos de Corte, o que torna a tecnologia acessível para produtores e profissionais do setor agropecuário, podendo contribuir para a melhoria da eficiência e precisão na avaliação das características da carcaça dos bovinos.

Figura 4: Fluxo de treinamento dos modelos.



Fonte: Elaboração do Autor.

Até onde foi possível verificar na literatura, não há contribuições científicas que tenham desenvolvido modelos capazes de fazer a segmentação de imagens de ultrassom integrados ao ANFIS, U-Net Fuzzy, que mensurem os indicadores de AOL, EGL e EGG. Consequentemente, não foram encontrados trabalhos que comparem a U-Net com a U-Net Fuzzy para esses indicadores. Os únicos trabalhos encontrados sobre esse tema compararam a U-Net com outras arquiteturas de redes neurais, exclusivamente para a AOL (MELO *et al.*, 2022).

O presente trabalho está organizado da seguinte forma. No Capítulo 1, são apresentados os conceitos fundamentais de redes neurais, desde a inspiração biológica do neurônio artificial até as arquiteturas de redes neurais convolucionais. No Capítulo 2 é introduzida a teoria dos conjuntos fuzzy e os sistemas baseados em regras fuzzy, que fornecem a base teórica para a construção do ANFIS. No Capítulo 3 é descrito detalhadamente a implementação dos modelos propostos, incluindo a arquitetura da U-Net, a integração do ANFIS e o processo de treinamento. No Capítulo 4 são apresentados a organização do conjunto de dados, o treinamento e os resultados obtidos, com uma análise comparativa entre os modelos e o aplicativo desenvolvido. Por fim, no Capítulo 5 é apresentada as considerações finais, destacando as contribuições e sugestões para pesquisas futuras.

Capítulo 1

Redes Neurais

“A sorte é uma função matemática com parâmetros desconhecidos.” (João N. S. Filho)

A ideia de máquinas inteligentes desperta fascínio e receio há décadas. No imaginário popular, esse medo costuma aparecer em obras de ficção científica, como *O Exterminador do Futuro*, em que sistemas artificiais ultrapassam o controle humano e passam a agir de forma autônoma. Embora esse cenário ainda pertença ao campo da ficção, ele revela uma inquietação real: até que ponto uma máquina pode aprender, decidir e interpretar o mundo? Na prática, os modelos atuais estão muito distantes de uma inteligência artificial consciente, mas representam o estágio mais avançado já alcançado pelo ser humano na construção de sistemas capazes de reconhecer padrões, extrair informações e auxiliar na resolução de problemas complexos. Nesse contexto, compreender os fundamentos das redes neurais torna-se essencial para entender como essas tecnologias funcionam e por que elas têm alcançado resultados tão expressivos em tarefas de geração de texto, classificação de objetos e análise de imagens.

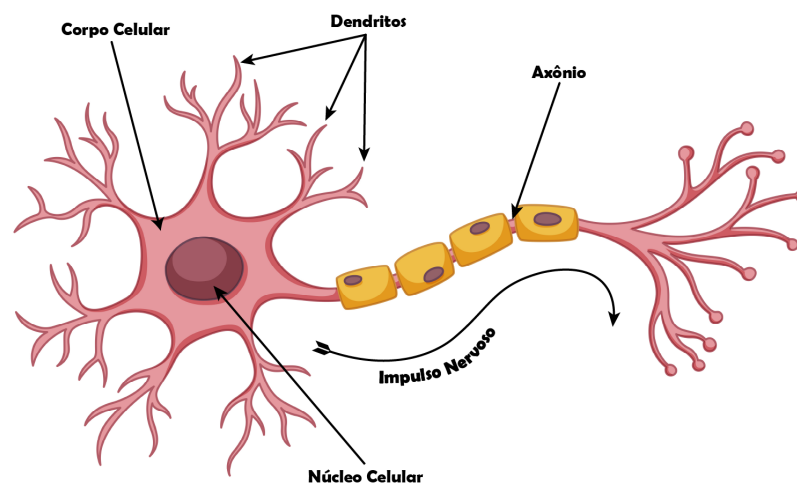
1.1 Neurônio Biológico e Artificial

Na área de Inteligência Artificial, comumente utiliza-se o termo *neurônio artificial* para se referir a uma unidade de processamento em um modelo de rede neural. O neurônio artificial é inspirado no funcionamento do neurônio biológico, que é a unidade básica do sistema nervoso. Assim como os neurônios biológicos, os artificiais recebem entradas, processam essas entradas e produzem uma saída. A principal função de um neurônio artificial é realizar uma combinação linear das entradas, seguida por uma função de ativação não linear, que determina a saída do neurônio. A semelhança entre esses dois objetos (Neurônio artificial/Neurônio Biológico) só está na inspiração, sendo bem diferentes em seu funcionamento, com isso, neste trabalho utiliza-se o termo *unidade* para se referir ao neurônio artificial (O’SHEA; NASH, 2015). Os conceitos apresentados a seguir são fundamentais para o entendimento do funcionamento das redes neurais, que são abordadas na Seção 1.2.

1.1.1 Neurônio Biológico

Para entender o funcionamento da unidade é necessário entender o funcionamento do neurônio biológico. Neurônios são “células excitáveis” que se comunicam ao transmitir impulsos elétricos. Células *excitáveis* são definidas como capazes de produzir sinais elétricos amplos e rápidos denominados *potências de ação* (veja (STANFIELD, 2014)). A maioria dos neurônios contém três componentes principais: um corpo celular, o(s) dendrito(s) e um axônio. O corpo celular, chamado de *soma* contém o núcleo celular, local onde parte da informação recebida de outras partes do corpo é manipulada. Os *dendritos* se ramificam a partir do corpo celular, são responsáveis por receber e integrar uma imensa quantidade de sinais provenientes de outros neurônios. Outra ramificação advinda do corpo celular é o *axônio*. Diferente dos dendritos, cuja função é *receber* informação, um axônio *envia* informações. Grande parte dos neurônios tem apenas um axônio, mas os axônios conseguem se ramificar, enviando, assim, sinais a mais de um destino. Esses três componentes, de forma resumida, formam o neurônio biológico, como mostrado na Figura 1.1.

Figura 1.1: Estrutura de um neurônio típico.



Fonte: Adaptado de (STANFIELD, 2014).

Outro fator importante em um neurônio biológico são os potenciais de ação, gerados nos dendritos, são avaliados e ponderados por meio da *força sináptica*, que diz o quão importante é a informação (BEAR; CONNORS; PARADISO, 2017). Após esse processo, os potenciais de ação são enviados para o núcleo celular. Entre o corpo celular e o axônio será decidido se o potencial é excitatório ou inibitório. A decisão é feita caso o potencial de ação supere ou não um *limiar*. Caso seja excitatório, a informação é passada a diante, caso seja inibitório, a informação é suprimida.

1.1.2 Unidade

A partir das informações obtidas com o estudo do neurônio biológico, pesquisadores tentaram simular seu funcionamento em computador. A implementação mais bem aceita de simulação foi proposta por dois pesquisadores, o neurobiologista Warren McCulloch e o matemático Walter Pitts em 1943 que utilizaram da lógica proposicional para descrever o funcionamento do sistema nervoso central (MCCULLOCH; PITTS, 1943). McCulloch acreditava que o neurônio biológico tinha uma natureza binária, ou seja, um neurônio influencia outro ou não, entretanto, posteriormente essa crença mostrou-se errônea com os avanços de estudos na área.

As variáveis $x_1, x_2, x_3, \dots, x_n$ são denominadas *dados de entrada*, em analogia às sinapses de um neurônio biológico. Os valores $w_1, w_2, w_3, \dots, w_n$ são os *pesos* associados a cada dado de entrada. O viés tem a mesma funcionalidade do limiar no neurônio biológico, determinando se o neurônio será ativado ou não. Para resumir a notação, defini-se o viés de b , do inglês *bias*. Na soma, a variável z é obtida fazendo a combinação linear das variáveis:

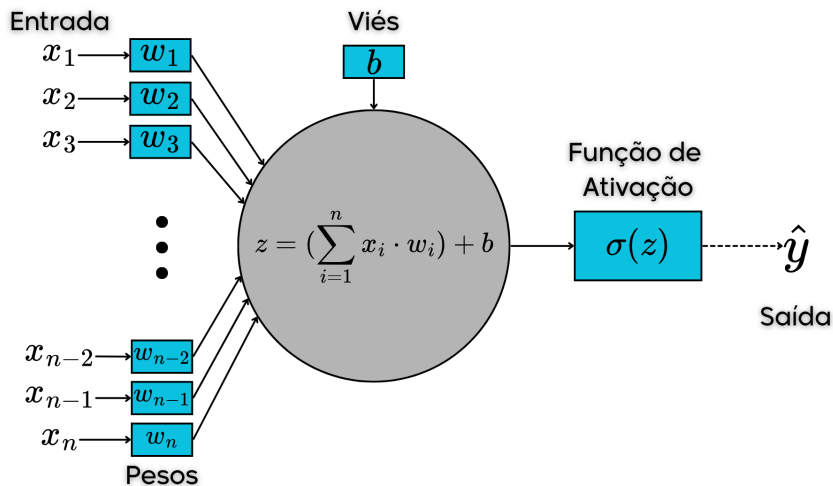
$$x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 + \dots + x_n \cdot w_n + b = z = \sum_{i=1}^n x_i \cdot w_i + b.$$

Em seguida, o valor z é aplicado a uma *função de ativação* $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, resultando na saída $\hat{y}$, ou seja:

$$\hat{y} = \sigma \left(\sum_{i=1}^n w_i \cdot x_i + b \right). \quad (1.1)$$

Na Figura 1.2 é possível visualizar o modelo de uma unidade de McCulloch e Pitts.

Figura 1.2: Modelo de uma unidade de McCulloch e Pitts.



Fonte: Elaboração do Autor.

Em que $x_1, x_2, x_3, \dots, x_n$ são os dados de entrada, $w_1, w_2, w_3, \dots, w_n$ são os pesos associados a cada dado de entrada e b é o viés. O valor z é a combinação linear das variáveis e $\hat{y}$ é a saída da unidade após a aplicação da função de ativação σ .

1.1.3 Tipos de Função de Ativação

A função de ativação, representada por σ , define a saída de uma unidade, pode-se definir diferentes tipos de funções de ativação, dependendo do comportamento desejado para a unidade. As funções de ativação mais comuns incluem:

- Função de Limiar: Para este tipo de função de ativação, descrito na equação (1.2), a unidade é ativada, ou seja, produz uma saída de 1, se a soma ponderada das entradas mais o viés for maior ou igual a zero. Caso contrário, a saída é 0, definida por:

$$\sigma(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}. \quad (1.2)$$

- Função Sigmoidal: A função sigmoidal, cujo o gráfico tem a forma de um “S”, é definida como uma função estritamente crescente que exibe um balanceamento adequado entre comportamento linear e não linear. Um exemplo de função sigmoidal é a *sigmoid* logística, que foi amplamente usada no início das pesquisas em redes neurais pois a resposta do neurônio biológico é aproximadamente similar a função sigmoidal. Isso ficou claro em estudos feitos nos neurônios dos olhos de animais, neste caso, sapos (LETTVIN *et al.*, 1959). O principal motivo para o seu desuso foi a dificuldade de treinamento em redes neurais profundas devido ao problema do *vanish gradient*, visto na Subseção 1.2.7. A função é definida por:

$$\sigma(z) = \frac{1}{1 + \exp(-\alpha z)}, \quad (1.3)$$

em que α é um parâmetro que controla a inclinação da curva. A função sigmoid logística tem um conjunto de propriedades que se mostram úteis nos cálculos relacionados à aprendizagem dos pesos:

1. Não linear;
2. Contínua e diferenciável em todo o domínio de $\mathbb{R}$;
3. A derivada tem forma simples e é expressa pela própria função:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z));$$

4. É estritamente monótona, ou seja, $z_1 \leq z_2 \iff \sigma(z_1) \leq \sigma(z_2)$.

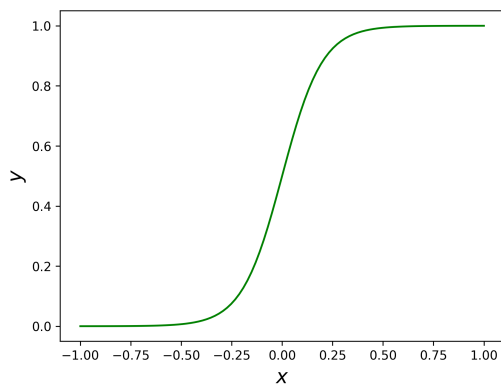
- Função *Rectified Linear Unit* (ReLU): A função ReLU, em português, Unidade Linear Retificada, é definida como $\sigma(z) = \max(0, z)$. Esta é amplamente utilizada em redes neurais profundas, estudadas na Seção 1.2, devido à sua simplicidade computacional. A função ReLU ativa a unidade apenas quando a soma ponderada das entradas mais o viés

é positiva, caso contrário, a saída é zero, ou seja:

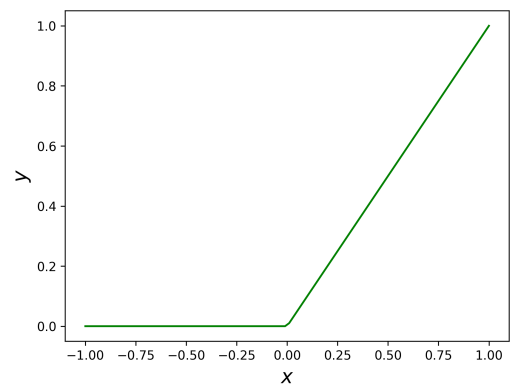
$$\sigma(z) = \begin{cases} z, & \text{se } z > 0 \\ 0, & \text{se } z \leq 0 \end{cases}. \quad (1.4)$$

Em geral, o objetivo da função de ativação é limitar a saída de uma unidade para um intervalo de valores. Na Figura 1.3(a) é mostrado o gráfico da função *sigmoid* logística, e na Figura 1.3(b) é mostrado o gráfico da função *Rectified Linear Unit*.

Figura 1.3: Gráficos das Funções de Ativação.



(a) Função *sigmoid*.



(b) Função ReLU.

Fonte: Elaboração do Autor.

A principal desvantagem na unidade de McCulloch e Pitts é a ausência de um algoritmo de aprendizagem, ou seja, uma maneira de ajustar os pesos e o viés da unidade para que este seja capaz de resolver um problema específico. Essa limitação foi superada em 1958 por Frank Rosenblatt, que propôs o *Perceptron* (ROSENBLATT, 1958), um modelo de unidade com um algoritmo de aprendizagem capaz de ajustar os pesos e o viés da unidade.

1.1.4 Perceptron e Algoritmo de Aprendizagem

O Perceptron é um modelo de unidade que utiliza um algoritmo de aprendizagem supervisionada para ajustar seus pesos e viés com base em um conjunto de dados “rotulados”. Dados rotulados são conjuntos de dados de entrada associados a saídas desejadas, ou seja, para cada elemento do conjunto de entradas, há uma saída correta conhecida definida por algum especialista. O objetivo do Perceptron é encontrar uma função que mapeie corretamente as entradas para as saídas desejadas, minimizando o erro entre as previsões do modelo e os valores reais.

Para esclarecer o funcionamento do Perceptron e do algoritmo de aprendizagem, é proposto o Exemplo 1.1.

Exemplo 1.1. *Um investidor busca decidir qual ação adquirir com base nas variáveis: valor da ação, confiabilidade e volatilidade. Para isso, dispõe das avaliações de quatro colegas que também analisam essas mesmas características antes de realizar seus investimentos.*

Na Tabela 1.1 são apresentadas as avaliações dos colegas e a decisão final de compra, ou seja, o valor da ação é considerado baixo ou alto, a confiabilidade é considerada baixa ou alta, a volatilidade é considerada baixa ou alta e se comprou ou não, com 0 representando “não”, 1 representando “sim”. $x^{(1)}, \dots, x^{(4)}$ representando as respostas de cada colega e $y = (y_1, y_2, y_3, y_4)$ a decisão final de cada um.

Tabela 1.1: Banco de dados com as avaliações dos colegas e a decisão final de compra.

	valor da ação	confiabilidade	volatilidade	y
$x^{(1)}$	0	1	0	1
$x^{(2)}$	1	0	1	0
$x^{(3)}$	1	1	1	0
$x^{(4)}$	0	0	0	1

Com isso, o Perceptron terá três variáveis de entrada e como saída, a decisão sobre a compra da ação. Os pesos iniciais são definidos aleatoriamente como: $w = (w_1, w_2, w_3) = (0, 0, 0)$ e o viés como $b = 0$, utilizando a função de ativação *sigmoid*, equação (1.3), para $\alpha = 1$. Para o caso geral, com $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, x_3^{(i)})$, têm-se:

$$\hat{y}_i = \sigma(z_i), \text{ em que } z_i = \sum_{j=1}^3 x_j^{(i)} \cdot w_j + b$$

para $i = 1, \dots, 4$, em que $\hat{y}_i$ é a resposta da rede, considerando $\hat{y} = (\hat{y}_1, \hat{y}_2, \hat{y}_3, \hat{y}_4)$. Testando para o primeiro dado de entrada, ou seja, $x^{(1)} = (0, 1, 0)$ e $y_1 = 1$:

$$\begin{cases} z_1 = \sum_{j=1}^3 x_j^{(1)} \cdot w_j + b = 0 \cdot 0 + 1 \cdot 0 + 0 \cdot 0 + 0 = 0 \\ \text{então } \hat{y}_1 = \sigma(0) = 0,5 \end{cases}$$

A saída da função de ativação pode ser interpretada como a probabilidade de compra da ação, ou seja, a rede prevê que a probabilidade de compra da ação é de 50%. Para que essa probabilidade seja interpretada como uma decisão binária, define-se um limiar, do inglês *threshold*. Neste exemplo, adota-se *threshold* igual a 0,5 e a convenção de que a rede prevê compra somente quando $\hat{y}_i > 0,5$; caso contrário, prevê não compra. Com isso, para o primeiro dado de entrada, a rede prevê $\hat{y}_1 = 0$.

Como $y_1 = 1 \neq \hat{y}_1 = 0$, o Perceptron comete um erro e é necessário aplicar o algoritmo de aprendizagem, a unidade de McCulloch e Pitts encerraria na etapa anterior. Para isso, define-se uma função de custo, responsável por quantificar o erro cometido durante o treinamento. Neste exemplo, utiliza-se o Erro Quadrático Médio, dado por:

$$C(w, b) = \frac{\sum_{i=1}^4 (y_i - \hat{y}_i)^2}{n}, \quad (1.5)$$

com n representando a quantidade de dados presentes no conjunto de treinamento. $C(w, b)$ não

depende de w e b explicitamente, mas sim por meio de $\hat{y}_i$, ou seja:

$$\begin{cases} C(w, b) &= C(\hat{y}_i(w, b)) \\ \hat{y}_i &= \sigma(z_i) \\ z_i &= \sum_{j=1}^3 x_j^{(i)} \cdot w_j + b \end{cases}.$$

O processo de treinamento ajusta iterativamente os pesos e o viés por meio do método da Descida do Gradiente, do inglês *Gradient Descent* - GD, cujo objetivo é minimizar a função de custo. Esse método utiliza as derivadas parciais da função de custo em relação aos pesos e ao viés, o que indica a direção de maior crescimento da função. Para reduzir o erro, atualizam-se os parâmetros na direção oposta ao gradiente. As derivadas são:

$$\frac{\partial C(w, b)}{\partial w_j} = -2 \cdot \frac{\sum_{i=1}^4 (y_i - \hat{y}_i) \cdot \sigma'(z_i) \cdot x_j^{(i)}}{n}; \quad (1.6)$$

$$\frac{\partial C(w, b)}{\partial b} = -2 \cdot \frac{\sum_{i=1}^4 (y_i - \hat{y}_i) \cdot \sigma'(z_i)}{n}. \quad (1.7)$$

No Exemplo 1.1 assume-se $\sigma'(z_i) = 1$ para simplificação. Como a derivada aponta para a região de maior valor da função, multiplica-se por -1 para obter a direção da região de minimização. O método de GD é apresentado no Algoritmo 1.

Entrada: Conjunto de treinamento $\{(x^{(q)}, y_q)\}_{q=1}^n$, taxa de aprendizado η , pesos iniciais $w_1, w_2, \dots, w_k$ e viés b

Saída: Pesos e viés ajustados

Repita

Para $q \leftarrow 1$ **Até** n **Faça**

$z_q \leftarrow \sum_{m=1}^k w_m \cdot x_m^{(q)} + b;$
 $\hat{y}_q \leftarrow \sigma(z_q);$

Fim

$C(w, b) \leftarrow \frac{1}{n} \sum_{q=1}^n (y_q - \hat{y}_q)^2;$

Para $m \leftarrow 1$ **Até** k **Faça**

$\Delta(w_m) \leftarrow \frac{2}{n} \sum_{q=1}^n (y_q - \hat{y}_q) \cdot \sigma'(z_q) \cdot x_m^{(q)};$

Fim

$\Delta(b) \leftarrow \frac{2}{n} \sum_{q=1}^n (y_q - \hat{y}_q) \cdot \sigma'(z_q);$

Para $m \leftarrow 1$ **Até** k **Faça**

$w_m \leftarrow w_m + \eta \Delta(w_m);$

Fim

$b \leftarrow b + \eta \Delta(b);$

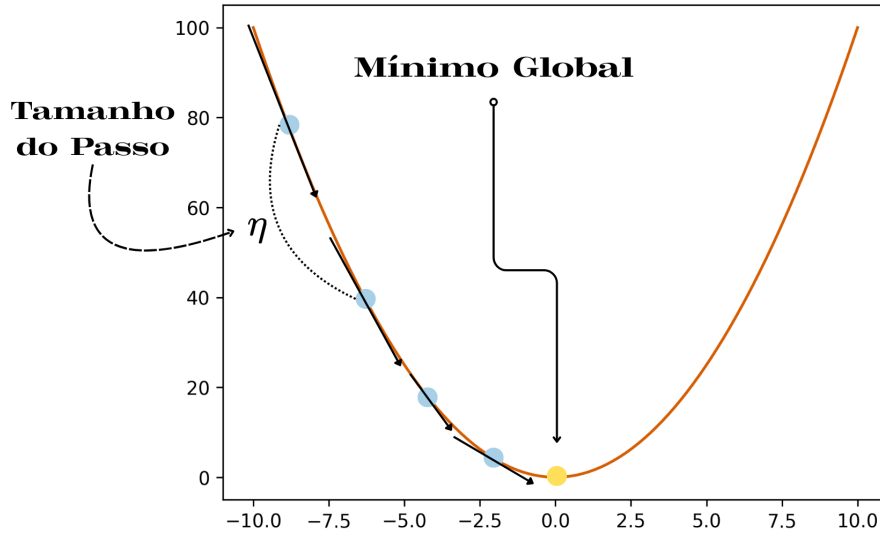
até critério de parada (convergência ou número máximo de épocas);

Retorne $w_1, w_2, \dots, w_k, b;$

Algoritmo 1: Método da Descida do Gradiente em lote

A taxa de aprendizagem (η) presente no Algoritmo 1 pode ser interpretada como o tamanho do passo dado em direção ao mínimo da função de custo, como mostrado na Figura 1.4. Quando o valor dessa taxa é muito pequeno, o processo de convergência torna-se mais lento, exigindo um número maior de iterações para atingir o mínimo. Por outro lado, se o passo for muito grande, o algoritmo pode ultrapassar o ponto de mínimo, dificultando ou até impedindo a convergência adequada, para o Exemplo 1.1 é utilizada uma taxa de aprendizado de 0,1.

Figura 1.4: Representação da taxa de aprendizagem.



Fonte: Elaboração do Autor.

Na Figura 1.4 é apresentada uma ilustração da taxa de aprendizagem, em que o ponto amarelo representa o ponto de mínimo da função de custo, representada pela cor laranja, os pontos azuis representam o momento em que é feito o ajuste dos pesos e do viés e as setas pretas indicam a direção inversa do gradiente. Aplicando o Algoritmo 1 no Exemplo 1.1, com taxa de aprendizado $\eta = 0,1$ e assumindo $\sigma'(z_i) = 1$, atualizando os pesos e o viés três vezes, é obtido o seguinte resultado.

Na época 1 de atualização, como $w^{(1)} = (0,0,0)$ e $b^{(1)} = 0$, tem-se $z_i = 0$ para todo i , logo

$$\hat{y} = (0,5, 0,5, 0,5, 0,5).$$

Em que $w^{(1)}$ e $b^{(1)}$ são os pesos e viés iniciais atualizados na época 1. Calculando os gradientes, tem-se,

$$\Delta w_1 = 2 \cdot \left(\frac{0,5 \cdot 0 + (-0,5) \cdot 1 + (-0,5) \cdot 1 + 0,5 \cdot 0}{4} \right) = -0,5,$$

$$\Delta w_2 = 2 \cdot \left(\frac{0,5 \cdot 1 + (-0,5) \cdot 0 + (-0,5) \cdot 1 + 0,5 \cdot 0}{4} \right) = 0,$$

$$\Delta w_3 = 2 \cdot \left(\frac{0,5 \cdot 0 + (-0,5) \cdot 1 + (-0,5) \cdot 1 + 0,5 \cdot 0}{4} \right) = -0,5,$$

e

$$\Delta b = 2 \cdot \left(\frac{0,5 - 0,5 - 0,5 + 0,5}{4} \right) = 0.$$

Portanto, os novos pesos são: $w^{(1)} = w^{(0)} + 0,1(\Delta w_1, \Delta w_2, \Delta w_3) = (-0,05, 0, -0,05)$ e o novo viés é: $b^{(1)} = b^{(0)} + 0,1\Delta b = 0$.

Na época 2, com $w^{(1)} = (-0,05, 0, -0,05)$ e $b^{(1)} = 0$, obtém-se,

$$z = (0, -0,1, -0,1, 0),$$

e, conseqüentemente, $\hat{y} \approx (0,5, 0,4750, 0,4750, 0,5)$. Com isso, os gradientes são:

$$(\Delta w_1, \Delta w_2, \Delta w_3, \Delta b) \approx (-0,2438, 0,0031, -0,2438, 0,0062).$$

Após a atualização, $w^{(2)} \approx (-0,0494, 0,0003, -0,0494)$ e $b^{(2)} \approx 0,0006$.

Na época 3, usando $w^{(2)}$ e $b^{(2)}$, tem-se,

$$z \approx (0,0009, -0,0981, -0,0978, 0,0006).$$

Então, $\hat{y} \approx (0,5002, 0,4755, 0,4756, 0,5002)$. Assim, os gradientes são,

$$(\Delta w_1, \Delta w_2, \Delta w_3, \Delta b) \approx (-0,2378, 0,0060, -0,2378, 0,0121),$$

de modo que $w^{(3)} \approx (-0,0732, 0,0009, -0,0732)$ e $b^{(3)} \approx 0,0018$.

Verificando as previsões da rede após a terceira época, obtém-se:

$$x^{(1)}: (0, 1, 0) \Rightarrow z_1 \approx 0,0028 \Rightarrow \sigma(z_1) \approx 0,5007 \Rightarrow \hat{y}_1 = 1;$$

$$x^{(2)}: (1, 0, 1) \Rightarrow z_2 \approx -0,1445 \Rightarrow \sigma(z_2) \approx 0,4639 \Rightarrow \hat{y}_2 = 0;$$

$$x^{(3)}: (1, 1, 1) \Rightarrow z_3 \approx -0,1435 \Rightarrow \sigma(z_3) \approx 0,4642 \Rightarrow \hat{y}_3 = 0;$$

$$x^{(4)}: (0, 0, 0) \Rightarrow z_4 \approx 0,0018 \Rightarrow \sigma(z_4) \approx 0,5005 \Rightarrow \hat{y}_4 = 1.$$

Assim, após três épocas de atualização, os parâmetros ajustados classificam corretamente todos os exemplos do conjunto de treinamento. Para um dado de entrada fora do banco de dados, $x = (1, 0, 0)$, ou seja, o valor da ação é alto, a confiabilidade é baixa e a volatilidade é baixa, a rede prevê

$$\begin{cases} z \approx -0,0732 \cdot 1 + 0,0009 \cdot 0 - 0,0732 \cdot 0 + 0,0018 = -0,0713; \\ \sigma(z) \approx 0,4822 \Rightarrow \hat{y} = 0, \end{cases}$$

ou seja, a rede prevê que a ação não deve ser comprada.

Comumente o banco de dados é dividido em dois conjuntos: o conjunto de treinamento e o conjunto de teste. O conjunto de treinamento é utilizado para ajustar os pesos e o viés do Perceptron, enquanto o conjunto de teste é utilizado para avaliar o desempenho do modelo em

dados não vistos durante o treinamento. Essa divisão é essencial para garantir que o modelo aprenda a generalizar bem para novos dados, evitando o problema do *overfitting*, visto na Seção 1.2.3, que ocorre quando o modelo se ajusta excessivamente aos dados de treinamento, perdendo a capacidade de generalização.

1.2 Redes Neurais

Nesta seção são apresentados os conceitos relacionados às redes neurais, os quais são fundamentais para o desenvolvimento deste trabalho. Os conceitos apresentados nesta seção são utilizados na construção da rede neural proposta na Seção 3.1.

As redes neurais consistem em um “encadeamento” de múltiplas unidades, em que a saída de uma unidade é a entrada de outra. Redes neurais podem ser organizadas em camadas: uma *camada de entrada*, uma ou mais *camadas ocultas* e uma *camada de saída*. Cada camada é composta por várias unidades e cada unidade em uma camada está conectada a todas as unidades na camada seguinte. Na Figura 1.5(a) é ilustrada a estrutura básica de uma unidade vista na Seção 1.1.2. Na Figura 1.5(b) é mostrada uma rede neural com duas camadas ocultas, em que a camada de entrada tem três unidades, a primeira camada oculta tem cinco unidades, a segunda camada oculta tem quatro unidades e a camada de saída tem duas unidades. Matematicamente, é possível representar uma rede neural com uma camada de entrada com 3 unidades, duas camadas ocultas, cada camada contendo 5 e 4 unidades, respectivamente e uma saída da seguinte maneira:

$$\begin{aligned}
 \hat{y}_1^{(4)} &= \sigma_1^{(4)} \left(x_1^{(4)} \right); \\
 x_1^{(4)} &= \sum_{i^{(3)}=1}^4 w_1^{(3)}(i^{(3)}) \cdot x_{i^{(3)}}^{(3)} + b_1^{(4)}; \\
 x_{i^{(3)}}^{(3)} &= \sigma_{i^{(3)}}^{(3)} \left(\sum_{i^{(2)}=1}^5 w_{i^{(3)}}^{(2)}(i^{(2)}) \cdot x_{i^{(2)}}^{(2)} + b_{i^{(3)}}^{(3)} \right); \\
 x_{i^{(2)}}^{(2)} &= \sigma_{i^{(2)}}^{(2)} \left(\sum_{i^{(1)}=1}^3 w_{i^{(2)}}^{(1)}(i^{(1)}) \cdot x_{i^{(1)}}^{(1)} + b_{i^{(2)}}^{(2)} \right);
 \end{aligned} \tag{1.8}$$

Em que $x_{i^{(1)}}^{(1)}$ representa a $i^{(1)}$ -ésima unidade da camada de entrada, $x_{i^{(2)}}^{(2)}$ representa a $i^{(2)}$ -ésima unidade da segunda camada ou da primeira camada oculta, $x_{i^{(3)}}^{(3)}$ representa a $i^{(3)}$ -ésima unidade da segunda camada oculta e $\hat{y}_1^{(4)}$ representa a saída da rede. Os pesos e os vieses são representados por $w_{i^{(l)}}^{(l-1)}(i^{(l-1)})$ e $b_{i^{(l)}}^{(l)}$, respectivamente, em que l representa o número da camada, para $l = 2, 3, 4$. As funções de ativação são representadas por $\sigma_{i^{(l)}}^{(l)}$, para $l = 2, 3, 4$.

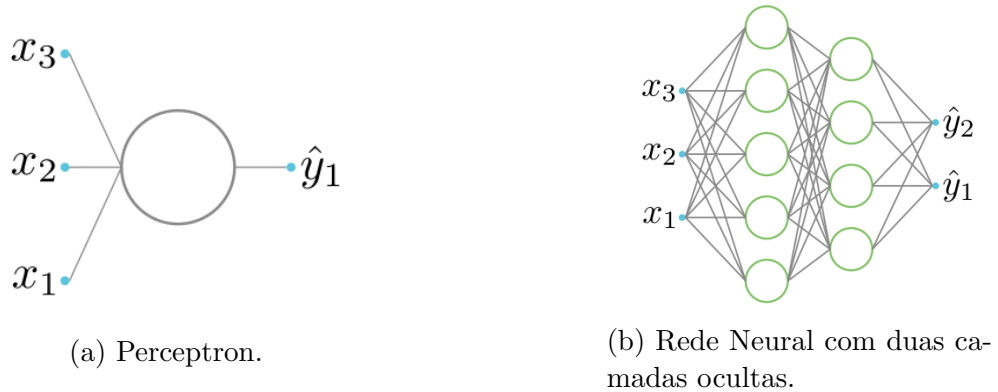
De forma geral, para uma rede neural com L camadas ocultas, a saída da rede pode ser

expressa pela seguinte equação:

$$\begin{aligned}
 \hat{y}_1^{(L)} &= \sigma_1^{(L)} \left(x_1^{(L)} \right); \\
 x_1^{(L)} &= \sum_{i^{(L-1)}=1}^{n_{L-1}} w_1^{(L-1)}(i^{(L-1)}) \cdot x_{i^{(L-1)}}^{(L-1)} + b_1^{(L)}; \\
 x_{i^{(L-1)}}^{(L-1)} &= \sigma_{i^{(L-1)}}^{(L-1)} \left(\sum_{i^{(L-2)}=1}^{n_{L-2}} w_{i^{(L-1)}}^{(L-2)}(i^{(L-2)}) \cdot x_{i^{(L-2)}}^{(L-2)} + b_{i^{(L-1)}}^{(L-1)} \right); \\
 &\vdots \\
 x_{i^{(2)}}^{(2)} &= \sigma_{i^{(2)}}^{(2)} \left(\sum_{i^{(1)}=1}^{n_1} w_{i^{(2)}}^{(1)}(i^{(1)}) \cdot x_{i^{(1)}}^{(1)} + b_{i^{(2)}}^{(2)} \right);
 \end{aligned} \tag{1.9}$$

em que n_l representa a quantidade de unidades na l -ésima camada, para $l = 1, 2, \dots, L-1$. Na Figura 1.5 é possível visualizar a estrutura visual básica de uma unidade gerando uma saída e uma rede neural com duas camadas ocultas, ilustrando a organização das unidades em camadas e as conexões entre elas.

Figura 1.5: Estrutura visual básica de uma unidade e uma rede neural com duas camadas ocultas.



Fonte: Elaboração do Autor.

Outra forma de representar uma rede neural é por meio da notação matricial, em que as entradas, os pesos e os vieses são representados por matrizes e vetores. Uma mudança de notação é feita para simplificar a representação, $w_{i^{(l+1)}}^{(l)}(i^{(l)}) = w_{i^{(l)}i^{(l+1)}}^{(l)}$. Com isso, para uma rede neural com L camadas, a saída da rede pode ser expressa pela seguinte equação matricial:

$$\begin{aligned}
 \hat{y}_1^{(L)} &= \sigma_1^{(L)} \left(x_1^{(L)} \right); \\
 x_1^{(L)} &= W_{(L \times L-1)}^{(L-1)} \odot X_{(L-1 \times 1)}^{(L-1)} + B_{(L \times 1)}^{(L)};
 \end{aligned} \tag{1.10}$$

em que $W^{(L-1)}$ é a matriz de pesos da $(L-1)$ -ésima camada, $X^{(L-1)}$ é o vetor de entradas da $(L-1)$ -ésima camada, $B^{(L)}$ é o vetor de vieses da L -ésima camada e $\odot$ representa a multiplicação

matricial, ou seja:

$$x_1^{(L)} = \underbrace{\begin{bmatrix} w_{11}^{(L-1)} & w_{12}^{(L-1)} & \cdots & w_{1(L-1)}^{(L-1)} \\ w_{21}^{(L-1)} & w_{22}^{(L-1)} & \cdots & w_{2(L-1)}^{(L-1)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{L1}^{(L-1)} & w_{L2}^{(L-1)} & \cdots & w_{L(L-1)}^{(L-1)} \end{bmatrix}}_{L \times (L-1)} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{L-1} \end{bmatrix}}_{(L-1) \times 1} + \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_L \end{bmatrix}}_{L \times 1}.$$

Um processo de iterativo é feito para calcular $X_{(L-1 \times 1)}^{(L-1)}$ semelhante ao processo apresentado na equação (1.9). Essa forma representa a saída da rede caso tenha apenas uma saída.

1.2.1 *Backpropagation*

[mudar aqui](#)

Assim como no perceptron, as redes neurais utilizam algoritmos de aprendizagem para ajustar seus pesos e vieses com o objetivo de minimizar uma função de custo. O algoritmo mais comum utilizado para esse propósito é o *backpropagation* (retropropagação) (HECHT-NIELSEN, 1992) e (WERBOS, 1990), que é uma extensão do método da descida do gradiente. O *backpropagation* é utilizado para calcular os gradientes da função de custo em relação aos pesos e vieses de todas as camadas da rede neural, permitindo a atualização eficiente desses parâmetros durante o treinamento. O algoritmo funciona propagando o erro da saída da rede de volta para as camadas anteriores, ajustando os pesos com base na contribuição de cada unidade para o erro total.

Esse processo é repetido iterativamente até que a função de custo seja minimizada, resultando em uma rede neural treinada capaz de realizar previsões precisas em novos dados. Essa propagação do erro é feita utilizando a regra da cadeia do cálculo diferencial, que permite calcular as derivadas parciais de funções compostas.

Na Figura 1.6 é ilustrada uma rede neural simples e a equação (1.11) mostra o cálculo do gradiente para ajustar o peso da primeira camada (w_1) utilizando a regra da cadeia,

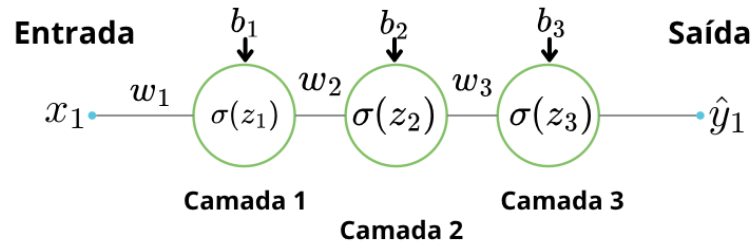
$$\nabla_w C = \frac{\partial C(w, b)}{\partial w_1} = \frac{C(w, b)}{\sigma(z_3)} \cdot \frac{\sigma(z_3)}{z_3} \cdot \frac{z_3}{\sigma(z_2)} \cdot \frac{\sigma(z_2)}{z_2} \cdot \frac{z_2}{\sigma(z_1)} \cdot \frac{\sigma(z_1)}{z_1} \cdot \frac{z_1}{w_1}, \quad (1.11)$$

em que $z_1 = w_1 \cdot x + b_1$, $z_2 = \sigma(z_1) \cdot w_2 + b_2$ e $z_3 = \sigma(z_2) \cdot w_3 + b_3$ são as saídas das camadas ocultas e $C(w_i, b_i)$ é a função de custo do erro quadrático médio, para $i = 1, 2, 3$, ou seja, $C(w_1, b_1) = \frac{(\hat{y} - \sigma(z_3))^2}{n}$, com n sendo o número de amostras no conjunto de dados.

[mudar aqui](#)

O algoritmo de *backpropagation* pode ser resumido nos seguintes passos, apresentados no Algoritmo 2.

Figura 1.6: Ilustração do processo de *backpropagation* em uma rede neural.



Fonte: Elaboração do Autor.

Entrada: Dados de treinamento (x, y) , taxa de aprendizado η , Matriz de pesos W e vetor de vieses b ;

Saída: Matriz de pesos W e vetor de vieses b atualizados;

Repita

Passo para frente: propagar x pela rede (camada a camada) e obter $\hat{y}$;

Cálculo do Erro: calcular o erro comparando $\hat{y}$ com y usando uma função de custo;

Passo para trás: retropropagar o erro e calcular os gradientes $\nabla_W C$ e $\nabla_b C$ (regra da cadeia);

Atualização: atualizar pesos e vieses (descida do gradiente);

$W \leftarrow W - \eta \nabla_W C$;

$b \leftarrow b - \eta \nabla_b C$;

até critério de parada (*ex.*: n^o de épocas ou convergência);

Algoritmo 2: Etapas do treinamento via *backpropagation*.

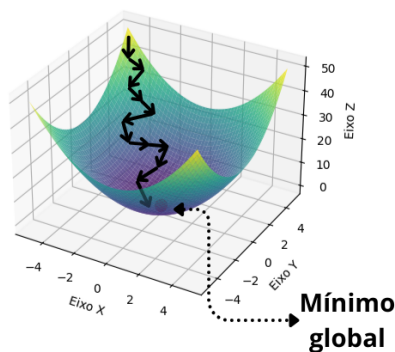
1.2.2 Gradiente Descendente Estocástico

O Gradiente Descendente Estocástico, em inglês *Stochastic Gradient Descent* (SGD) é uma variação do método de descida do gradiente tradicional, amplamente utilizado no treinamento de redes neurais e outros modelos de aprendizagem de máquina (RUDER, 2016). A principal diferença entre o SGD e o método de descida do gradiente padrão é a forma como os gradientes são calculados e atualizados durante o treinamento. No método de descida do gradiente tradicional, os gradientes são calculados usando todo o conjunto de dados de treinamento. Assim, os pesos são atualizados uma vez em cada época de treinamento, o que pode ser computacionalmente caro e demorado, especialmente para grandes conjuntos de dados. Em contraste, o

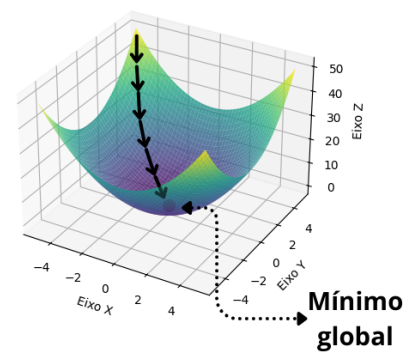
SGD calcula os gradientes usando apenas uma amostra aleatória, ou um pequeno lote, do inglês *mini-batch*, do conjunto de dados em cada iteração, ou seja, caso o conjunto de dados tenha um tamanho de 1000 e o tamanho do lote seja 10, o SGD atualiza os pesos 100 vezes por época. Isso torna o processo de atualização dos pesos mais rápido e eficiente, permitindo que o modelo aprenda mais rapidamente. Apesar da Descida do Gradiente padrão fornecer uma direção mais precisa para o mínimo global da função de custo, o SGD é capaz de chegar próximo do mínimo de forma mais rápida e eficiente.

Na Figura 1.7(a) é ilustrado o funcionamento do SGD, em que cada seta preta indica uma atualização dos pesos do modelo. Na Figura 1.7(b) é ilustrado o caminho percorrido pelo método de descida do gradiente tradicional.

Figura 1.7: Comparação entre os métodos de descida do gradiente.



(a) Gradiente Descendente Estocástico.



(b) Descida do Gradiente Tradicional.

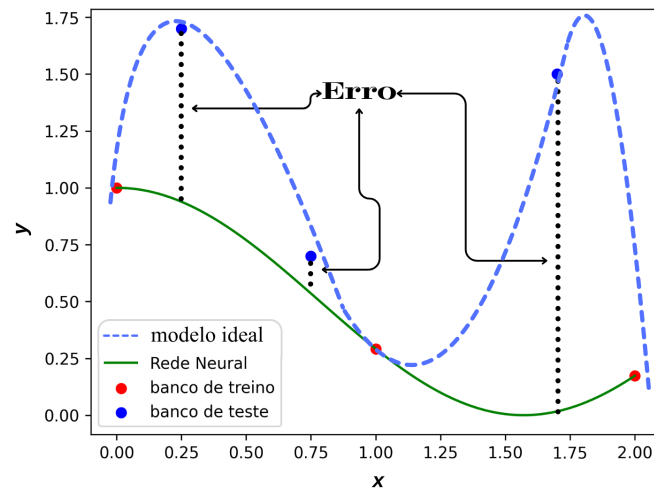
Fonte: Elaboração do Autor.

O gráfico da Figura 1.7 representa a função de custo em um espaço de parâmetros.

1.2.3 *Overfitting*

O *overfitting*, em português “Sobreajuste” ou “Ajuste Excessivo”, é um problema comum em aprendizagem de máquina, em que um modelo aprende os detalhes e o ruído dos dados de treinamento a tal ponto que este se torna incapaz de generalizar para novos dados (DEEP LEARNING BOOK, 2025b). Isso significa que o modelo tem um desempenho excelente nos dados de treinamento, mas falha ao fazer previsões precisas em dados não vistos anteriormente.

O *overfitting* ocorre quando o modelo é muito complexo em relação à quantidade e qualidade dos dados disponíveis para treinamento. Modelos com muitos parâmetros, como redes neurais profundas, são particularmente suscetíveis ao *overfitting*, especialmente quando o conjunto de dados de treinamento é pequeno ou contém ruído significativo. Na Figura 1.8 é apresentada uma ilustração do *overfitting*, em que os pontos vermelhos são os dados de treinamento e os pontos azuis são os dados de teste. A curva verde representa um modelo que se ajusta perfeitamente aos dados de treinamento, mas falha em generalizar para novos dados, enquanto a curva azul seria um modelo ideal mais general.

Figura 1.8: Ilustração do *overfitting*.

Fonte: Elaboração do Autor.

1.2.4 Hiperparâmetros

Um algoritmo de aprendizado pode ser visto como uma função que recebe dados de treinamento como entrada e produz como saída um modelo matemático. No entanto, na prática muitos algoritmos de aprendizado possuem parâmetros adicionais que não são aprendidos diretamente a partir dos dados de treinamento, mas que influenciam o processo de aprendizado. Esses parâmetros são chamados de hiperparâmetros e sua escolha adequada é crucial para o desempenho do modelo (BENGIO, 2012). Alguns exemplos comuns de hiperparâmetros incluem:

- **Taxa de Aprendizado:** Controla o tamanho dos passos dados durante a atualização dos pesos do modelo. Uma taxa de aprendizado muito alta pode levar a oscilações e impedir a convergência, enquanto uma taxa muito baixa pode resultar em um processo de treinamento lento. Valores típicos para a taxa de aprendizado variam entre 0,01 e 0,1, dependendo do problema e do algoritmo utilizado (BENGIO, 2012).
- **Número de Épocas:** Define quantas vezes o algoritmo de aprendizado passará por todo o conjunto de dados de treinamento. Um número muito alto pode levar ao *overfitting*, enquanto um número muito baixo pode resultar em *underfitting* (subajuste). Uma técnica comum para determinar o número ideal de épocas é o uso do *Early Stopping* (do inglês Parada Antecipada), que monitora o desempenho do modelo em um conjunto de validação e interrompe o treinamento quando o desempenho começa a piorar (BENGIO, 2012).
- **Tamanho do Lote:** Refere-se ao número de amostras de dados usadas para calcular o gradiente em cada iteração do treinamento. Tamanhos de lote menores podem levar a atualizações mais ruidosas, enquanto tamanhos maiores podem resultar em um treinamento mais estável, embora mais lento. Os valores escolhidos geralmente variam entre 1 e 32. (BENGIO, 2012).
- **Arquitetura do Modelo:** Inclui o número de camadas, o número de unidades em cada

camada e o tipo de funções de ativação utilizadas. Modelos mais complexos podem capturar padrões mais sofisticados, mas também são mais propensos ao *overfitting* (BENGIO, 2012).

1.2.5 Regularização

[mudar aqui](#)

A regularização é uma técnica utilizada para prevenir o *overfitting* em modelos de aprendizagem de máquina, adicionando uma penalidade à função de custo (DEEP LEARNING BOOK, 2025c). Essa penalidade incentiva o modelo a manter os valores reduzidos para os pesos, o que ajuda a evitar que ele se ajuste excessivamente aos dados de treinamento. Existem várias técnicas de regularização, sendo as mais comuns:

- Regularização L1 (Norma 1): Adiciona uma penalidade proporcional à soma dos valores absolutos dos pesos à função de custo. Isso pode levar a soluções esparsas, onde alguns pesos são reduzidos a zero. A equação da função de custo com regularização L1 é dada por:

$$C(w_j, b) = C_0(w_j, b) + \frac{\lambda}{n} \sum_i |w_i|, \quad (1.12)$$

em que $C_0(w_j, b)$ é a função de custo original, λ é o parâmetro de regularização que controla a força da penalidade, w_i, w_j são os pesos do modelo, b é o viés e n é o número de amostras no conjunto de dados (DEEP LEARNING BOOK, 2025c).

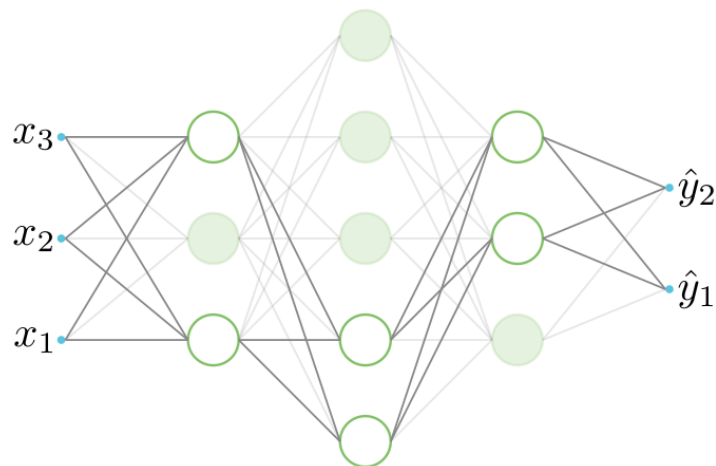
- Regularização L2 (Norma 2): Adiciona uma penalidade proporcional à soma dos quadrados dos pesos à função de custo. Isso incentiva o modelo a distribuir os pesos de forma mais uniforme, evitando que alguns pesos se tornem muito grandes. A equação da função de custo com regularização L2 é dada por:

$$C(w_j, b) = C_0(w_j, b) + \frac{\lambda}{n} \sum_i w_i^2, \quad (1.13)$$

em que os termos são definidos da mesma forma que na regularização L1 (DEEP LEARNING BOOK, 2025c).

- *Dropout*: Durante o treinamento, essa técnica desativa aleatoriamente uma fração das unidades em cada camada da rede neural para cada época. Isso força a rede a aprender representações mais robustas e reduz a dependência de unidades específicas, ajudando a prevenir o *overfitting*. Em termos gerais, ao aplicar o *dropout* estão sendo treinadas uma coleção de sub-redes dentro da rede original, o que pode promover a generalização do modelo (DEEP LEARNING BOOK, 2025c).
- *Early Stopping* (Parada Antecipada): Monitora o desempenho do modelo em um conjunto de validação durante o treinamento e interrompe o processo quando o desempenho começa a piorar. Isso evita que o modelo continue a se ajustar aos dados de treinamento além do

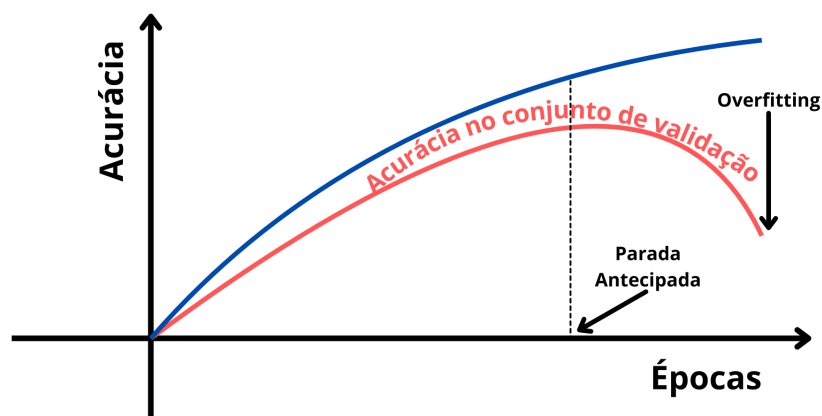
Figura 1.9: Ilustração do *Dropout* removendo 60% das unidades aleatoriamente durante uma época de treinamento.



Fonte: Elaboração do Autor.

ponto ideal. Como ilustrado na Figura 1.10, o ponto de parada ideal é onde a acurácia na validação começa a diminuir, indicando o início do *overfitting*.

Figura 1.10: Ilustração do *Early Stopping*.



Fonte: Adaptado de (DEEP LEARNING BOOK, 2025c).

1.2.6 *Momentum* e Adam

O *Momentum*, do português Momento, é uma técnica utilizada para acelerar o processo de treinamento de redes neurais, especialmente em situações onde a função de custo apresenta regiões íngremes ou planas (SUTSKEVER, 2013). A ideia principal do *momentum* é acumular uma média ponderada dos gradientes anteriores para suavizar as atualizações dos pesos, permitindo que o modelo mantenha uma direção consistente durante o treinamento. Isso ajuda a evitar

oscilações e acelera a convergência para o mínimo da função de custo. Dada uma função de custo $C(w_t, b_t)$ a ser minimizada, o *momentum* clássico é definido por:

$$\begin{cases} v_{t+1} = \beta v_t - 1 + \epsilon \nabla C(w_t, b_t) \\ \theta_{t+1} = \theta_t + v_{t+1}, \end{cases} \quad (1.14)$$

em que $\epsilon > 0$ é a taxa de aprendizagem, $\beta \in [0, 1]$ é o coeficiente de *momentum*, v_t é a velocidade acumulada no tempo t , $\theta_t = (w_t, b_t)$ representa os parâmetros do modelo (pesos e vieses) no tempo t e $\nabla C(w_t, b_t)$ é o gradiente da função de custo em relação aos parâmetros do modelo. O coeficiente de *momentum* β controla a influência dos gradientes anteriores nas atualizações atuais. *Adaptive Moment Estimation - Adam* é um método que calcula taxas de aprendizagem (η) adaptativas para cada parâmetro do modelo, em cada passo de atualização dos pesos durante o treinamento. O Adam combina as vantagens do *momentum* e do método de descida do gradiente (RUDER, 2016). Atualmente, o Adam é um dos algoritmos de otimização mais populares para treinamento de redes neurais, devido à sua eficiência e capacidade de lidar com grandes conjuntos de dados e parâmetros.

1.2.7 *Vanishing Gradient*

O problema do *vanishing gradient*, do português gradiente que desaparece, ocorre durante o treinamento de redes neurais profundas, em que os gradientes calculados durante o processo de *backpropagation* tornam-se extremamente pequenos à medida que são propagados para as camadas mais próximas da entrada. Isso acontece devido à multiplicação repetida de pequenas derivadas associadas às funções de ativação e aos pesos das camadas. Como resultado, os pesos das camadas iniciais da rede recebem atualizações insignificantes, dificultando o aprendizado efetivo dessas camadas. Como neste trabalho serão utilizadas redes neurais profundas, é importante entender o impacto do problema do *vanishing gradient* e as técnicas para mitigá-lo.

Exemplo 1.2. Considere uma rede neural profunda com N camadas ocultas, em que todas as unidades em cada camada utilizam a função de ativação sigmoid logística, definida como:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.15)$$

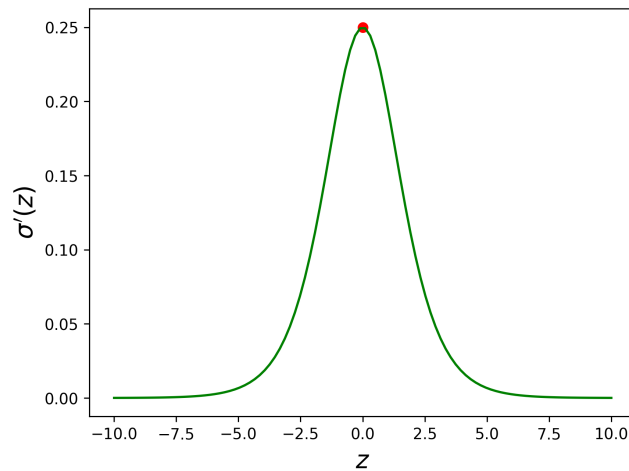
A derivada dessa função é dada por:

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)). \quad (1.16)$$

Durante o processo de *backpropagation*, o gradiente da função de custo em relação aos pesos das camadas iniciais é calculado utilizando a regra da cadeia. Isso envolve a multiplicação das derivadas das funções de ativação de cada camada, como mostra a equação (1.11). Como a derivada da função sigmoid é sempre menor que 0,25, o valor máximo ocorre em $z = 0$, como mostrado na Figura 1.11. A multiplicação repetida dessas pequenas derivadas resulta em gradientes que se tornam cada vez menores à medida que são propagados para trás através das

camadas da rede. Por exemplo, se uma rede possui 10 camadas ocultas, o gradiente pode ser reduzido a um valor extremamente pequeno, como 10^{-9} ou menor, tornando as atualizações dos pesos nas camadas iniciais praticamente insignificantes.

Figura 1.11: Gráfico da derivada da função sigmoid logística.



Fonte: Elaboração do Autor.

O problema do *vanishing gradient* é particularmente comum em redes neurais que utilizam funções de ativação como a *sigmoid* ou a tangente hiperbólica. Para contornar o problema do *vanishing gradient*, várias técnicas podem ser empregadas, incluindo:

- **Funções de Ativação Alternativas:** Funções de ativação como a ReLU e suas variantes (Leaky ReLU, Parametric ReLU) têm derivadas constantes para valores positivos, o que ajuda a manter os gradientes maiores durante o *backpropagation*.
- **Batch Normalization:** Normaliza as ativações de cada camada durante o treinamento, o que pode ajudar a estabilizar os gradientes e acelerar o processo de aprendizagem.

1.2.8 Função de Custo *Binary Cross-Entropy*

O *Vanishing Gradient* acontece principalmente em redes neurais profundas que utilizam funções de ativação como a *sigmoid*, uma das formas de contornar esse problema é utilizando a função de custo *Binary Cross-Entropy* - BCE, que é comumente utilizada em tarefas de classificação binária (ELHARROUSS *et al.*, 2025). A função de custo BCE é definida como:

$$\text{BCE}(y, \hat{y}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (1.17)$$

em que y_i é o valor de saída verdadeiro da i -ésima amostra (0 ou 1), $\hat{y}_i$ é a previsão da rede para a i -ésima amostra (probabilidade de pertencer à classe 1) e n é o número total de amostras no conjunto de dados. A função de custo BCE é especialmente eficaz para lidar com o problema do *vanishing gradient*, pois ela penaliza fortemente as previsões incorretas, o que pode ajudar a manter os gradientes maiores durante o processo de *backpropagation*.

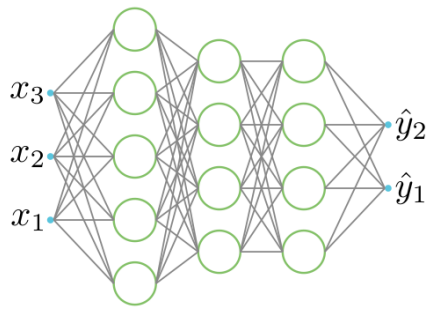
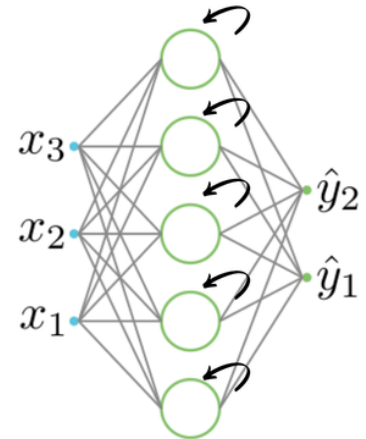
1.2.9 Arquiteturas de Redes Neurais

Existem diversas arquiteturas de redes neurais, cada uma projetada para lidar com diferentes tipos de problemas e dados. Algumas das arquiteturas mais comuns incluem (DEEP LEARNING BOOK, 2025a):

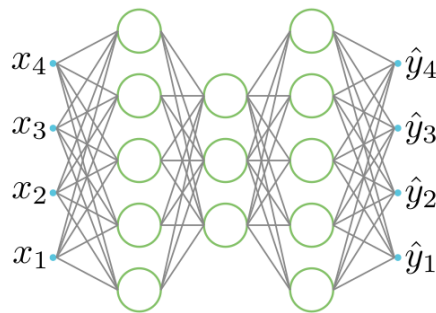
- Redes Neurais *Multi Layer Perceptron* (MLP): São redes neurais *feedforward*, ou seja, a entrada passa por uma ou mais camadas ocultas até ser processada pela camada de saída. Cada unidade em uma camada oculta está conectada a todas as unidades na camada seguinte, e as informações fluem apenas em uma direção, da entrada para a saída. Após cada saída a rede ajusta os pesos utilizando o algoritmo de *backpropagation*. MLPs são amplamente utilizadas para tarefas de classificação e regressão (DEEP LEARNING BOOK, 2025a).
- Redes Neurais Convolucionais (RNCs): São projetadas para processar dados com uma estrutura de grade, como imagens. Estas utilizam operações de convolução para extrair características espaciais dos dados de entrada. RNCs são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos e detecção de faces (DEEP LEARNING BOOK, 2025a).
- Redes Neurais Recorrentes (RNNs): São projetadas para lidar com dados sequenciais, onde a saída em um determinado instante depende das entradas anteriores. RNNs possuem conexões recorrentes que permitem que informações sejam mantidas ao longo do tempo. Estas são amplamente utilizadas em tarefas como processamento de linguagem natural e reconhecimento de fala (DEEP LEARNING BOOK, 2025a).
- Redes Neurais Generativas Adversariais (GANs): Consistem em duas redes neurais que competem entre si: um gerador e um discriminador. O gerador cria dados sintéticos, enquanto o discriminador tenta distinguir entre dados reais e sintéticos. GANs são amplamente utilizadas para geração de imagens realistas e outras tarefas criativas (DEEP LEARNING BOOK, 2025a).
- *Encoder-Decoder*: São arquiteturas que consistem em duas partes principais: o *encoder*, que comprime os dados de entrada em uma representação latente, e o *decoder*, que reconstrói os dados a partir dessa representação. Essas redes são amplamente utilizadas em tarefas como tradução automática e geração de texto (DEEP LEARNING BOOK, 2025a).

Também é possível combinar diferentes arquiteturas de redes neurais, como abordado na Seção 3.1. Na Figura 1.12 são ilustradas as arquiteturas de Redes Neurais.

Figura 1.12: Arquiteturas de Redes Neurais.

(a) Redes Neurais *Multi Layer Perceptron* (MLP).

(b) Redes Neurais Recorrentes (RNNs).

(c) *Encoder-Decoder*.

Fonte: Elaboração do Autor.

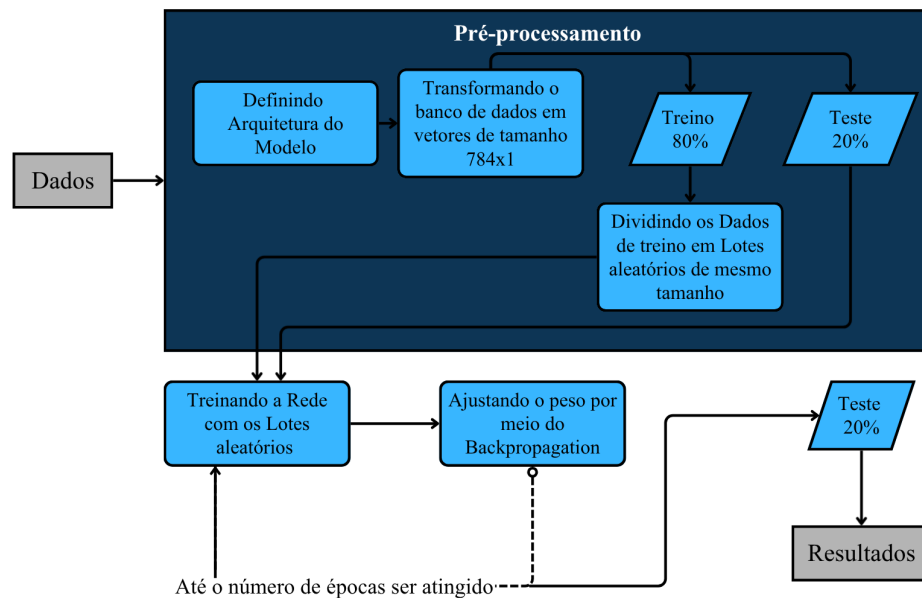
Nas Figuras 1.12(a) e 1.12(c) são ilustrados a arquitetura de uma rede neural *multi layer perceptron*, com o funcionamento semelhante ao desenvolvido na equação (1.9). Na Figura 1.12(b) é mostrada a arquitetura de uma rede neural recorrente, estas redes possuem conexões recorrentes, ou seja, a saída de uma unidade é realimentada como entrada para a mesma unidade ou para unidades anteriores, permitindo que informações sejam mantidas ao longo do tempo, por não estar no escopo deste trabalho, não é abordado os detalhes matemáticos de funcionamento das RNNs.

1.2.10 Aplicação da Arquitetura MLP para Classificação de Dígitos Manuscritos

Para exemplificar o funcionamento de como uma rede neural faz o reconhecimento de imagens, considere a tarefa de classificação de dígitos manuscritos utilizando o conjunto de dados MNIST (LECUN, 1998). Na Figura 1.13 é ilustrado o fluxo de dados durante o processo de treinamento e avaliação da rede neural para essa tarefa. O processo começa com a preparação dos dados, em

que as imagens são pré-processadas, transformadas em vetores e normalizadas para garantir que os valores dos pixels estejam em uma faixa adequada para o treinamento da rede. Em seguida, os dados são divididos em conjuntos de treinamento e teste, onde o conjunto de treinamento é utilizado para ajustar os pesos da rede neural, enquanto o conjunto de teste é utilizado para avaliar o desempenho do modelo após o treinamento. Em seguida é utilizado o algoritmo de SGD para atualizar os pesos da rede com base nos gradientes calculados pelo algoritmo de *backpropagation* e dividir o conjunto de dados de treinamento em lotes.

Figura 1.13: Fluxo de dados durante o processo de treinamento e avaliação da rede neural.



Fonte: Elaboração do Autor.

O conjunto de dados MNIST consiste em 70.000 imagens em escala de cinza de dígitos manuscritos (0 a 9), cada uma com uma resolução de 28×28 pixels, como mostrado na Figura 1.14.

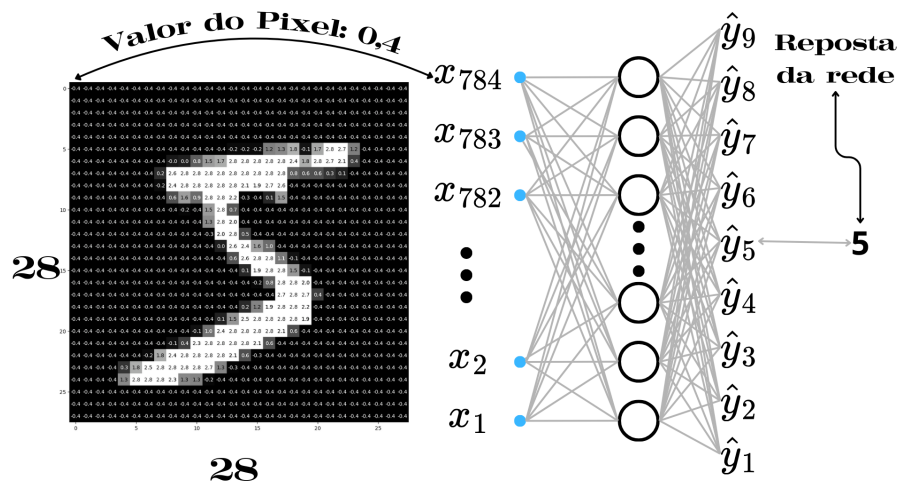
Figura 1.14: Exemplos de dígitos manuscritos do conjunto de dados MNIST.



Fonte: Adaptado de (LECUN, 1998).

Cada imagem é representada como uma matriz de 28×28 , em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco). Para o reconhecimento dos dígitos é utilizado uma rede MLP e os métodos presentes nas Seções 1.2.1, 1.2.2, 1.2.5 e 1.2.7 para o treinamento e otimização dos pesos. Para compor a camada de entrada da rede neural, cada imagem é “achatada” em um vetor de 784 elementos ($28 \times 28 = 784$), em que cada elemento corresponde à intensidade de um pixel. O objetivo da rede com camadas totalmente conectadas é aprender a classificar corretamente cada imagem em uma das 10 classes correspondentes aos dígitos de 0 a 9. Como a camada de saída da rede neural deve ter 10 unidades (uma para cada classe), a arquitetura da rede pode ser definida como $[784, 30, 10]$, onde a camada de entrada tem 784 unidades, a camada oculta tem 30 unidades, valor escolhido aleatoriamente, e a camada de saída tem 10 unidades, como mostrado na Figura 1.15.

Figura 1.15: Arquitetura da rede neural para classificação de dígitos manuscritos.



Fonte: Elaboração do Autor.

Na Figura 1.15 uma imagem do dígito 5 é processada pela rede neural. A camada de entrada recebe o valor de cada entrada do vetor da imagem, com esses valores sendo processados pela camada oculta e a camada de saída fornecendo as probabilidades para cada classe de dígitos, sendo a classe com a maior probabilidade escolhida como a previsão da rede.

A função de ativação utilizada é a função *sigmoid* logística vista na equação (1.3), que é comumente usada em tarefas de classificação. A saída da rede é um vetor de 10 elementos, em que cada elemento representa a probabilidade da imagem pertencer a uma das classes de dígitos.

Para o treinamento da rede é utilizado o algoritmo de *backpropagation* e o SGD, com uma taxa de aprendizagem de 0,1 e a normalização *L1* visto na equação (1.12). O treinamento é realizado para 30 épocas. O banco de dados é dividido em um conjunto de treinamento (80%) com 60.000 imagens e um conjunto de teste (20%) com 10.000 imagens, divisão realizada aleatoriamente. Após o treinamento, a rede neural é avaliada no conjunto de teste para medir

sua acurácia, que é a proporção de previsões corretas em relação ao total de previsões feitas:

$$\text{Acurácia} = \frac{\text{Número de Previsões Corretas}}{\text{Número Total de Previsões}}. \quad (1.18)$$

Com as configurações mencionadas, a rede neural alcançou uma acurácia de aproximadamente 95% no conjunto de teste, o que indica que esta foi capaz de aprender a classificar corretamente a maioria das imagens de dígitos manuscritos. Esta aplicação resultou em um trabalho apresentado no *I Workshop* de Matemática da Universidade Federal de Uberlândia (SENA; JAFELICE; SANTOS, 2025).

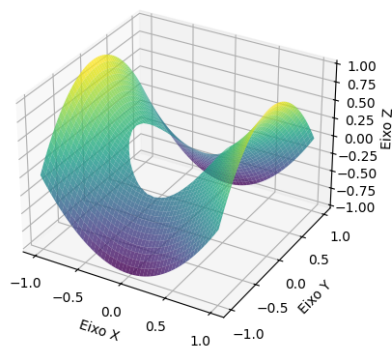
1.2.11 Aplicação da Arquitetura MLP para Reconstrução de Figura Geométrica

Pode-se usar redes neurais para reconstruir malhas geométricas (SENA; JAFELICE; SANTOS, 2024). Neste caso, utilizou-se uma MLP com duas unidades na camada de entrada, dez unidades na camada oculta e uma unidade na camada de saída, para aproximar o gráfico do parabolóide hiperbólico $z = x^2 - y^2$. A rede é treinada utilizando o algoritmo de *backpropagation* e o SGD, com uma taxa de aprendizagem de 0,01 e a normalização *L2* visto na equação (1.13). O treinamento é realizado para 1000 épocas com todos as unidades tendo função de ativação *sigmoid* logística. Os dados considerados são nós de uma malha cartesiana uniforme $x, y \in [-1, 1]$, divididos em duzentas partes com $z = x^2 - y^2$. São considerados 20% para os dados de teste e 80% para os dados de treino, retirando os pontos pertencentes à bola fechada de centro 0 e raio 0,5, ou seja, é construído um corte no parabolóide hiperbólico. Como resposta, a rede fornece o valor de $z \approx x^2 - y^2$.

Na Figura 1.16(a) são apresentados os dados utilizados no treinamento, enquanto na Figura 1.16(b) são mostrados os valores reconstruídos pela rede.

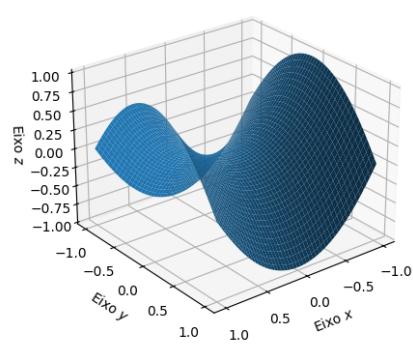
Figura 1.16: Parabolóide hiperbólico.

Superfície: $z = x^2 - y^2$ com corte em $x^2 + y^2 \leq 0.25$



(a)

Aproximação da Superfície: $z = x^2 - y^2$

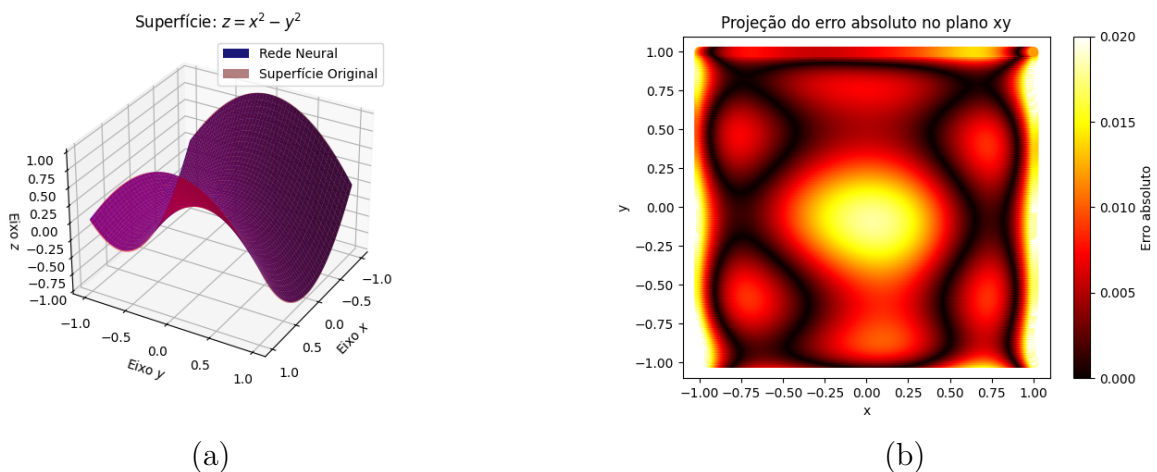


(b)

Fonte: Elaboração do Autor.

A abordagem proposta obteve uma acurácia de 93% na reconstrução do paraboloide. O cálculo da acurácia foi realizado a partir dos dados de teste, considerando a condição $|\hat{y} - y| \leq 0,01$, em que $\hat{y}$ representa a resposta fornecida pela rede e y a solução analítica. Como toda aproximação, existe um erro entre a forma analítica e a aproximada pela MLP, ao calcular a saída para os dados $x = 1, y = 1$ a rede responde $-0,0192$. Observa-se que o erro excede a precisão definida na acurácia, uma vez que, dos 6443 pontos no conjunto de teste, aproximadamente 6014 estão dentro da margem de tolerância, resultando na acurácia de 93%. Caso a precisão fosse ampliada para 0,02, a acurácia alcançaria 99%, podendo apresentar erro superior a 0,02. Na Figura 1.17(a) são apresentados o gráfico da função z e a superfície aproximada pela rede. A projeção da superfície do erro absoluto, dado por: $E = |\hat{y} - y| \leq 0,02$, em que $\hat{y}$ representa a resposta fornecida pela rede e y a solução analítica é exibida na Figura 1.17(b), em que os pontos de “E” com as cores mais claras estão próximos de 0,02 e os mais escuros, próximos de 0.

Figura 1.17: Gráfico da superfície e aproximação da superfície obtida pela MLP e a projeção do E no plano xy .



Fonte: Elaboração do Autor.

É possível observar que na Figura 1.17(b) a região de maior erro está concentrada na região do corte, ou seja, onde os dados de treinamento foram retirados.

1.3 Rede Neural para Análise de Imagens

1.3.1 Córtex Visual Humano e Visão Computacional

O córtex visual humano é a parte do cérebro responsável pelo processamento das informações visuais recebidas pelos olhos (STANFIELD, 2014). Está localizado na parte posterior do cérebro, na região occipital. O córtex visual é organizado em várias áreas, cada uma especializada em processar diferentes aspectos da visão, como cor, forma, movimento e profundidade. Essas áreas trabalham em conjunto para interpretar e compreender o mundo visual ao nosso redor. A

estrutura do córtex visual é altamente complexa, com bilhões de unidades interconectadas que formam redes neurais intrincadas. Essas redes são capazes de aprender e adaptar-se com base nas experiências visuais, permitindo-nos reconhecer objetos, rostos e padrões visuais de maneira eficiente. Uma característica importante do córtex visual é a presença de campos receptivos, que são regiões específicas do campo visual onde a estimulação de uma unidade resulta em uma resposta. Esses campos receptivos são organizados de forma hierárquica, com unidades em níveis mais baixos respondendo a características simples, como bordas e texturas, enquanto unidades em níveis mais altos respondem a características mais complexas, como formas e objetos completos. É por meio desse mecanismo e também pela capacidade de aprendizagem que o ser humano é capaz de reconhecer e interpretar o mundo visual ao seu redor.

Traduzir a visão para uma representação digital é um processo extremamente complexo. Em parte, porque é um problema inverso, no qual busca-se recuperar algumas variáveis desconhecidas a partir de informações insuficientes para especificar completamente a solução. Portanto, deve-se recorrer a modelos baseados na física, modelos probabilísticos ou aprendizagem de máquina com grandes conjuntos de exemplos para resolver ambiguidades entre possíveis soluções. Entretanto, modelar o mundo visual em toda a sua rica complexidade é muito mais difícil do que, por exemplo, modelar o trato vocal que produz sons da fala (SZELISKI, 2010).

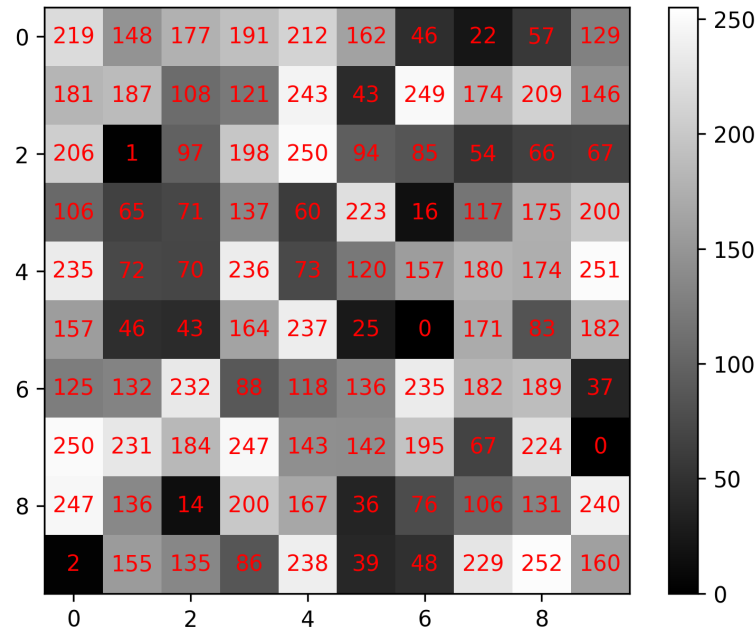
É surpreendente que humanos e animais façam isso com tanta facilidade, enquanto algoritmos de visão computacional são tão propensos a erros. Pessoas que nunca trabalharam na área frequentemente subestimam a dificuldade do problema. Essa percepção equivocada de que a visão deveria ser fácil remonta aos primeiros dias da inteligência artificial, quando se acreditava inicialmente que as partes cognitivas (prova lógica e planejamento) da inteligência eram intrinsecamente mais difíceis do que os componentes perceptivos (MÜLLER, 2008).

Uma imagem virtual é uma representação visual de um objeto, cena ou conceito, composta por uma matriz de pixels que contém informações sobre cor e intensidade. Cada pixel é uma célula na matriz da imagem que possui um valor específico. As imagens podem ser em escala de cinza, em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco), como mostrado na Figura 1.18, ou em cores, em que cada pixel é representado por uma combinação de valores para as cores primárias (vermelho, verde e azul - VVA). Uma imagem colorida pode ser interpretada como a sobreposição de três matrizes, cada matriz representando uma das cores primárias, essas matrizes são chamadas de canais, por exemplo, uma imagem colorida de 10×10 pixels seria representada por uma matriz de $10 \times 10 \times 3$, onde cada elemento da matriz contém os valores de intensidade para as cores vermelha, verde e azul. Entretanto, nem sempre os canais representam uma matriz de cor, podendo representar outras informações, como a profundidade de um objeto em uma cena ou uma característica específica da imagem.

1.3.2 Redes Neurais Convolucionais

As redes neurais convolucionais, do inglês *Convolutional Neural Networks* - CNNs, são uma classe especial de redes neurais projetadas para processar dados com uma estrutura de grade e que tenham uma relação de vizinhança entre os elementos, como imagens (TREVISAN, 2021),

Figura 1.18: Exemplo de uma imagem/matriz $10 \times 10 \times 1$ em escala de cinza, em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco).



Fonte: Elaboração do Autor.

foram baseadas do Cortex Visual Humano, Seção 1.3.1. Estas são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos, detecção de faces e segmentação¹ de imagens. As CNNs são inspiradas na organização do córtex visual, onde diferentes regiões do cérebro são responsáveis por processar diferentes partes do campo visual.

1.3.3 Convolução

Exemplo 1.3. Supondo que na tentativa de localizar uma nave com um sensor a laser (GO-ODFELLOW; BENGIO; COURVILLE, 2016), este sensor retorna uma única saída $x(t)$, a posição da nave no tempo t . $x(t)$ e t são valores reais, ou seja, é possível adquirir diferentes leituras do sensor em qualquer instante de tempo. Supondo que o sensor seja ruidoso, para obter pouco ruído da estimativa da posição da nave é possível fazer várias medições e calcular a média destas, entretanto, medições mais recentes são mais relevantes, então, pode-se fazer uma média ponderada que dê pesos às medições mais recentes. Seja esta média a função $w(a)$, em que a é o tempo em que a medição foi obtida. Ao aplicar $w(a)$ à $x(t)$, para todo t , é obtida a função s , que é a estimativa suavizada da posição da nave, ou seja:

$$s(t) = \int x(a)w(t-a)da. \quad (1.19)$$

¹Segmentação de imagens é uma modificação feita na área de visão computacional que envolve a divisão de uma imagem em regiões ou segmentos distintos, cada um representando um objeto ou parte de um objeto. O objetivo da segmentação é atribuir uma etiqueta a cada pixel da imagem, indicando a qual classe ou categoria ele pertence.

Essa operação é chamada de convolução (SOVIERZOSKI, 2010). A operação de convolução é tipicamente denotada com um asterístico:

$$s(t) = (x * w)(t).$$

No Exemplo 1.3, é preciso que w seja uma função densidade de probabilidade válida para que a saída seja uma média ponderada e w precisa ser igual a zero para $a < 0$. Estas restrições são particulares do exemplo, em geral, convolução é definida para quaisquer funções e integrais.

Na terminologia da rede convolucional, o primeiro termo (x) é frequentemente chamado de *input* (entrada), o segundo termo (w) é chamado de *kernel* (núcleo) ou filtro, e o resultado (s) é chamado de *feature map* (mapa de características).

A operação de convolução é comumente aplicada a dados provenientes de domínios discretos (dados de computador). Se (x) e (w) são definidas somente para valores inteiros de t , a convolução discreta é definida por:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a), \quad (1.20)$$

em muitos materiais, a convolução discreta pode ser representada por:

$$s[t] = (x * w)[t] = \sum_{a=-\infty}^{\infty} x[a]w[t-a]$$

Exemplo 1.4. Seja $f(t) = e^{-t}u(t)$ e $g(t) = u(t)$, onde $u(t)$ é a função limiar equação (1.2). A convolução $(f * g)(t)$ é dada por:

$$(f * g)(t) = \int_{-\infty}^{\infty} e^{-a}u(a)u(t-a)da.$$

Para $t < 0$, a função limiar $u(t-a)$ é zero para todos os valores de a , o que resulta em $(f * g)(t) = 0$. Para $t \geq 0$, a função limiar $u(t-a)$ é igual a 1 para $a \leq t$ e zero para $a > t$. Portanto, a convolução pode ser reescrita como:

$$(f * g)(t) = \int_0^t e^{-a}da.$$

Calculando a integral, obtem-se:

$$(f * g)(t) = [-e^{-a}]_0^t = 1 - e^{-t}.$$

Portanto, a convolução entre as funções $f(t)$ e $g(t)$ é dada por:

$$(f * g)(t) = \begin{cases} 0, & t < 0 \\ 1 - e^{-t}, & t \geq 0 \end{cases} \quad (1.21)$$

Intuitivamente, a função g é invertida e deslocada por t e depois multiplicada ponto a ponto com a função f , e o resultado é integrado sobre todo o domínio contínuo. Para domínios discretos, o Exemplo 1.5 mostra o processo de convolução entre duas sequências finitas.

Exemplo 1.5. *Seja $x[n] = \{1, 2, 3\}$ e $w[n] = \{0, 1\}$, em que $x[0] = 1$, $x[1] = 2$, $x[2] = 3$, $w[0] = 0$ e $w[1] = 1$ e $w[n] = x[n] = 0$ para outros valores de n . A convolução $(x * w)[n]$ é dada por:*

$$s[n] = (x * w)[n] = \sum_{a=0}^3 x[a]w[n-a].$$

Calculando os valores para cada n , obtém-se:

$$\begin{aligned} (n=0) \quad (x * w)[0] &= x[0]w[0] + x[1]w[-1] + x[2]w[-2] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 0 = 0 \\ (n=1) \quad (x * w)[1] &= x[0]w[1] + x[1]w[0] + x[2]w[-1] = 1 \cdot 1 + 2 \cdot 0 + 3 \cdot 0 = 1 \\ (n=2) \quad (x * w)[2] &= x[0]w[2] + x[1]w[1] + x[2]w[0] = 1 \cdot 0 + 2 \cdot 1 + 3 \cdot 0 = 2 \\ (n=3) \quad (x * w)[3] &= x[0]w[3] + x[1]w[2] + x[2]w[1] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 1 = 3 \end{aligned}$$

Portanto, a convolução entre as sequências $x[n]$ e $w[n]$ é dada por:

$$(x * w)[n] = \{0, 1, 2, 3\}. \quad (1.22)$$

Proposição 1.1. *O tamanho do vetor resultante da convolução entre dois vetores de comprimento M e N é dado por $M + N - 1$.*

Demonstração. Seja $x[n] \neq 0$ para $0 \leq n \leq M - 1$ e $w[n] \neq 0$ para $0 \leq n \leq N - 1$. Duas sequências finitas de comprimentos M e N respectivamente. Então, a convolução $(x * w)[n]$ é dada por:

$$y[n] = (x * w)[n] = \sum_{a=-\infty}^{\infty} x[a]w[n-a],$$

para que a soma seja diferente de zero, é necessário que $x[a] \neq 0$ e $w[n-a] \neq 0$, ou seja:

$$0 \leq a \leq M - 1 \quad \text{e} \quad 0 \leq n - a \leq N - 1.$$

Somando a na segunda desigualdade é obtido:

$$a \leq n \leq a + N - 1$$

Como a pode variar entre 0 e $M - 1$, quando $a = 0$, então $n \geq 0$ e quando $a = M - 1$, então $n \leq M - 1 + N - 1 = M + N - 2$. Portanto:

$$0 \leq n \leq M + N - 2,$$

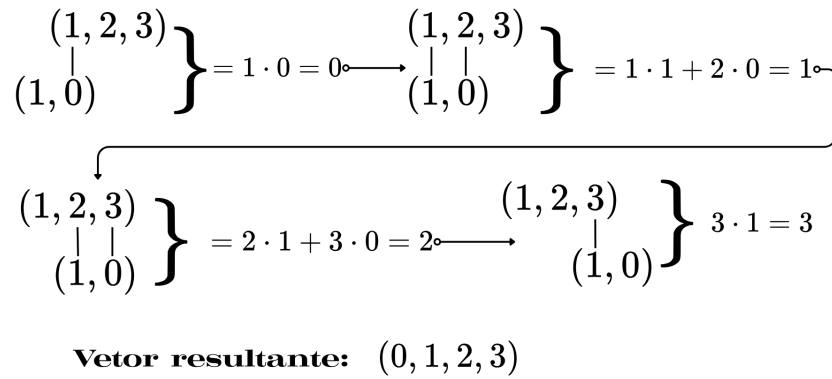
o que implica que o vetor resultante $(y[n])$ tem comprimento $M + N - 1$ □

No Exemplo 1.5, a convolução entre os vetores $x[n]$ e $w[n]$ resultou em um vetor de comprimento 4, que é igual a $3 + 2 - 1 = 4$. Para voltar ao comprimento original do vetor $x[n]$, pode-se fazer um truncamento do vetor resultante, ou seja, remover a mesma quantidade de elementos do início e do final do vetor resultante, o que é conhecido como convolução “*same*”, do português, mesma. Esta convolução é usada quando a dimensão do dado de entrada deve ser mantida na saída, quando isso não é necessário, a convolução pode receber outro nome, como por exemplo, convolução *valid*, utilizada quando os valores da saída são somente calculados quando o *kernel* está totalmente contido na entrada (CHOWDHURY, 2024). No exemplo 1.5, removendo o primeiro e o último elemento do vetor resultante, obtém-se a convolução *same*:

$$(x * w)_{same}[n] = \{1, 2\}.$$

Pode-se pensar na operação de convolução discreta como o processo de multiplicação e soma, em que o kernel w vai sendo “passado” sobre o vetor x ponto a ponto, como é mostrado na Figura 1.19.

Figura 1.19: Ilustração da operação de convolução discreta. O kernel w é deslizado sobre o vetor x , e em cada posição, a soma ponderada dos valores de x cobertos pelo kernel é calculada para gerar o valor correspondente na saída.



Fonte: Elaboração do Autor.

No contexto de redes neurais convolucionais, a convolução é aplicada a dados discretos, como imagens representadas por matrizes de pixels. Neste caso, o sinal x é uma matriz bidimensional que representa a imagem de entrada e o *kernel* w é uma matriz menor que atua como um filtro para extrair características específicas da imagem. A operação de convolução em dados discretos é realizada deslizando o *kernel* sobre a imagem e calculando a soma ponderada dos valores dos pixels cobertos pelo *kernel*. A saída da convolução é uma nova matriz, chamada de Mapa de Características, que destaca as características extraídas pela aplicação do *kernel*. No

caso de uma convolução bidimensional discreta, a operação é definida como:

$$S[i, j] = (X * W)[i, j] = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} X[m, n]W[i - m, j - n], \quad (1.23)$$

convolução é uma operação comutativa, ou seja:

$$S[i, j] = (X * W)[i, j] = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} X[i - m, j - n]W[m, n],$$

considerando que X é a matriz da imagem de entrada, W é a matriz do kernel e S é o mapa de características resultante.

Exemplo 1.6. *Considere a convolução de duas matrizes, em que a matriz de entrada*

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

é convoluída com o kernel

$$W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

para produzir a saída S . Então, como nos exemplos anteriores, é necessário inverter o kernel W e depois deslizar o kernel sobre a matriz de entrada X . Neste exemplo, a convolução é a “same”.

$$W \rightarrow \hat{W} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix},$$

$\hat{W} = J_i W J_j$ em que J_i e J_j são matrizes de permutação que realizam a rotação de 180 graus do kernel original W , ou seja:

$$J_i = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad J_j = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Para facilitar o cálculo da convolução, pode-se separar X com blocos de 2×2 (dimensão do kernel) e calcular a convolução para cada bloco. Para o bloco superior esquerdo de X até o

bloco inferior direito, temos:

$$\begin{aligned}
S(0,0) &= \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (1)(-1) + (2)(0) + (4)(0) + (5)(1) = 4, \\
S(0,1) &= \begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (2)(-1) + (3)(0) + (5)(0) + (6)(1) = 4, \\
S(1,0) &= \begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (4)(-1) + (5)(0) + (7)(0) + (8)(1) = 4, \\
S(1,1) &= \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (5)(-1) + (6)(0) + (8)(0) + (9)(1) = 4,
\end{aligned}$$

em que a operação $\odot$ significa a multiplicação de cada elemento correspondente das matrizes e a soma dos resultados. Portanto, a saída da convolução é dada por:

$$S = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}. \quad (1.24)$$

Observa-se que a saída da convolução é uma matriz de 2×2 . O cálculo da dimensão da matriz de saída é feito da seguinte forma: $Largura_S = largura_X - largura_W + 1 = 3 - 2 + 1 = 2$ e $Altura_S = altura_X - altura_W + 1 = 3 - 2 + 1 = 2$.

Na Figura 1.20 é ilustrado um exemplo de convolução bidimensional, em que uma matriz de entrada X (matriz 7×7) é convoluída com um *kernel* W (matriz 2×2) para produzir a saída S (matriz 6×6). W é deslizado sobre a matriz de entrada, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo *kernel* é calculada e o resultado é aplicado em uma função de ativação (σ) para gerar o valor correspondente na matriz de saída, assim como no Exemplo 1.6. No caso da Figura 1.20, é utilizado a convolução válida, ou seja, a saída é gerada apenas para as posições onde o *kernel* pode ser completamente aplicado à matriz de entrada sem ultrapassar seus limites.

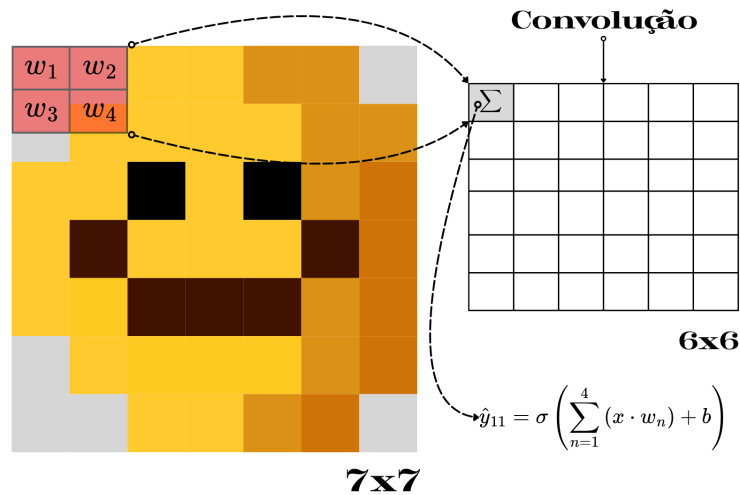
O cálculo da largura e da altura da imagem de saída pode ser feito utilizando as seguintes fórmulas:

$$w_{out} = w_{in} - w_{ker} + 1, \quad (1.25)$$

$$h_{out} = h_{in} - h_{ker} + 1, \quad (1.26)$$

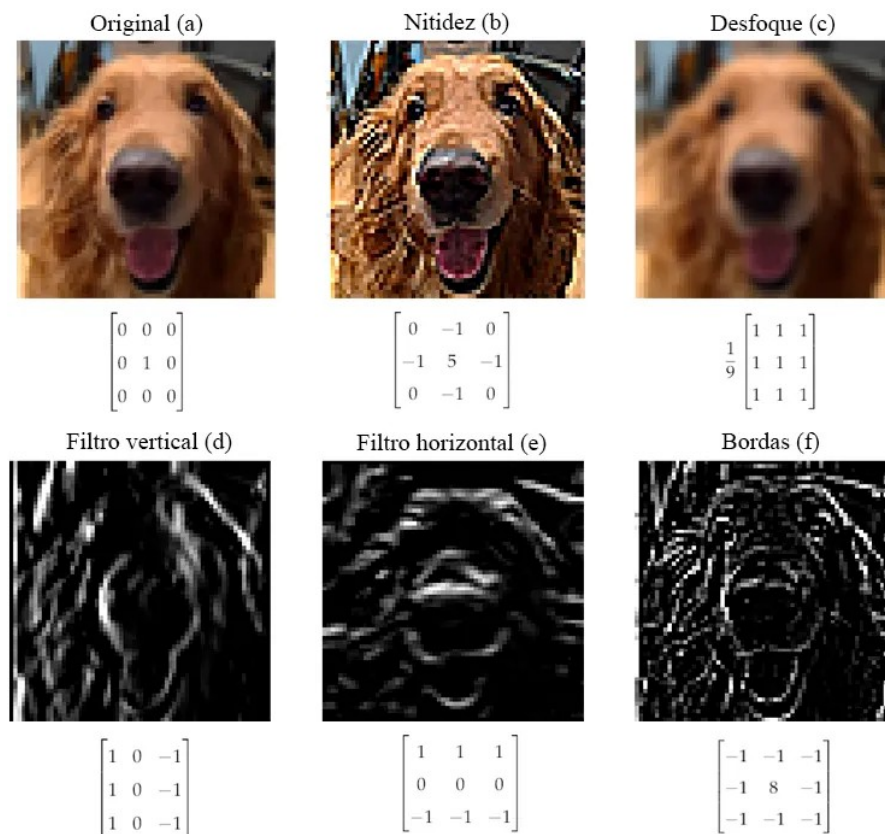
em que w_{out} e h_{out} são a largura e a altura da imagem de saída, w_{in} e h_{in} são a largura e a altura da imagem de entrada, e w_{ker} e h_{ker} são a largura e a altura do *kernel*, respectivamente. Na Figura 1.21 são mostrados diferentes aplicações de *kernels* em uma imagem.

Figura 1.20: Exemplo de convolução bidimensional. A matriz de entrada X (matriz 7×7) é convoluída com o *kernel* W (matriz 2×2) para produzir a saída S (matriz 6×6). O *kernel* é “deslizado” sobre a matriz de entrada, formando sub-matrizes, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo *kernel* é calculada e o resultado é aplicado em uma função de ativação (σ) para gerar o valor correspondente ($\hat{y}_{11}$) na matriz de saída. b sendo o viés adicionado à soma ponderada.



Fonte: Elaboração do Autor.

Figura 1.21: Aplicações de diferentes *kernels* em uma imagem. A imagem original é representada em (a), juntamente com o *kernel* identidade, que não faz modificações. Em (b) a imagem passou por um *kernel* que aumenta a nitidez reforçando o pixel central e aumentando a diferença dele em relação à sua vizinhança. Em (c) foi usado um *kernel* de desfoque, que substitui um pixel pela média dele com seus vizinhos. A imagem em (d) é o resultado de aplicar na imagem em escala de cinza um *kernel* que reforça suas linhas verticais, enquanto (e) reforça suas linhas horizontais. O *kernel* em (f) reforça os contornos (bordas) da imagem.



Fonte: Adaptado de (TREVISAN, 2021).

Normalmente, os *kernels* são muito menores do que as imagens (geralmente 3×3), e através da operação de convolução um *kernel* é utilizado para processar toda uma imagem, portanto, diz-se que os parâmetros são compartilhados (pesos e viés). Por conta dessa representação reduzida, cada camada da rede convolucional deve aprender uma quantidade menor de parâmetros. Em CNNs, considera-se a convolução como uma camada da rede, e cada camada possui múltiplos *kernels*, produzindo a mesma quantidade de mapas de atributos, cada um responsável por extrair diferentes características da imagem de entrada. Durante o treinamento da rede, os valores dos *kernels* são ajustados para otimizar a capacidade da rede de reconhecer padrões específicos nas imagens. Por exemplo, uma camada convolucional com 3 *kernels* que recebe como entrada uma imagem em escala de cinza de dimensões $(w_{in} \times h_{in} \times 1)$ produzirá uma saída com dimensões $(w_{out} \times h_{out} \times 3)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em (1.25) e (1.26). A saída não é mais uma imagem comum, mas sim um conjunto de três mapas de características, cada um representando a resposta da imagem de entrada a um dos três *kernels* aplicados. Encadeando os mapas de atributos, a rede pode aprender representações hierárquicas das características presentes nas imagens, desde bordas e texturas simples até formas e objetos mais complexos.

1.3.4 Correlação Cruzada

As redes neurais convolucionais geralmente não utilizam a definição tradicional de convolução (GOODFELLOW; BENGIO; COURVILLE, 2016), mas sim uma operação chamada correlação cruzada, definida como:

$$S[i, j] = (X * W)[i, j] = \sum_m \sum_n X[i + m, j + n] W[m, n], \quad (1.27)$$

em que X é a matriz da imagem de entrada, W é a matriz do *kernel* e S é o mapa de características resultante. Isso ocorre porque no processo de treinamento os valores do *kernel* não são definidos manualmente, mas sim aprendidos por meio dos processos de otimização dos pesos. Por conta disso, a rede aprende a melhor configuração do *kernel* sem a necessidade de realizar a rotação de 180 graus, sendo exatamente o que a definição de correlação cruzada descreve, já que W não depende de parâmetros negativos.

1.3.5 Kernel Multicamadas

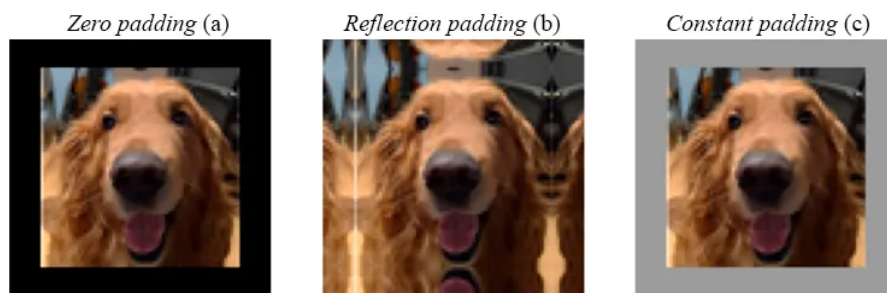
Em uma imagem com mais de um canal, como no caso de uma imagem VVA (vermelho, verde, azul) do inglês RGB (*Red, Green, Blue*), a convolução acontece com um *kernel* em cada canal da imagem, um conjunto de canais é chamado de filtro. Por exemplo, considere uma imagem RGB de dimensões $(w_{in} \times h_{in} \times 3)$ e um *kernel* de dimensões $(k_w \times k_h \times 3)$. A convolução é realizada separadamente em cada canal (vermelho, verde e azul) da imagem, utilizando o respectivo canal do *kernel*. Após a aplicação do *kernel* em cada canal, os resultados são somados ou é aplicada uma média para produzir um único mapa de características. Portanto, a saída da

convolução terá dimensões $(w_{out} \times h_{out} \times 1)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em (1.25) e (1.26).

1.3.6 *Padding*

Durante a aplicação da convolução, é comum que a dimensão espacial (largura e altura) da imagem de saída seja menor do que a da imagem de entrada. Isso ocorre porque o *kernel* não pode ser aplicado nas bordas da imagem sem ultrapassar seus limites. Para mitigar essa redução de dimensão, uma técnica chamada *padding* (O'SHEA; NASH, 2015) é frequentemente utilizada. O *padding* envolve adicionar uma borda de pixels ao redor da imagem original antes de aplicar a convolução. Esses pixels adicionais podem ser preenchidos com zeros (*zero padding*) ou com valores replicados dos pixels mais próximos da borda da imagem original, (*reflection padding*). O uso de *padding* permite que o *kernel* seja aplicado em todas as regiões da imagem, incluindo as bordas, preservando assim as dimensões espaciais da imagem de entrada na saída. Na Figura 1.22 é ilustrada a aplicação de diferentes tipos de *padding* em uma imagem.

Figura 1.22: Aplicação de *padding* em uma imagem. Em (a) a imagem original passou por *zero padding*. Em (b) a imagem original passou por *reflection padding*. A imagem em (c) passou por um *padding* constante, semelhante em (a), mas com valor diferente de zero.



Fonte: Adaptado de (TREVISAN, 2021).

O cálculo da largura e da altura da imagem de saída com *padding* pode ser ajustado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= w_{in} - w_{kernel} + 2w_{pad} + 1, \\ h_{out} &= h_{in} - h_{kernel} + 2h_{pad} + 1, \end{aligned} \tag{1.28}$$

em que w_{pad} e h_{pad} são a quantidade de pixels adicionados como *padding* na largura e na altura, respectivamente.

1.3.7 *Stride*

O *stride*, em português, passo, é um parâmetro utilizado nas operações de convolução e *pooling* em redes neurais convolucionais que determina o número de pixels que o *kernel* de *pooling* se desloca ao longo da imagem de entrada. Em outras palavras, o *stride* define o quanto o *kernel* avança a cada aplicação da operação. Um *stride* de 1 significa que o *kernel* se mova um pixel

por vez, enquanto um *stride* maior que 1 faz com que o *kernel* pule pixels, resultando em uma redução mais rápida das dimensões espaciais da imagem. O uso do *stride* afeta diretamente a dimensão da saída gerada pela operação de convolução. O cálculo da largura e da altura da imagem de saída com *stride* pode ser ajustado conforme as seguintes fórmulas:

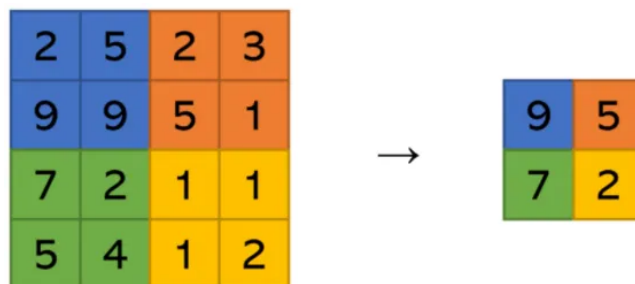
$$\begin{aligned} w_{out} &= \frac{w_{in} - w_{kernel} + 2w_{pad}}{s_w} + 1, \\ h_{out} &= \frac{h_{in} - h_{kernel} + 2h_{pad}}{s_h} + 1, \end{aligned} \quad (1.29)$$

em que s_w e s_h são os valores do *stride* na largura e na altura, respectivamente.

1.3.8 Pooling

O *pooling* (em tradução livre, *agrupamento*) é uma operação utilizada em redes neurais convolucionais para reduzir as dimensões espaciais (largura e altura) dos mapas de características, mantendo as informações mais relevantes. Essa redução ajuda a diminuir a complexidade computacional da rede. Existem diferentes tipos de operações de *pooling*, sendo as mais comuns o *Max pooling* e o *Average pooling* (Agrupamento Médio, técnica que leva em consideração a média dos valores dos pixels em cada bloco coberto pelo *kernel* de *pooling*). A técnica de *pooling* substitui agrupamentos de pixels por um valor que os represente. Na Figura 1.23 é ilustrado o funcionamento do *Max pooling* de dimensão 2×2 , ou seja, os pixels são agrupados em quadrados de dimensões 2×2 e são substituídos pelo maior valor de intensidade de pixel do grupo. Em CNNs o *stride* do *pooling* é geralmente igual a dimensão do *kernel* de *pooling*, o que resulta em uma redução de dimensão espacial pela metade a cada aplicação de *pooling*. Por exemplo, um mapa de características de dimensões 8×8 aplicado a um *Max pooling* com *kernel* de dimensões 2×2 e *stride* de 2 resultará em um mapa de características reduzido para dimensões 4×4 .

Figura 1.23: Exemplo de *Max pooling* com dimensões 2×2 . A imagem original é reduzida para a imagem menor após a aplicação do *Max pooling*. Cada bloco de 2×2 pixels na imagem original é substituído pelo valor máximo dentro desse bloco na imagem reduzida.



Fonte: Adaptado de (TREVISAN, 2021).

O *Max pooling* pode ser implementado utilizando o Algoritmo 3, em que X é a matriz de entrada, k é a dimensão do *kernel* de *pooling* e Y é a matriz de saída resultante do *pooling*. O algoritmo percorre a matriz de entrada em blocos de dimensões $k \times k$, calculando o valor

máximo dentro de cada bloco e atribuindo esse valor à posição correspondente na matriz de saída Y . O resultado é uma matriz reduzida que mantém as características mais importantes da imagem original.

Entrada: X (matriz de entrada), k (dimensão do *kernel* de *pooling*)
Saída: Y (matriz de saída)
 $w_{out} \leftarrow \lfloor w_{in}/k \rfloor$;
 $h_{out} \leftarrow \lfloor h_{in}/k \rfloor$;
 $Y \leftarrow$ matriz de zeros de dimensões $w_{out} \times h_{out}$;
Para $i \leftarrow 0$ **Até** $w_{out} - 1$ **Faça**
 Para $j \leftarrow 0$ **Até** $h_{out} - 1$ **Faça**
 $Y[i][j] \leftarrow \max(X[i * k : (i + 1) * k][j * k : (j + 1) * k]);$
 Fim
Fim

Algoritmo 3: *Max pooling*

O cálculo da largura e da altura da imagem de saída após a aplicação do *pooling*, quando o *stride* é igual à dimensão do *kernel*, pode ser realizado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= \lfloor \frac{w_{in}}{k} \rfloor, \\ h_{out} &= \lfloor \frac{h_{in}}{k} \rfloor, \end{aligned} \tag{1.30}$$

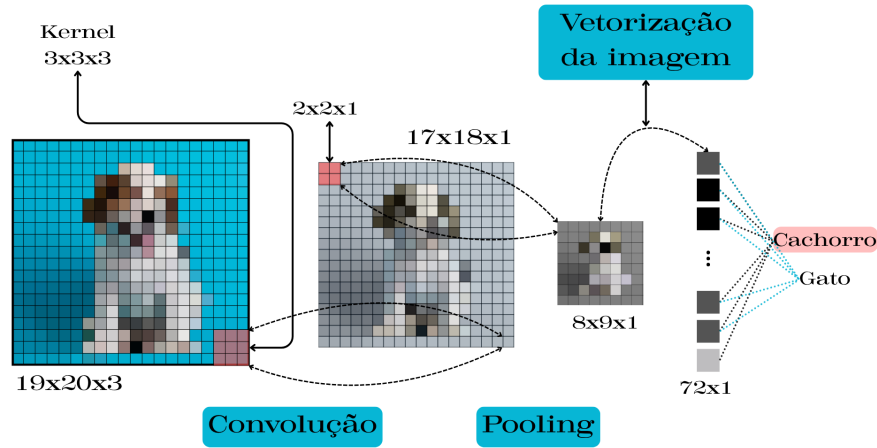
em que k é a dimensão do *kernel* de *pooling*, e $\lfloor \cdot \rfloor$ denota a função piso, que arredonda para baixo o resultado da divisão. Como consequência imediata da aplicação do *pooling*, a informação fica mais condensada, o que pode levar a uma perda de detalhes finos na imagem. No entanto, essa perda é compensada pela capacidade de aumentar a quantidade dos mapas de característica da imagem, melhorando sua capacidade de generalização em tarefas de reconhecimento de padrões.

Exemplo 1.7. Na Figura 1.24 é apresentada uma arquitetura típica de uma CNN, com uma camada de convolução e uma camada de *pooling*. Como a imagem têm três canais (RGB) de dimensões $19 \times 20 \times 3$ o *kernel* de convolução também tem três canais ($3 \times 3 \times 3$), a aplicação da convolução resulta em um mapa de características com um canal ($17 \times 18 \times 1$). O mapa de características é então processado por uma camada de *pooling* ($2 \times 2 \times 1$), que reduz suas dimensões espaciais ($8 \times 9 \times 1$). Após essa etapa a rede transforma o mapa de características em um vetor unidimensional (72×1), que é processado por camadas totalmente conectadas, visto na Seção 1.2 para realizar a classificação do objeto em cachorro ou gato, com a rede escolhendo a classe com maior probabilidade como resultado final (cachorro).

1.3.9 Convolução Transposta

A convolução transposta, também conhecida como deconvolução, é uma operação utilizada em redes neurais convolucionais para aumentar as dimensões espaciais (largura e altura) dos mapas de características. O nome sugere uma operação inversa à convolução, entretanto não é isso

Figura 1.24: Arquitetura típica de uma Rede Neural Convolucional com uma camada de convolução e uma camada de *pooling*, em que é enviado para rede a imagem de um cachorro e é feita a classificação do objeto na imagem.



Fonte: Elaboração do Autor.

que acontece. Esse nome provavelmente veio da ideia de “inverter” a operação de convolução no sentido de expandir as dimensões espaciais da entrada e não diminuir. Essa operação é frequentemente empregada em tarefas de geração de imagens, em que o objetivo é reconstruir imagens a partir de representações compactas. A convolução transposta funciona de maneira inversa à convolução válida. Enquanto a convolução reduz as dimensões espaciais da imagem de entrada, a convolução transposta expande essas dimensões. Isso é alcançado gerando uma saída com dimensões calculadas da seguinte forma:

$$\begin{aligned} w_{out} &= (w_{in} - 1) \cdot s_w - 2w_{pad} + w_{kernel}, \\ h_{out} &= (h_{in} - 1) \cdot s_h - 2h_{pad} + h_{kernel}, \end{aligned} \quad (1.31)$$

em que w_{in} e h_{in} são a largura e a altura da imagem de entrada, s_w e s_h são os valores do *stride* na largura e na altura, w_{pad} e h_{pad} são a quantidade de pixels adicionados como *padding* na largura e na altura (geralmente os valores para o *padding* são zero), e w_{kernel} e h_{kernel} são a largura e a altura do *kernel*, respectivamente. Cada entrada da imagem de saída é calculada como uma soma ponderada dos valores da imagem de entrada multiplicados pelos pesos do *kernel*. A convolução transposta pode ser implementada utilizando o seguinte Algoritmo 4.

Na Figura 1.25 é exemplificada a operação de convolução transposta, em que uma matriz de entrada X (matriz 2×2) é operada com um *kernel* W (matriz 2×2) para produzir a saída Y (matriz 3×3).

Entrada: X (matriz de entrada), W (*kernel*), s_w (*stride* na largura), s_h (*stride* na altura), w_{pad} (*padding* na largura), h_{pad} (*padding* na altura)

Saída: Y (matriz de saída)

$w_{out} \leftarrow (w_{in} - 1) \cdot s_w - 2w_{pad} + w_{kernel};$

$h_{out} \leftarrow (h_{in} - 1) \cdot s_h - 2h_{pad} + h_{kernel};$

$Y \leftarrow$ matriz de zeros de dimensão $w_{out} \times h_{out};$

Para $i \leftarrow 0$ **Até** $w_{in} - 1$ **Faça**

Para $j \leftarrow 0$ **Até** $h_{in} - 1$ **Faça**

Para $m \leftarrow 0$ **Até** $w_{kernel} - 1$ **Faça**

Para $n \leftarrow 0$ **Até** $h_{kernel} - 1$ **Faça**

$Y[i \cdot s_w + m - w_{pad}][j \cdot s_h + n - h_{pad}] += X[i][j] \cdot W[m][n];$

Fim

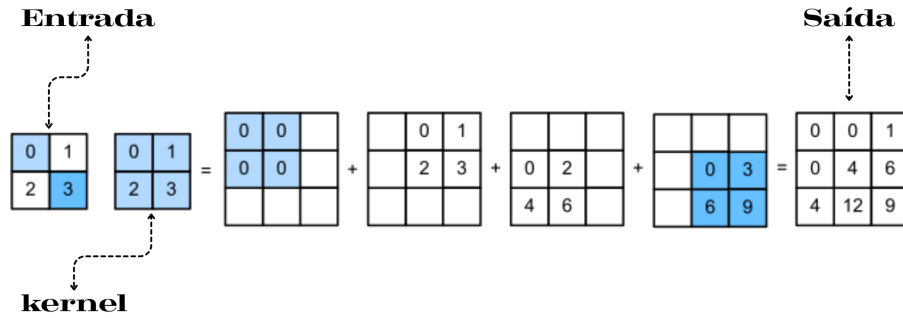
Fim

Fim

Fim

Algoritmo 4: Convolução Transposta.

Figura 1.25: Exemplo da operação de convolução transposta.



Fonte: Elaboração do Autor.

As cores (por exemplo, azul claro) na imagem de entrada representam a posição do pixel e quais casas ele influencia na operação de convolução transposta.

Desta forma, este capítulo abordou desde os conceitos fundamentais das unidades até a definição formal da operação de convolução, passando por exemplos e aplicações de redes neurais. A compreensão desses conceitos é essencial para o desenvolvimento de modelos de aprendizado profundo, especialmente em tarefas relacionadas ao processamento de imagens e reconhecimento de padrões.

Capítulo 2

Teoria dos Conjuntos Fuzzy

No cotidiano, o ser humano raramente raciocina por categorias absolutamente rígidas. Ao avaliar se um dia está quente, se uma pessoa é alta ou se uma imagem está nítida, por exemplo, normalmente não é pensado em termos de “pertence” ou “não pertence”, como estabelece a lógica clássica dos conjuntos. Em vez disso, é pensado em graus: algo pode ser um pouco quente, muito quente, relativamente alto ou razoavelmente nítido. É justamente essa forma mais gradual e flexível de interpretar a realidade que motiva a teoria dos conjuntos fuzzy. Ao permitir que um elemento pertença a um conjunto com diferentes graus de pertinência, essa abordagem se torna mais adequada para representar incertezas, imprecisões e transições suaves entre categorias. Neste capítulo, são apresentados os principais conceitos da teoria dos conjuntos fuzzy e dos sistemas baseados em regras fuzzy, que são utilizadas no desenvolvimento deste trabalho.

A teoria dos conjuntos fuzzy, introduzida por Lotfi A. Zadeh em 1965 (ZADEH, 1965), é uma extensão da teoria clássica dos conjuntos, permitindo a representação de incerteza e imprecisão. Diferentemente dos conjuntos clássicos, em que um elemento pertence ou não a um conjunto, nos conjuntos fuzzy cada elemento possui um grau de pertinência, variando entre 0 e 1. Essa abordagem é especialmente útil em análise de imagens, em que as características dos objetos podem ser ambíguas ou imprecisas, dependendo do contexto, e a classificação binária pode não ser suficiente para capturar a complexidade dos dados. As definições e propriedades dos conjuntos fuzzy presentes neste capítulo foram baseadas em Jafelice, Barros e Bassanezi (2023).

2.1 Conjuntos Fuzzy

Um *subconjunto fuzzy* A de um conjunto universo X é definido por uma função de pertinência μ_A que atribui a cada elemento $x \in X$ um valor $\mu_A(x) \in [0, 1]$, representando o grau de pertencimento de x ao conjunto A . Assim:

$$\mu_A : X \rightarrow [0, 1].$$

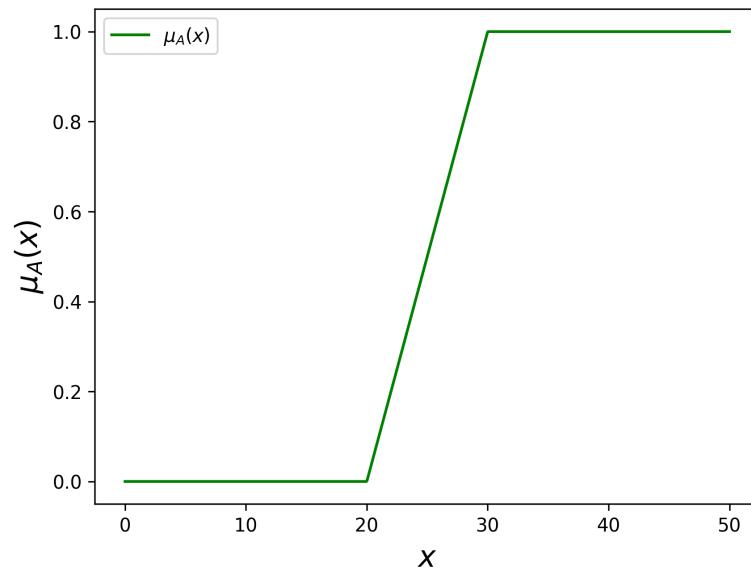
O valor $\mu_A(x) = 0$ indica que x não pertence ao conjunto A , enquanto $\mu_A(x) = 1$ indica que x pertence completamente ao conjunto. Valores intermediários representam graus parciais de pertencimento, permitindo uma representação mais rica e flexível de conceitos vagos ou imprecisos.

Exemplo 2.1. Considere o conjunto universo X representando a temperatura em graus Celsius, e o conjunto fuzzy A representando a categoria “temperatura alta”. A função de pertinência μ_A pode ser definida da seguinte forma:

$$\mu_A(x) = \begin{cases} 0 & \text{se } 0^\circ\text{C} \leq x \leq 20^\circ\text{C} \\ \frac{x-20}{10} & \text{se } 20^\circ\text{C} < x < 30^\circ\text{C} \\ 1 & \text{se } x \geq 30^\circ\text{C} \end{cases} \quad (2.1)$$

Neste exemplo, temperaturas de 20°C ou menos não são consideradas altas ($\mu_A(x) = 0$), enquanto temperaturas de 30°C ou mais são consideradas completamente altas ($\mu_A(x) = 1$). Temperaturas entre 20°C e 30°C têm um grau de pertinência que varia linearmente entre 0 e 1, refletindo a natureza gradual do conceito de temperatura alta. Na Figura 2.1 é ilustrado o gráfico da função de pertinência $\mu_A(x)$. Pode-se observar que a função de pertinência $\mu_A(x)$ é

Figura 2.1: Função de pertinência $\mu_A(x)$ para o conjunto fuzzy “temperatura alta”.



uma função crescente para valores entre 20°C e 30°C , refletindo a ideia de que à medida que a temperatura aumenta, o grau de pertencimento ao conjunto fuzzy “temperatura alta” também aumenta.

2.1.1 Operações entre Conjuntos Fuzzy

Os conceitos e definições que sucedem este capítulo são essenciais para a compreensão da U-Net Fuzzy, desenvolvida posteriormente. As operações entre conjuntos fuzzy, como união,

interseção e complemento, são definidas de maneira a preservar a estrutura fuzzy. A união de dois conjuntos clássicos A e B é dada por:

$$A \cup B = \{x \in U; x \in A \text{ ou } x \in B\},$$

enquanto a interseção é definida por:

$$A \cap B = \{x \in U; x \in A \text{ e } x \in B\}.$$

O complemento de um conjunto clássico A é dado por:

$$\bar{A} = \{x \in U; x \notin A\}.$$

Essas três operações têm respectivamente as funções características, $\forall x \in X$

$$u_{A \cup B}(x) = \max(u_A(x), u_B(x)), \quad u_{A \cap B}(x) = \min(u_A(x), u_B(x)), \quad u_{\bar{A}}(x) = 1 - u_A(x).$$

A união, interseção e complemento de conjuntos fuzzy são definidas de maneira análoga, utilizando as funções de pertinência.

Definição 2.1. *Sejam A e B conjuntos fuzzy em um conjunto universo X . A união, interseção e complemento de A e B , denotadas por $A \cup B$, $A \cap B$ e A' , respectivamente, são as funções de pertinência, presentes na Figura 2.2, definidas por:*

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x)) \quad \text{para todo } x \in X,$$

$$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x)) \quad \text{para todo } x \in X,$$

$$\mu_{\bar{A}}(x) = 1 - \mu_A(x) \quad \text{para todo } x \in X.$$

De fato, observa-se que a definição (2.1) é uma extensão natural das operações clássicas de união, interseção e complemento para o contexto dos conjuntos fuzzy. Nos conjuntos clássicos, a função de pertinência assume apenas valores binários, ou seja:

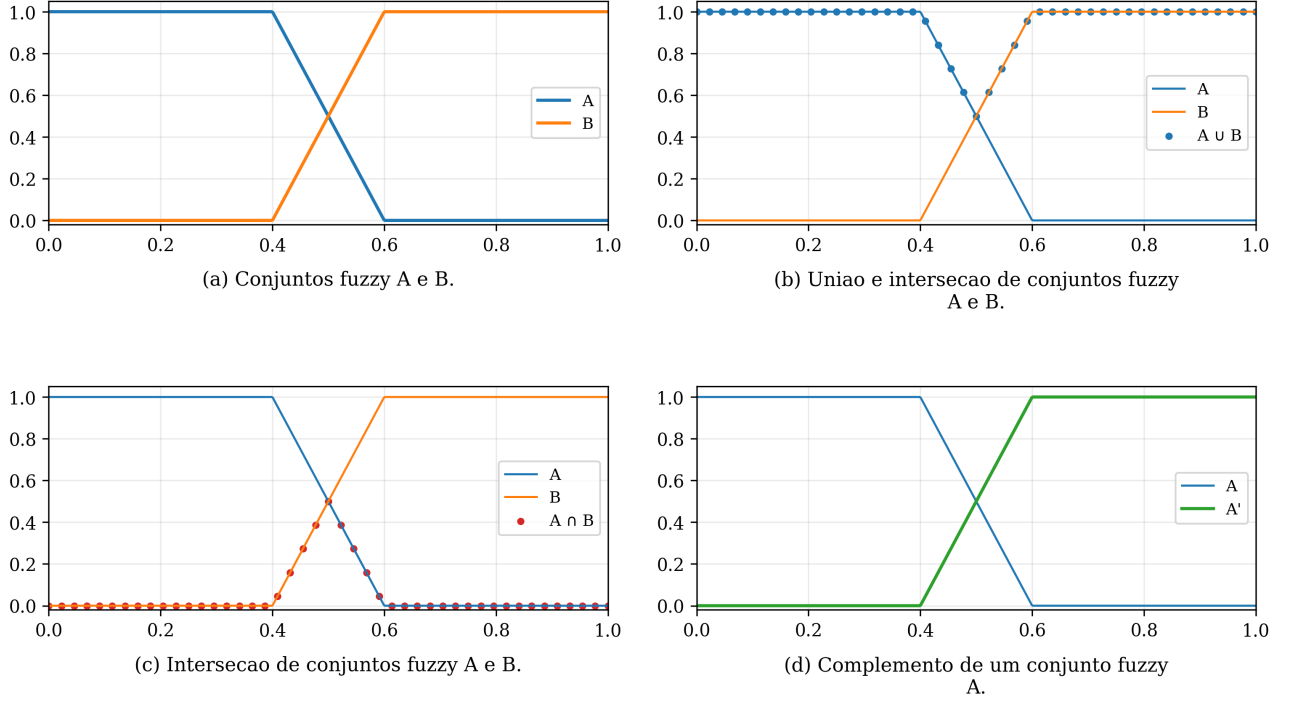
$$\mu_A(x) = \begin{cases} 1, & \text{se } x \in A \\ 0, & \text{se } x \notin A \end{cases}.$$

Assim, se $A = \{1, 2, 3\}$ e $B = \{3, 4, 5\}$, é obtida a seguinte tabela:

x	$\mu_A(x)$	$\mu_B(x)$	$\mu_{A \cup B}(x)$	$\mu_{A \cap B}(x)$	$\mu_{\bar{A}}(x)$
1	1	0	1	0	0
2	1	0	1	0	0
3	1	1	1	1	0
4	0	1	1	0	1
5	0	1	1	0	1

Observa-se que os valores obtidos para $\mu_{A \cup B}(x)$, $\mu_{A \cap B}(x)$ e $\mu_{\bar{A}}(x)$ correspondem exatamente à definição (2.1) de união, interseção e complemento, respectivamente. Na Figura 2.2 são ilustrados os gráficos das funções de pertinência para a união, interseção e complemento dos conjuntos fuzzy A e B . Os pontos presentes nos gráficos da Figura 2.2(a) e Figura 2.2(b) repre-

Figura 2.2: Operações entre conjuntos fuzzy.



Fonte: Adaptado de (JAFELICE; BARROS; BASSANEZI, 2023).

sentam a região dos valores de $\mu_A(x)$ e $\mu_B(x)$ obtidos após as operações de união e interseção, respectivamente.

2.1.2 Normas Triangulares

As normas triangulares são generalizações dos operadores união e intersecção.

Definição 2.2. Uma co-norma triangular (*s-norma*) é uma operação binária $S : [0, 1] \times [0, 1] \rightarrow [0, 1]$ que satisfaz as seguintes propriedades:

- *Comutatividade:* $xSy = ySx$
- *Associatividade:* $xs(ySz) = (xSy)Sz$
- *Monotonicidade:* Se $x \leq x'$ e $y \leq y'$, então $xSy \leq x'Sy'$
- *Condição de fronteira:* $xS0 = x$ e $xS1 = 1$

Exemplo 2.2. União Padrão: $S : [0, 1] \times [0, 1] \rightarrow [0, 1]$ com $xSy = \max(x, y)$ é uma co-norma triangular.

De fato, $\max(x, y) = \max(y, x)$, $\max(x, \max(y, z)) = \max(\max(x, y), z)$, $\max(x, y) \leq \max(x', y')$ se $x \leq x'$ e $y \leq y'$, $\max(x, 0) = x$ e $\max(x, 1) = 1$ para todo $x \in [0, 1]$.

Definição 2.3. Uma norma triangular (*t-norma*) é uma operação binária $T : [0, 1] \times [0, 1] \rightarrow [0, 1]$ que satisfaz as seguintes propriedades:

- *Comutatividade:* $xTy = yTx$
- *Associatividade:* $xT(yTz) = (xTy)Tz$
- *Monotonicidade:* Se $x \leq x'$ e $y \leq y'$, então $xTy \leq x'Ty'$
- *Condição de fronteira:* $0Tx = 0$ e $1Tx = x$

Exemplo 2.3. Intersecção Padrão: $T : [0, 1] \times [0, 1] \rightarrow [0, 1]$ com $xTy = \min(x, y)$ é uma norma triangular.

De fato, observa-se que $\min(x, y) = \min(y, x)$, $\min(x, \min(y, z)) = \min(\min(x, y), z)$, $\min(x, y) \leq \min(x', y')$ se $x \leq x'$ e $y \leq y'$, $\min(0, x) = 0$ e $\min(1, x) = x$ para todo $x \in [0, 1]$.

2.1.3 Níveis de um Conjunto Fuzzy

Definição 2.4. Sejam A um conjunto fuzzy e $\alpha \in (0, 1]$. O α -nível de A , denotado por $[A]^\alpha$, é o conjunto clássico definido por:

$$[A]^\alpha = \{x \in X \mid \mu_A(x) \geq \alpha\}.$$

Definição 2.5. O suporte de um conjunto fuzzy A são todos os elementos de X que têm grau de pertinência maior que zero em A , ou seja:

$$\text{supp}(A) = \{x \in X \mid \mu_A(x) > 0\}.$$

Definição 2.6. O nível zero de um conjunto fuzzy A do universo X , em que X é um espaço topológico, é o fecho topológico do suporte de A , ou seja:

$$[A]^0 = \overline{\text{supp}(A)}.$$

Exemplo 2.4. Seja A o conjunto fuzzy representando a categoria “temperatura alta” do Exemplo 2.1, dada pela função de pertinência (2.1).

O α -nível de A para $\alpha = 0,5$ é dado por:

$$[A]^{0,5} = \{x \in \mathbb{R} \mid \mu_A(x) \geq 0,5\}.$$

Neste caso, tem-se:

$$\mu_A(x) \geq 0,5 \implies \frac{x - 20}{10} \geq 0,5 \implies x \geq 25^\circ\text{C}.$$

Portanto, o α -nível de A para $\alpha = 0,5$ é o intervalo $[25^\circ C, \infty)$, representando a “temperatura alta” com um grau de pertinência de pelo menos 0,5.

O suporte de A é dado por:

$$\mu_A(x) > 0 \implies \frac{x - 20}{10} > 0 \implies x > 20^\circ C.$$

Portanto, o suporte de A é o intervalo $(20^\circ C, \infty)$, representando a “temperatura alta” que tem um grau de pertinência maior que zero em A .

2.1.4 Números Fuzzy

Um número fuzzy é um subconjunto fuzzy do conjunto dos números reais, caracterizado por uma função de pertinência que atribui a cada número real um grau de pertencimento. Os números fuzzy são frequentemente utilizados para representar quantidades imprecisas ou vagamente definidas.

Definição 2.7. Um conjunto fuzzy N é chamado de número fuzzy quando o conjunto universo, em que N é definido, é o conjunto dos números reais $\mathbb{R}$, e a função de pertinência μ_N é tal que:

- $\mu_N(x) = 1$ para pelo menos um valor x de $\text{supp}(N)$;
- $[N]^\alpha$ é um intervalo fechado para todo $\alpha \in (0, 1]$;
- O suporte de N é limitado.

Exemplo 2.5. Um número fuzzy A é dito trapezoidal se o gráfico de sua função de pertinência tem a forma de um trapézio. A função de pertinência de um número fuzzy trapezoidal pode ser definida por quatro parâmetros a, b, c, d (com $a \leq b \leq c \leq d$) da seguinte forma:

$$\mu_A(x) = \begin{cases} \frac{x-a}{b-a}, & \text{se } a < x < b \\ 1, & \text{se } b \leq x < c \\ \frac{d-x}{d-c}, & \text{se } c \leq x < d \\ 0, & \text{caso contrário} \end{cases}$$

Neste exemplo, o número fuzzy A é representado por um trapézio definido pelos parâmetros a, b, c, d . Para $a < x < b$, a função de pertinência aumenta linearmente de 0 a 1, indicando um aumento gradual no grau de pertencimento. Para $b \leq x < c$, a função de pertinência é constante em 1, indicando que esses valores pertencem completamente ao conjunto fuzzy. Para $c \leq x < d$, a função de pertinência diminui linearmente de 1 a 0, indicando uma diminuição gradual no grau de pertencimento.

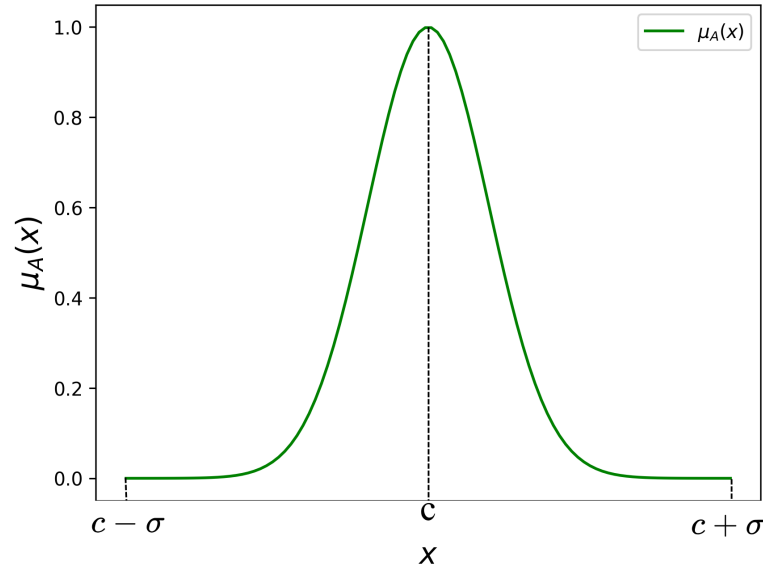
Exemplo 2.6. Um número fuzzy A é dito gaussiano se o gráfico de sua função de pertinência tem a forma de uma curva gaussiana, dada por:

mudar aqui

$$\mu_A(x) = \begin{cases} e^{-\frac{(x-c)^2}{2\sigma^2}}, & \text{se } c - \sigma \leq x \leq c + \sigma, \\ 0, & \text{caso contrário} \end{cases}, \quad (2.2)$$

em que c é o centro do número fuzzy e σ é o desvio padrão, que controla a largura da curva. Na Figura 2.3 é ilustrado o gráfico da função de pertinência de um número fuzzy gaussiano. O

Figura 2.3: Função de pertinência de um número fuzzy gaussiano.



número fuzzy gaussiano é caracterizado por uma função de pertinência suave e contínua, que atinge seu valor máximo em $x = c$ e decresce à medida que x se afasta de c . Observa-se que o gráfico da função é limitado para o intervalo $[c - \sigma, c + \sigma]$, para que seja satisfeita a condição de suporte limitado.

2.2 Sistema Baseado em Regras Fuzzy

Para que seja possível mesclar os conceitos de redes neurais com a teoria fuzzy, é necessário compreender a estrutura e o funcionamento de um Sistema Baseado em Regras Fuzzy - SBRF. Um SBRF é um sistema que utiliza a teoria dos conjuntos fuzzy para produzir saídas a partir de entradas dadas por conjuntos fuzzy (JAFELICE; BARROS; BASSANEZI, 2023). Tais sistemas foram desenvolvidos com o objetivo de imitar o comportamento humano considerando que o conhecimento e as ações humanas são apoiadas em uma sequência de regras linguísticas, traduzidas por um conjunto de regras. As regras formuladas pelo ser humano são usualmente da forma:

SE (um conjunto de condições é satisfeito) *ENTÃO* (um conjunto de consequências pode ser inferido).

Para que seja possível formalizar as regras fuzzy, é necessário definir o conceito de relação fuzzy.

Definição 2.8. *Uma relação fuzzy R é um subconjunto fuzzy do produto cartesiano entre conjuntos universos $X_1, X_2, \dots, X_n$, ou seja, $R \subseteq X_1 \times X_2 \times \dots \times X_n$. O produto cartesiano fuzzy $A_1 \times A_2 \times \dots \times A_n$ dos subconjuntos fuzzy $A_1, A_2, \dots, A_n$ de $X_1, X_2, \dots, X_n$, respectivamente, é a relação fuzzy R cuja função de pertinência é dada por:*

$$\mu_R(x_1, x_2, \dots, x_n) = \min(\mu_{A_1}(x_1), \mu_{A_2}(x_2), \dots, \mu_{A_n}(x_n)).$$

2.2.1 Regras e Inferência Fuzzy

Uma regra fuzzy é uma sentença da forma “SE X é A ENTÃO Y é B ”, em que A e B são conjuntos fuzzy em X e Y , respectivamente. Tal regra pode ser interpretada como uma relação fuzzy R entre os conjuntos A e B cuja a função de pertinência $\mu_R(x, y)$ depende de $\mu_A(x)$ e $\mu_B(y)$ para cada $(x, y) \in X \times Y$. Em geral utiliza-se a função:

$$\mu_R(x, y) = \min(\mu_A(x), \mu_B(y)).$$

Na maioria dos casos, as regras fuzzy são formuladas em termos de variáveis linguísticas, ou seja, variáveis que assumem valores qualitativos representados por termos linguísticos, como por exemplo, “SE a área é grande ENTÃO a produção é alta”.

Definição 2.9. *Uma variável linguística é uma variável cujo o valor é expresso qualitativamente por termos linguísticos, que são representados por conjuntos fuzzy e quantitativamente por uma função de pertinência.*

Por exemplo, a variável linguística “área” pode assumir os termos linguísticos “pequena”, “média” e “grande” representados por conjuntos fuzzy com funções de pertinência que descrevem o grau de pertencimento de cada valor de área a esses termos.

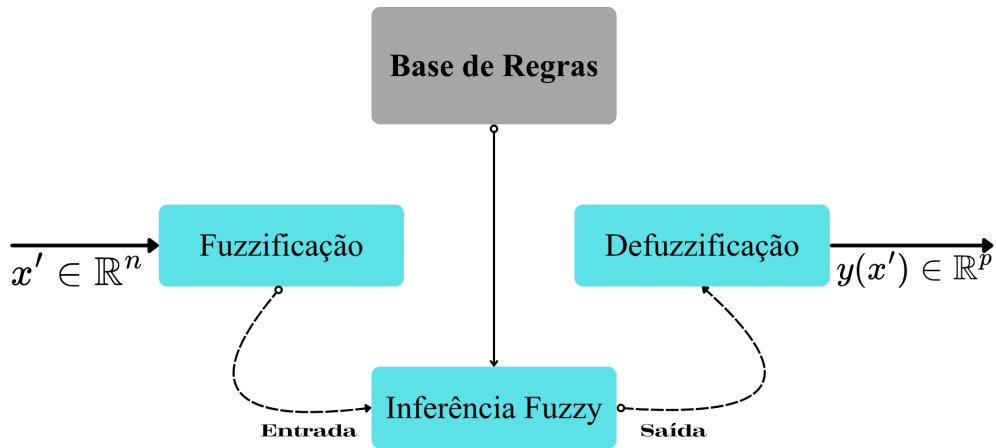
2.2.2 Componentes de um SBRF

Um SBRF é composto por quatro componentes: a fuzzificação dos dados de entrada, uma coleção de regras, chamada base de regras, um método de inferência fuzzy e a defuzzificação, que fornece um número real como saída. Estes componentes são conectados como mostrado na Figura 2.4.

O SBRF pode ser visto com um mapeamento entre a entrada e a saída da forma $y = f(x)$, $x \in \mathbb{R}^n$ e $y \in \mathbb{R}^p$. Os componentes do SBRF realizam as seguintes funções:

- A **fuzzificação** realiza a transformação dos valores de entrada em conjuntos fuzzy com seus respectivos domínios. A atuação de um especialista é necessária para definir as funções de pertinência que são utilizadas na fuzzificação, bem como os termos linguísticos que são associados a cada variável de entrada.
- A **base de regras** contém um conjunto de regras fuzzy que descrevem o comportamento do sistema, essas regras são formuladas por especialistas com base em seu conhecimento

Figura 2.4: Arquitetura de um Sistema Baseado em Regras Fuzzy (SBRF).



Fonte: Adaptado de Jafelice, Barros e Bassanezi (2023).

sobre o domínio do problema. A base de regras descreve relações entre as variáveis linguísticas para serem utilizadas pelo método de inferência fuzzy.

- O **método de inferência fuzzy** fornece a saída a partir de cada entrada fuzzy e da relação definida pela base de regras. O método de inferência fuzzy pode ser visto como um mecanismo de mapeamento entre os conjuntos fuzzy de entrada e os conjuntos fuzzy de saída, utilizando as regras da base de regras para inferir a saída correspondente a cada entrada. Neste trabalho é utilizado o método de inferência de Takagi-Sugeno, que é um dos métodos mais comuns para sistemas baseados em regras fuzzy com variáveis linguísticas.

- **Método de Takagi-Sugeno:** Neste caso, o consequente de cada regra é uma função das variáveis de entrada. Por exemplo, pode-se supor que a função que mapeia a entrada e a saída para cada regra é uma combinação linear das entradas, ou seja:

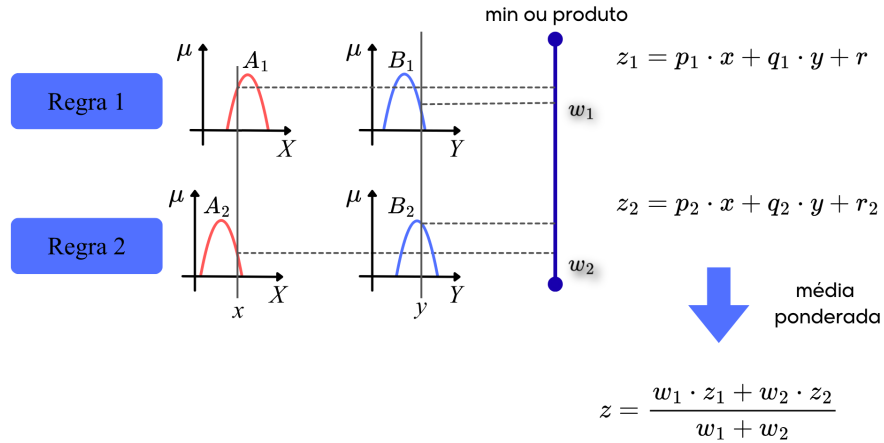
$$z = a_1x_1 + a_2x_2 + \dots + a_nx_n + b.$$

Assim, as regras do método de Takagi-Sugeno podem ser formuladas da seguinte forma:

Regra r : SE x_1 é A_1 E x_2 é $A_2 \dots x_n$ é A_n ENTAO $z = f_r(x_1, x_2, \dots, x_n)$.

Na Figura 2.5 é ilustrado um exemplo do método de Takagi-Sugeno, em que duas regras são aplicadas para inferir a saída z a partir das entradas x e y .

Figura 2.5: Exemplo de inferência fuzzy utilizando o método de Takagi-Sugeno.



Fonte: Adaptado de Jafelice, Barros e Bassanezi (2023).

- A **defuzzificação** converte os conjuntos fuzzy de saída em números reais. Sistemas fuzzy, em geral, a saída é um conjunto fuzzy, assim, deve-se escolher um método de defuzzificação para obter um número real a partir do conjunto fuzzy de saída. Para o método de Takagi-Sugeno, o método de defuzzificação é dado pela média ponderada das saídas de cada regra, usando-se o grau de ativação destas regras como ponderação, ou seja:

$$z = \frac{\sum_{r=1}^R w_r f_r(x_1, x_2, \dots, x_n)}{\sum_{r=1}^R w_r},$$

em que R é o número de regras, w_r é o grau de ativação da regra r e $f_r(x_1, x_2, \dots, x_n)$ é a função de saída da regra r . É possível observar o cálculo na Figura 2.5 para obter “ z ”.

Exemplo 2.7. Considere um sistema de controle de velocidade de um ventilador com duas variáveis de entrada: temperatura x (em °C) e umidade do ar y (em %). O objetivo é determinar a velocidade z do ventilador (em Rotações por Minuto - RPM) utilizando um SBRF com o método de Takagi-Sugeno.

As variáveis linguísticas são definidas pelas seguintes funções de pertinência trapezoidais:

- Temperatura “Baixa” (A_1):

$$\mu_{A_1}(x) = \begin{cases} 1, & \text{se } x \leq 20 \\ \frac{25-x}{5}, & \text{se } 20 < x < 25 \\ 0, & \text{se } x \geq 25 \end{cases}$$

- Temperatura “Média” (A_2):

$$\mu_{A_2}(x) = \begin{cases} 0, & \text{se } x < 20 \text{ ou } x \geq 35 \\ \frac{x-20}{5}, & \text{se } 20 \leq x < 25 \\ 1, & \text{se } 25 \leq x < 30 \\ \frac{35-x}{5}, & \text{se } 30 \leq x < 35 \end{cases}.$$

- Temperatura “Alta” (A_3):

$$\mu_{A_3}(x) = \begin{cases} 0, & \text{se } x < 30 \\ \frac{x-30}{5}, & \text{se } 30 \leq x < 35 \\ 1, & \text{se } x \geq 35 \end{cases}.$$

- Umidade “Baixa” (B_1):

$$\mu_{B_1}(y) = \begin{cases} 1, & \text{se } y < 20 \\ \frac{40-y}{20}, & \text{se } 20 \leq y < 40 \\ 0, & \text{se } y \geq 40 \end{cases}.$$

- Umidade “Média” (B_2):

$$\mu_{B_2}(y) = \begin{cases} 0, & \text{se } y < 20 \text{ ou } y \geq 80 \\ \frac{y-20}{20}, & \text{se } 20 \leq y < 40 \\ 1, & \text{se } 40 \leq y < 60 \\ \frac{80-y}{20}, & \text{se } 60 \leq y < 80 \end{cases}.$$

- Umidade “Alta” (B_3):

$$\mu_{B_3}(y) = \begin{cases} 0, & \text{se } y < 60 \\ \frac{y-60}{20}, & \text{se } 60 \leq y < 80 \\ 1, & \text{se } y \geq 80 \end{cases}.$$

As regras fuzzy pelo método de Takagi-Sugeno são:

$$\left\{ \begin{array}{l} \textbf{Regra 1} : \text{SE } (x \text{ é } A_1 \text{ E } y \text{ é } B_1) \text{ ENTÃO } z_1 = 5x + 2y; \\ \textbf{Regra 2} : \text{SE } (x \text{ é } A_2 \text{ E } y \text{ é } B_2) \text{ ENTÃO } z_2 = x + y; \\ \textbf{Regra 3} : \text{SE } (x \text{ é } A_3 \text{ E } y \text{ é } B_3) \text{ ENTÃO } z_3 = 2x + 3y; \\ \textbf{Regra 4} : \text{SE } (x \text{ é } A_1 \text{ E } y \text{ é } B_2) \text{ ENTÃO } z_4 = 3x + y; \\ \textbf{Regra 5} : \text{SE } (x \text{ é } A_2 \text{ E } y \text{ é } B_1) \text{ ENTÃO } z_5 = 4x + 2y; \\ \textbf{Regra 6} : \text{SE } (x \text{ é } A_3 \text{ E } y \text{ é } B_1) \text{ ENTÃO } z_6 = 2x + y; \\ \textbf{Regra 7} : \text{SE } (x \text{ é } A_1 \text{ E } y \text{ é } B_3) \text{ ENTÃO } z_7 = 3x + 4y; \\ \textbf{Regra 8} : \text{SE } (x \text{ é } A_2 \text{ E } y \text{ é } B_3) \text{ ENTÃO } z_8 = 4x + 3y; \\ \textbf{Regra 9} : \text{SE } (x \text{ é } A_3 \text{ E } y \text{ é } B_2) \text{ ENTÃO } z_9 = 5x + y \end{array} \right. .$$

Para as entradas $x = 35^\circ\text{C}$ e $y = 70\%$, os graus de pertinência são:

$$\mu_{A_1}(35) = 0, \quad \mu_{A_2}(35) = 0, \quad \mu_{A_3}(35) = 1.$$

$$\mu_{B_1}(70) = 0, \quad \mu_{B_2}(70) = 0,5, \quad \mu_{B_3}(70) = 0,5.$$

Os graus de ativação de cada regra, utilizando o operador do mínimo, são:

$$\left\{ \begin{array}{l} w_1 = \min(\mu_{A_1}(35), \mu_{B_1}(70)) = \min(0; 0) = 0; \\ w_2 = \min(\mu_{A_2}(35), \mu_{B_2}(70)) = \min(0; 0,5) = 0; \\ w_3 = \min(\mu_{A_3}(35), \mu_{B_3}(70)) = \min(1; 0,5) = 0,5; \\ w_4 = \min(\mu_{A_1}(35), \mu_{B_2}(70)) = \min(0; 0,5) = 0; \\ w_5 = \min(\mu_{A_2}(35), \mu_{B_1}(70)) = \min(0; 0) = 0; \\ w_6 = \min(\mu_{A_3}(35), \mu_{B_1}(70)) = \min(1; 0) = 0; \\ w_7 = \min(\mu_{A_1}(35), \mu_{B_3}(70)) = \min(0; 0,5) = 0; \\ w_8 = \min(\mu_{A_2}(35), \mu_{B_3}(70)) = \min(0; 0,5) = 0; \\ w_9 = \min(\mu_{A_3}(35), \mu_{B_2}(70)) = \min(1; 0,5) = 0,5. \end{array} \right.$$

As saídas de cada regra são:

$$\left\{ \begin{array}{l} z_1 = 5 \cdot 35 + 2 \cdot 70 = 315; \\ z_2 = 35 + 70 = 105; \\ z_3 = 2 \cdot 35 + 3 \cdot 70 = 280; \\ z_4 = 3 \cdot 35 + 70 = 175; \\ z_5 = 4 \cdot 35 + 2 \cdot 70 = 280; \\ z_6 = 2 \cdot 35 + 70 = 140; \\ z_7 = 3 \cdot 35 + 4 \cdot 70 = 385; \\ z_8 = 4 \cdot 35 + 3 \cdot 70 = 350; \\ z_9 = 5 \cdot 35 + 70 = 245. \end{array} \right.$$

A saída final é obtida pela média ponderada:

$$\begin{aligned} z &= \frac{w_1 \cdot z_1 + w_2 \cdot z_2 + w_3 \cdot z_3 + w_4 \cdot z_4 + w_5 \cdot z_5 + w_6 \cdot z_6 + w_7 \cdot z_7 + w_8 \cdot z_8 + w_9 \cdot z_9}{w_1 + w_2 + w_3 + w_4 + w_5 + w_6 + w_7 + w_8 + w_9} = \\ &= \frac{0 \cdot 315 + 0 \cdot 105 + 0,5 \cdot 280 + 0 \cdot 175 + 0 \cdot 280 + 0 \cdot 140 + 0 \cdot 385 + 0 \cdot 350 + 0,5 \cdot 245}{0 + 0 + 0,5 + 0 + 0 + 0 + 0 + 0 + 0,5} = \\ &= \frac{140 + 122,5}{1} = 262,5 \text{ RPM.} \end{aligned}$$

Portanto, para uma temperatura de 35°C e umidade do ar de 70%, o sistema baseado em regras fuzzy com o método de Takagi-Sugeno determina que a velocidade do ventilador deve ser de 262,5 RPM, uma boa velocidade para refrescar o ambiente.

2.3 ANFIS

O Sistema Adaptativo de Inferência Neuro-Fuzzy, do inglês *Adaptive Neuro-Fuzzy Inference System - ANFIS*, é uma técnica híbrida em inteligência artificial, que combina a capacidade de aprendizado das redes neurais artificiais com a representação de conhecimento e raciocínio aproximado dos sistemas baseados em regras fuzzy (JAFELICE; BARROS; BASSANEZI, 2023). Essa abordagem permite determinar um SBRF a partir de conhecimentos obtidos por meio de um processo de treinamento, utilizando um conjunto de dados de entrada e saída, sendo o principal método de inferência utilizado o método de Takagi-Sugeno (JANG, 1993).

O ANFIS é composto por cinco camadas, cada uma com uma função específica. Para exemplificar cada camada, considere o sistema com duas entradas, x_1 e x_2 , e uma saída y , com quatro regras fuzzy do tipo Takagi-Sugeno, ou seja:

Regra 1: SE x_1 é $A_1^{(1)}$ E x_2 é $A_1^{(2)}$ ENTÃO $y = f_{1,1}(x_1, x_2) = k_{10} + k_{11}x_1 + k_{12}x_2$;

Regra 2: SE x_1 é $A_1^{(1)}$ E x_2 é $A_2^{(2)}$ ENTÃO $y = f_{1,2}(x_1, x_2) = k_{20} + k_{21}x_1 + k_{22}x_2$;

Regra 3: SE x_1 é $A_2^{(1)}$ E x_2 é $A_1^{(2)}$ ENTÃO $y = f_{2,1}(x_1, x_2) = k_{30} + k_{31}x_1 + k_{32}x_2$;

Regra 4: SE x_1 é $A_2^{(1)}$ E x_2 é $A_2^{(2)}$ ENTÃO $y = f_{2,2}(x_1, x_2) = k_{40} + k_{41}x_1 + k_{42}x_2$.

em que $A_1^{(1)}, A_2^{(1)}$ são os conjuntos fuzzy do universo X_1 , associados à variável de entrada x_1 e $A_1^{(2)}, A_2^{(2)}$ são os conjuntos fuzzy do universo X_2 , associados à variável de entrada x_2 ; e k_{i0}, k_{i1}, k_{i2} são os parâmetros de cada regra, para $i = 1, 2, 3, 4$. Para este exemplo, considera-se que as funções de pertinência dos conjuntos fuzzy $A_j^{(1)}$ e $A_j^{(2)}$ são gaussianas, definidas pela Equação (2.2).

Na primeira camada os valores de entrada são transformados em graus de pertinência dos conjuntos fuzzy, ou seja:

$$\begin{cases} y_i^{(1)} = \mu_{A_i^{(1)}}(x_1); \\ y_{i+2}^{(1)} = \mu_{A_i^{(2)}}(x_2) \end{cases},$$

para $i = 1, 2$.

Na segunda camada, as regras fuzzy são criadas com base no operador lógico “E” definido por uma t-norma, ou seja:

$$\begin{cases} y_{1,1}^{(2)} = w_{1,1} = \mu_{A_1^{(1)}}(x_1) \cdot \mu_{A_1^{(2)}}(x_2); \\ y_{1,2}^{(2)} = w_{1,2} = \mu_{A_1^{(1)}}(x_1) \cdot \mu_{A_2^{(2)}}(x_2); \\ y_{2,1}^{(2)} = w_{2,1} = \mu_{A_2^{(1)}}(x_1) \cdot \mu_{A_1^{(2)}}(x_2); \\ y_{2,2}^{(2)} = w_{2,2} = \mu_{A_2^{(1)}}(x_1) \cdot \mu_{A_2^{(2)}}(x_2) \end{cases};$$

em que $w_{h,j}$ é o peso de ativação da regra associada à combinação $(A_h^{(1)}, A_j^{(2)})$, para $h, j = 1, 2$. Na terceira camada os pesos de cada regra são normalizados, ou seja:

$$y_{h,j}^{(3)} = \bar{w}_{h,j} = \frac{w_{h,j}}{w_{1,1} + w_{1,2} + w_{2,1} + w_{2,2}};$$

para $h, j = 1, 2$. Na quarta camada a parte “ENTÃO” das regras fuzzy são criadas, ou seja:

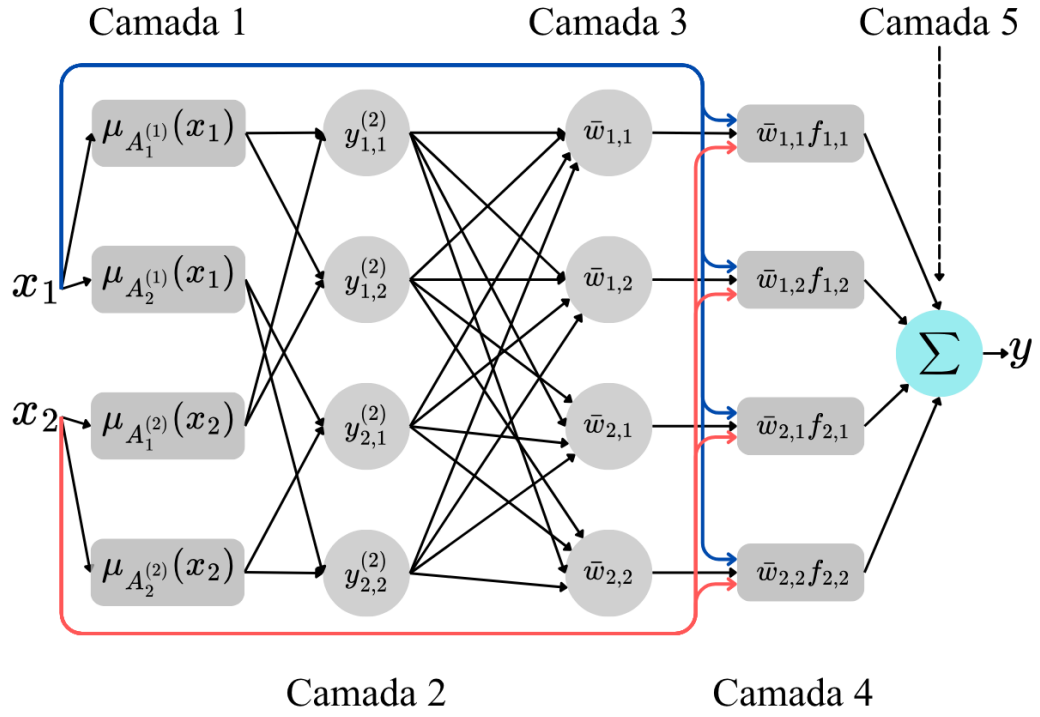
$$y_{h,j}^{(4)} = \bar{w}_{h,j} f_{h,j} = \bar{w}_{h,j} (k_{h0} + k_{h1}x_1 + k_{h2}x_2)$$

Para a última camada, a saída é obtida pela média ponderada:

$$y = \frac{\sum_{h=1}^2 \sum_{j=1}^2 \bar{w}_{h,j} f_{h,j}}{\sum_{h=1}^2 \sum_{j=1}^2 \bar{w}_{h,j}}.$$

Na Figura 2.6 é ilustrada a arquitetura do ANFIS para o exemplo apresentado. As setas na Figura 2.6 representam as conexões entre as camadas do ANFIS. Observa-se que para dois antecedentes, x_1 e x_2 , e dois conjuntos fuzzy para cada antecedente, $A_1^{(1)}, A_2^{(1)}$ e $A_1^{(2)}, A_2^{(2)}$, são criadas quatro regras fuzzy, ou seja, $n^k = 2^2 = 4$ regras fuzzy. Para um número geral de k antecedentes e n conjuntos fuzzy para cada antecedente, o número de regras fuzzy criadas é dado por n^k .

Figura 2.6: Arquitetura do ANFIS



Fonte: Adaptado de Jafelice, Barros e Bassanezi (2023).

Neste capítulo foram apresentados os principais conceitos relacionados à teoria fuzzy, iniciando com a definição de conjuntos fuzzy e terminando com a definição da ANFIS. No próximo capítulo, é introduzida a principal rede neural utilizada neste trabalho.

Capítulo 3

Estrutura das Redes Neurais U-Net e U-Net Fuzzy

Redes neurais convolucionais são frequentemente utilizadas em tarefas de classificação de imagens, nas quais o objetivo é atribuir uma classe à imagem como um todo. Entretanto, em muitas aplicações visuais, não basta apenas classificar a imagem inteira, é necessário também identificar com precisão a localização das estruturas de interesse. Nesses casos, utiliza-se a segmentação de imagens, isto é, o processo de classificar cada pixel da imagem em uma categoria específica. Para atender a essa necessidade, foi proposta a arquitetura U-Net (RONNEBERGER; FISCHER; BROX, 2015), uma rede neural convolucional desenvolvida especificamente para tarefas de segmentação. Sua estrutura é composta por duas partes principais: o *encoder*, responsável por extrair características da imagem de entrada, geralmente reduzindo suas dimensões espaciais e aumentando o número de canais, e o *decoder*, responsável por gerar a imagem a partir das características extraídas no *encoder*. Este processo permite a geração de uma máscara de segmentação, isto é, uma imagem que indica a presença ou ausência da região de interesse. Quando estas máscaras são desenvolvidas manualmente por especialistas, são conhecidas como *ground truth*. Essas máscaras são utilizadas como rótulos, ou seja, cada imagem enviada para treinamento da U-Net tem seu respectivo *ground truth*. Essa organização dá origem ao formato em “U”, que motivou o nome da arquitetura.

3.1 Noções Básicas da Arquitetura U-Net

Para compreender melhor a arquitetura da U-Net, é necessário entender as operações básicas envolvidas e como uma imagem é processada para gerar a máscara de segmentação. Para isso, o Exemplo 3.1 apresenta o funcionamento simplificado da U-Net, ilustrando as principais operações e etapas envolvidas no processo de segmentação de imagens. O exemplo a seguir é adaptado de TileStats (2025).

Exemplo 3.1. Considere uma imagem de entrada normalizada X de dimensões $5 \times 5 \times 1$ e um *ground truth* de dimensões $5 \times 5 \times 1$. O objetivo deste exemplo é mostrar, de forma simplificada, como uma U-Net transforma a imagem de entrada X no *ground truth*. Para isso, considera-se

uma arquitetura reduzida, composta por um *encoder* com uma camada de convolução e uma camada de *Max pooling*, e por um *decoder* com uma camada de convolução transposta e uma camada de convolução final.

Objetivo do exemplo. O exemplo ilustra as principais etapas do funcionamento da U-Net, desde a extração de características no *encoder* até a criação da máscara de segmentação no *decoder*. Como se trata de uma versão simplificada, o foco está na compreensão do fluxo de processamento, e não na complexidade completa da arquitetura original.

Simplificações adotadas. Para facilitar a compreensão, são adotadas as seguintes simplificações: a imagem é monocromática, com dimensões $5 \times 5 \times 1$; os pesos e os vieses já estão ajustados para o desenvolvimento da máscara de segmentação desejada. A rede possui apenas uma camada de convolução, uma camada de *Max pooling*, uma camada de convolução transposta e uma camada final de saída. As cores utilizadas nas imagens possuem apenas finalidade ilustrativa, servindo para destacar visualmente as operações realizadas. Elas não representam valores reais de pixel, uma vez que $5 \times 5 \times 1$ corresponde a uma imagem monocromática.

Na Figura 3.1 são apresentadas a imagem de entrada X e *ground truth*. Neste exemplo, os pixels com valor “1” representam a região de interesse, enquanto os pixels com valor “0” representam o fundo. Valores intermediários, como “0.5”, representam áreas de transição ou incerteza.

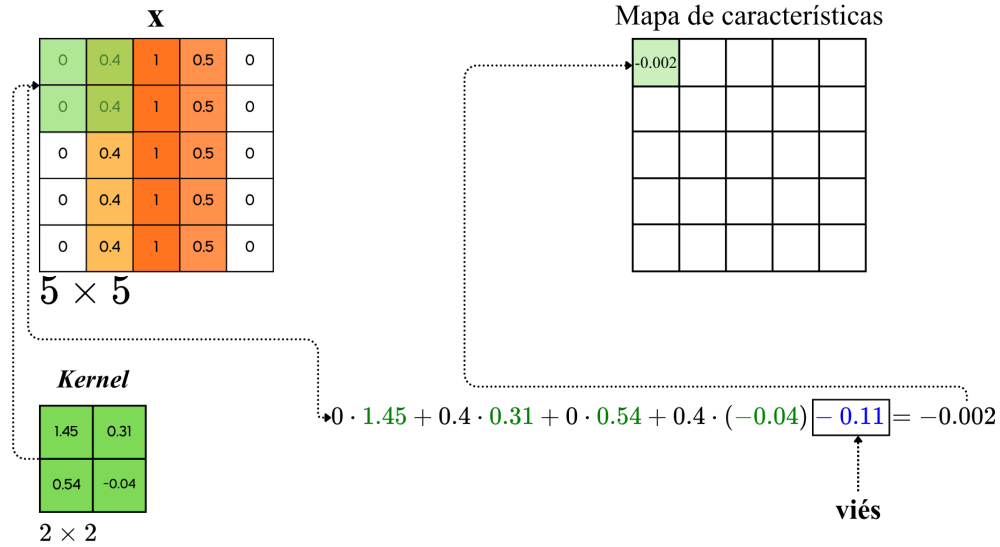
Figura 3.1: Imagem de entrada X e *ground truth*.

X					<i>Ground Truth</i>				
0	0.4	1	0.5	0	0	1	1	1	0
0	0.4	1	0.5	0	0	1	1	1	0
0	0.4	1	0.5	0	0	1	1	1	0
0	0.4	1	0.5	0	0	1	1	1	0
0	0.4	1	0.5	0	0	1	1	1	0
5×5					5×5				

Fonte: Elaboração do Autor.

Etapa 1: Convolução. A primeira operação realizada no *encoder* é a convolução, equação (1.27). Nessa etapa, um *kernel* de dimensões $2 \times 2 \times 1$ percorre a imagem de entrada X , combinando localmente os valores dos pixels, somando o viés e gerando um mapa de características, como mostrado na Figura 3.2.

Figura 3.2: Aplicação de um *kernel* de convolução na imagem de entrada X .

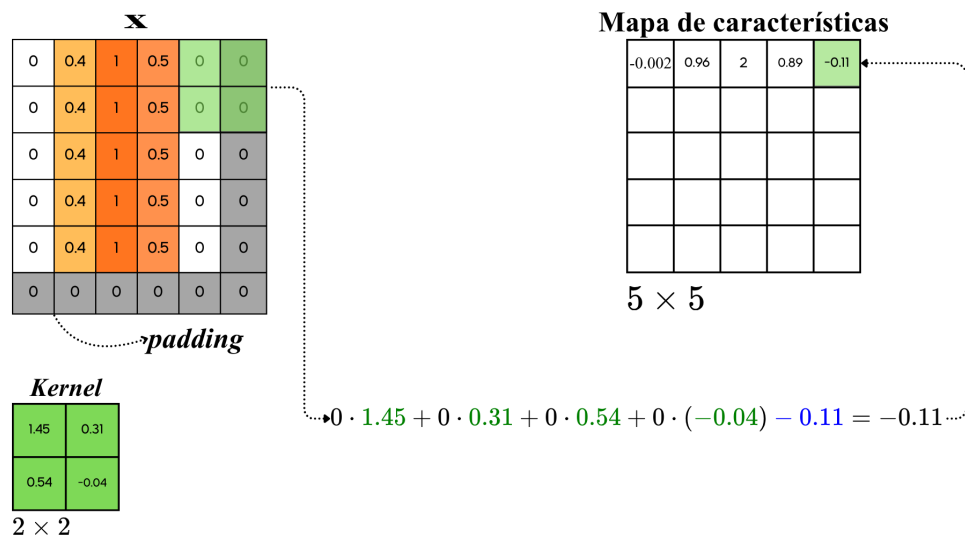


Fonte: Elaboração do Autor.

Na Figura 3.2, os cálculos indicam os valores da imagem de entrada cobertos pelo *kernel*. A soma ponderada desses valores, acrescida do viés, gera cada elemento do mapa de características.

Etapa 2: Padding. Para que o mapa de características resultante mantenha as mesmas dimensões espaciais da imagem de entrada, isto é, $5 \times 5 \times 1$, aplica-se a técnica de *padding*, Subseção 1.3.6, adicionando uma borda de zeros ao redor da imagem. Neste exemplo, para simplificar a visualização, o preenchimento foi adicionado apenas na parte inferior e à direita, como ilustrado em cinza na Figura 3.3.

Figura 3.3: Aplicação de *padding* na imagem de entrada X .

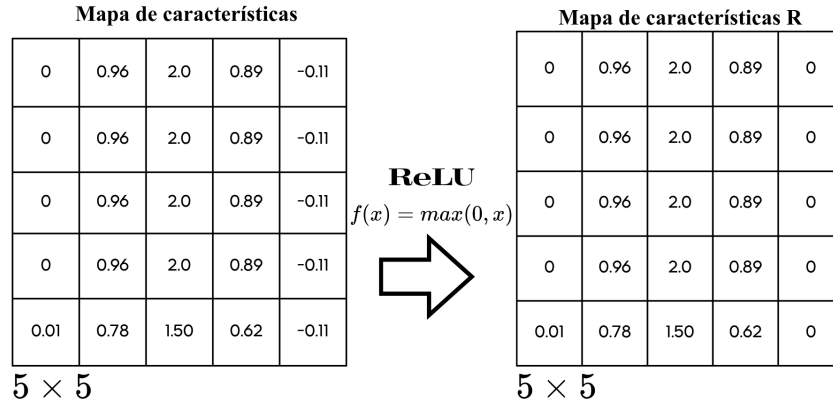


Fonte: Elaboração do Autor.

Etapa 3: Ativação ReLU. Após a convolução, é aplicada uma função de ativação a cada valor

do mapa de características. Neste exemplo, utiliza-se a função ReLU, apresentada na equação (1.4). Sua função é introduzir não linearidade no modelo, permitindo que a rede represente padrões mais complexos. O resultado dessa operação é mostrado na Figura 3.4.

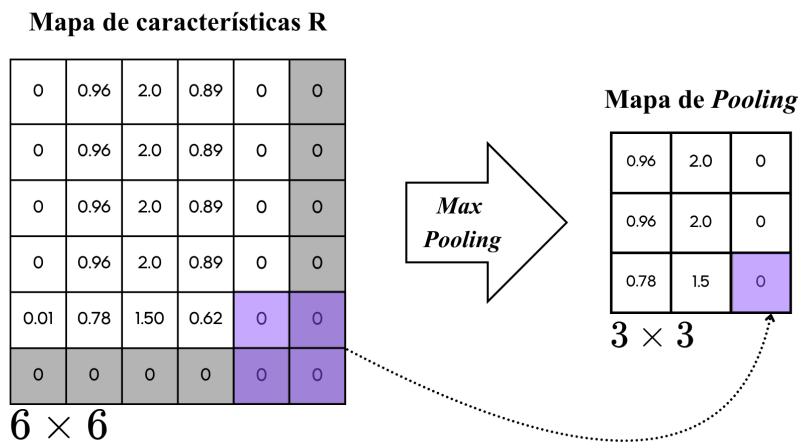
Figura 3.4: Aplicação da função de ativação ReLU no mapa de características.



Fonte: Elaboração do Autor.

Etapa 4: Max pooling. Em seguida, o mapa de características é processado por uma camada de *Max pooling* com dimensões $2 \times 2 \times 1$ e *stride* igual a 2, conforme descrito no algoritmo 3. Essa operação reduz as dimensões espaciais do mapa, preservando os valores mais relevantes em cada região. Como a dimensão do mapa neste exemplo é ímpar, foi adicionado um *padding* para simplificar o processo, como mostrado na Figura 3.5.

Figura 3.5: Aplicação de uma camada de *Max pooling* no mapa de características resultante após a ReLU.



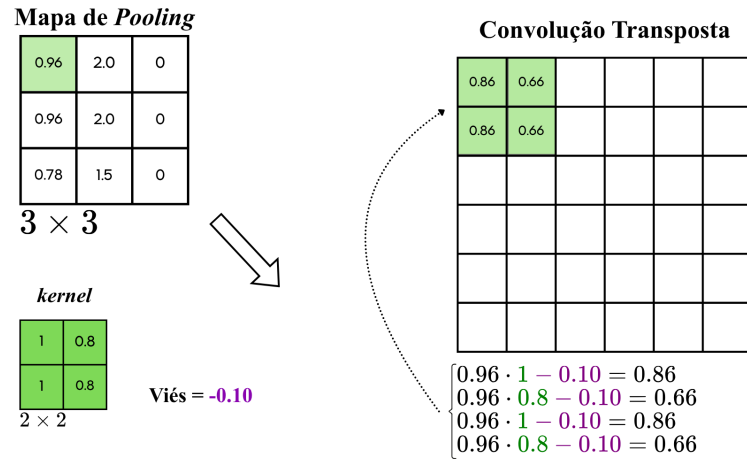
Fonte: Elaboração do Autor.

Ao final dessa etapa, conclui-se a parte do *encoder*, resultando em um mapa de características

com dimensões $3 \times 3 \times 1$. O processo de extração de características realizado pelo *encoder* é fundamental para que o *decoder* possa desenvolver a máscara de segmentação desejada.

Etapa 5: Convolução transposta. No *decoder*, o primeiro passo é expandir novamente as dimensões espaciais do mapa de características. Isso é feito por meio de uma convolução transposta com *kernel* de dimensões $2 \times 2 \times 1$, conforme descrito no algoritmo (4), como mostrado na Figura 3.6. Essa operação pode ser entendida como um processo de reconstrução espacial da informação comprimida no *encoder*. Cada valor do mapa de *pooling* é multiplicado

Figura 3.6: Aplicação da convolução transposta no mapa de *pooling*.

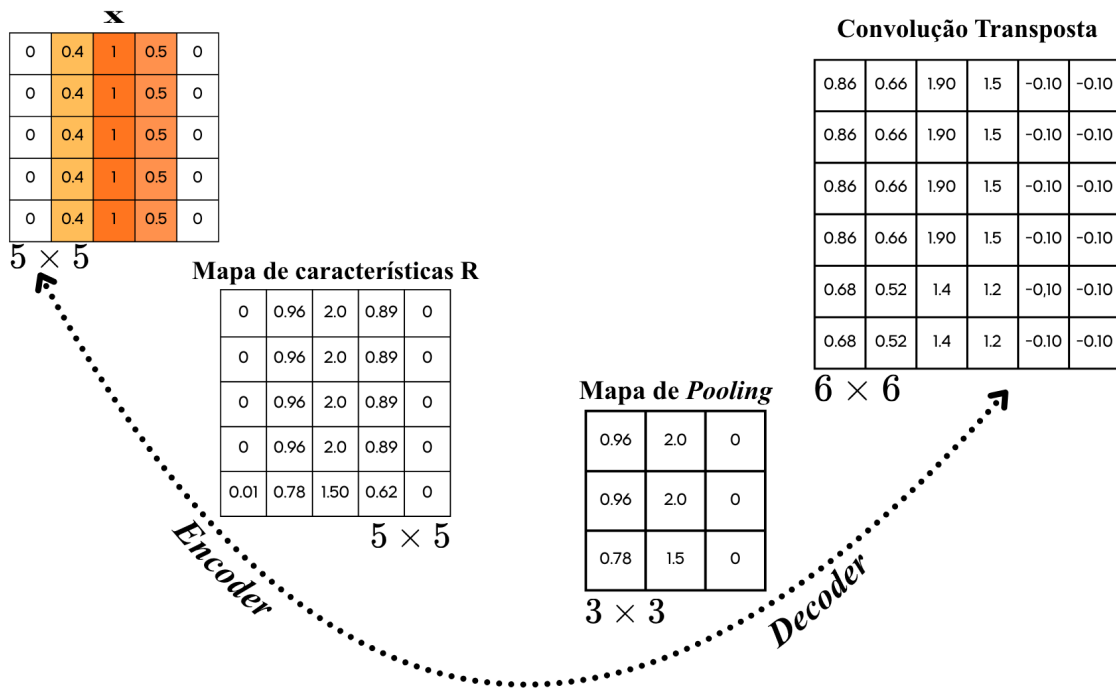


Fonte: Elaboração do Autor.

pelo *kernel* e adicionado às entradas do mapa de características que estão sendo cobertas pelo *kernel*, resultando em um mapa de características com dimensões $6 \times 6 \times 1$.

Na Figura 3.7 é resumida a dinâmica geral da U-Net: enquanto o *encoder* reduz progressivamente as dimensões espaciais para extrair características, o *decoder* expande essas dimensões para reconstruir a saída desejada.

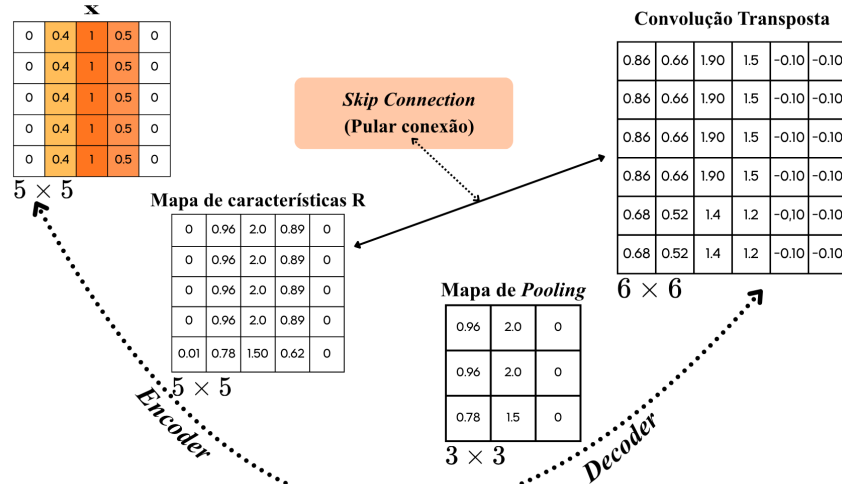
Figura 3.7: Estrutura simplificada em forma de U da arquitetura U-Net.



Fonte: Elaboração do Autor.

Etapa 6: *Skip connection, copy and crop* e concatenação. Uma característica fundamental da U-Net é a presença das conexões de atalho (*skip connections*) entre camadas correspondentes do *encoder* e do *decoder*. Estas conexões permitem combinar as informações obtidas no *encoder*, como bordas e texturas, com as informações obtidas no *decoder*, favorecendo uma segmentação mais precisa. A ideia geral é ilustrada na Figura 3.8. As combinações são feitas por meio da operação de concatenação, que une os canais dos mapas de características extraídos pelo *encoder* com os canais dos mapas de características do *decoder* antes de cada etapa de convolução no *decoder*.

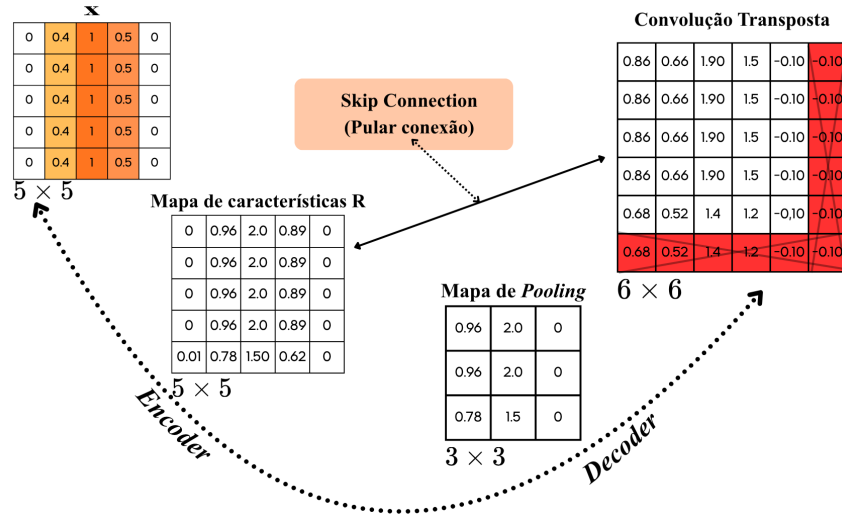
Figura 3.8: Exemplo de conexões de atalho (*skip connections*) na arquitetura U-Net.



Fonte: Elaboração do Autor.

Para que os mapas de características possam ser concatenados, é necessário que possuam as mesmas dimensões espaciais. Quando isso não ocorre, aplica-se a técnica de Cópia e Corta, do inglês *copy and crop*, isto é, recorta-se o mapa de maior dimensão para compatibilizá-lo com o outro. Esse procedimento é ilustrado na Figura 3.9.

Figura 3.9: Exemplo da compatibilização de dimensões para a concatenação.



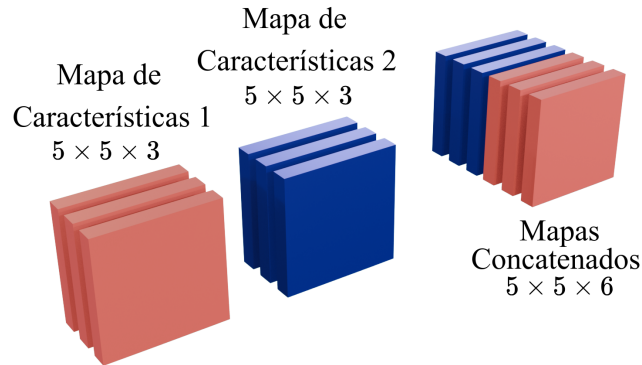
Fonte: Elaboração do Autor.

A cor vermelha, presente na Figura 3.9, sinaliza quais linhas e colunas serão recortadas do mapa de características do *encoder*. Após a compatibilização das dimensões, a concatenação é realizada ao longo do eixo dos canais. Se dois mapas de características possuem dimensões espaciais iguais, a operação pode ser representada por:

$$X_1 = (A, L, C_1), \quad X_2 = (A, L, C_2) \quad \text{então} \quad X_1 \oplus X_2 = (A, L, C_1 + C_2),$$

em que A representa a altura, L a largura, C_1 e C_2 o número de canais, e $\oplus$ a concatenação ao longo do eixo dos canais. Na Figura 3.10 é ilustrada essa operação.

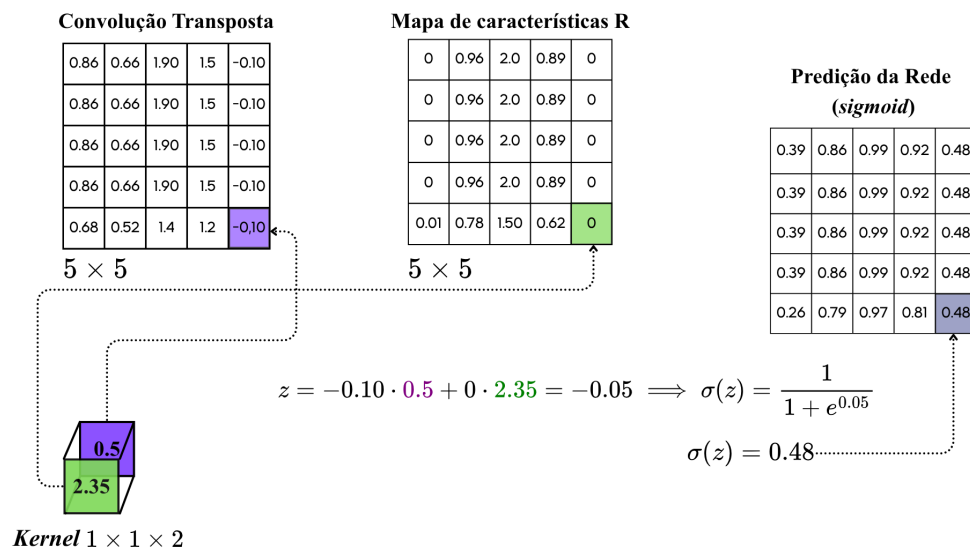
Figura 3.10: Exemplo de concatenação de dois mapas de características ao longo do eixo dos canais.



Fonte: Elaboração do Autor.

Etapa 7: Convolução final e limiar. Após a concatenação, aplica-se uma última camada de convolução que não reduz a dimensão espacial, ou seja, o *kernel* tem tamanho $(1 \times 1 \times 2)$, com dois canais pois o mapa de características após a concatenação possui dois canais, para este exemplo, como mostrado na Figura 3.11. Essa camada combina a informação dos canais e produz a saída da rede.

Figura 3.11: Aplicação da última camada de convolução para gerar a saída da rede.

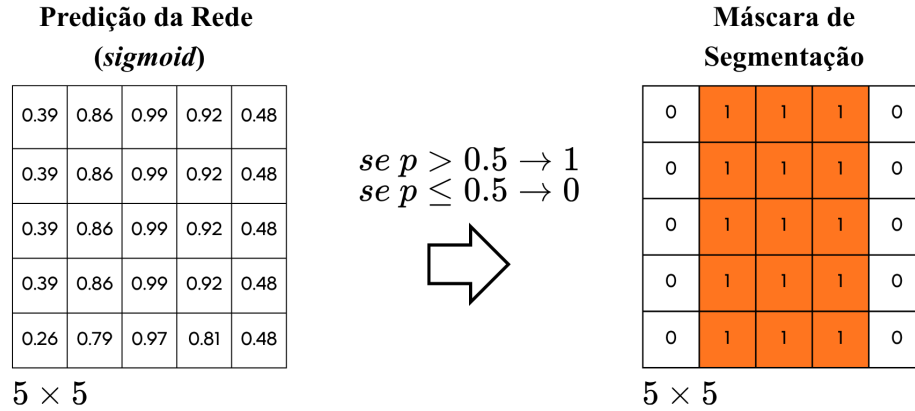


Fonte: Elaboração do Autor.

A aplicação da função *sigmoid* produz valores no intervalo $[0, 1]$, interpretados como probabilidades de cada pixel pertencer à região de interesse. Para que a saída da rede seja convertida

na máscara de segmentação, aplica-se um limiar (*threshold*) para converter as probabilidades em valores binários, como mostrado na Figura 3.12.

Figura 3.12: Aplicação de um limiar (*threshold*) para gerar a máscara de segmentação.



Fonte: Elaboração do Autor.

A máscara de segmentação resultante é então comparada com o *ground truth* utilizando métricas de similaridade, descritas na Seção 3.3, para avaliar a precisão da segmentação realizada pela U-Net.

O processo descrito neste exemplo é repedito para cada imagem de entrada durante o treinamento da rede, ajustando os pesos e vieses para minimizar a diferença entre a máscara de segmentação e o *ground truth*. Após o treinamento, a U-Net é capaz de generalizar e segmentar novas imagens de entrada, produzindo máscaras de segmentação precisas mesmo para imagens que não foram vistas durante o treinamento.

Resumo do exemplo. Neste exemplo simplificado, a U-Net recebeu a imagem de entrada X , extraiu características relevantes no *encoder*, reduziu sua dimensão espacial por meio do *Max pooling*, reconstruiu a informação no *decoder* com auxílio das *skip connections* e, por fim, produziu a máscara de segmentação compatível com o *ground truth*. Embora simplificado, o exemplo evidencia os três elementos centrais da arquitetura: extração de características, reconstrução espacial e reaproveitamento de informações do *encoder* no *decoder*.

3.1.1 Arquitetura Original da U-Net

A rede convolucional desenvolvida por Ronneberger, Fischer e Brox (2015) é bem similar à arquitetura de rede utilizada no Exemplo 3.1, entretanto, a arquitetura completa da U-Net é composta por múltiplas camadas de convolução e *pooling* no *encoder*, e múltiplas camadas de convolução transposta no *decoder*. No total são quatro blocos de convolução e *pooling* no *encoder*

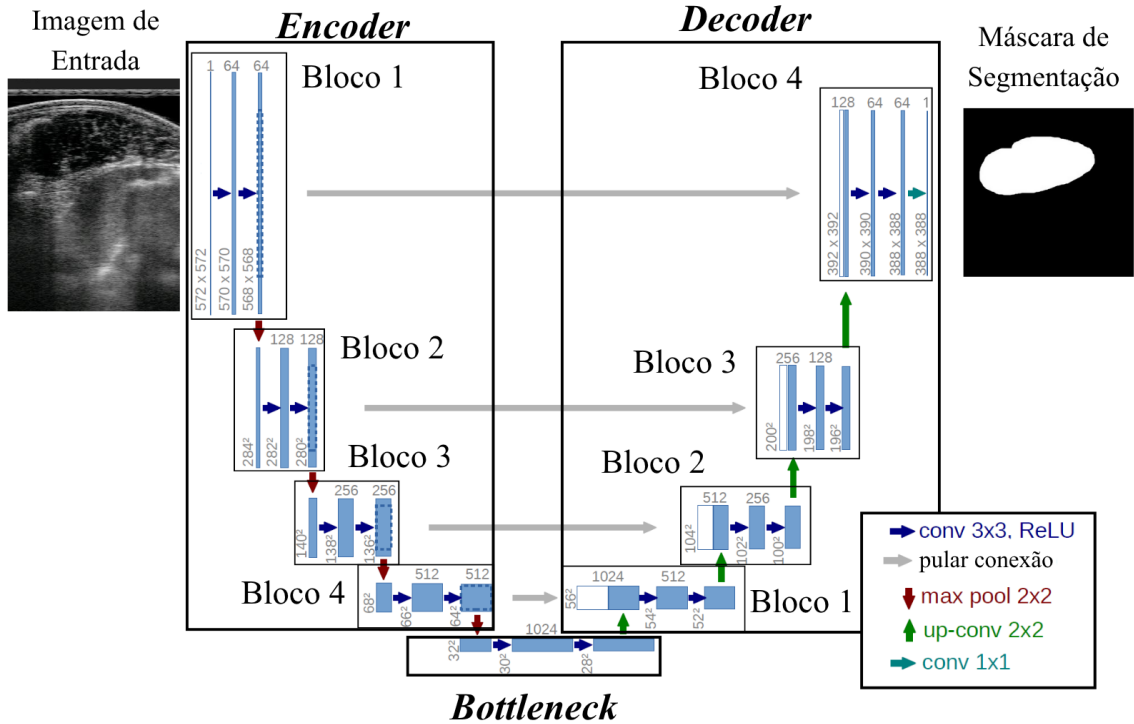
e quatro blocos de convolução transposta no *decoder*, cada bloco contendo duas convoluções e um *pooling*, como é mostrado na Figura 3.13.

O número de *kernels* em cada camada de convolução aumenta à medida que se avança no *encoder*, começando com 64 *kernels* na primeira camada e dobrando a quantidade a cada camada subsequente, chegando a 512 *kernels* na última camada do *encoder*, os blocos do *encoder* são compostos por duas camadas de convolução seguidas de uma operação de *pooling*. No *decoder*, o número de *kernels* é reduzido pela metade a cada camada subsequente, começando com 512 *kernels* na primeira camada do *decoder* e terminando com 64 *kernels* na última camada antes da saída, os blocos do *decoder* são compostos por uma camada de convolução transposta seguida de duas camadas de convolução. As conexões de atalho (*skip connections*) entre as camadas correspondentes do *encoder* e do *decoder* permitem que sejam feitas as concatenações dos mapas de características, e, caso a dimensão entre estes seja diferente, utiliza-se a técnica de *copy and crop*. No final do *encoder* tem-se um bloco conhecido como *baseline* ou *bottleneck*, em português, linha base e gargalo, respectivamente. É o bloco mais profundo da rede, em que as características extraídas pela U-Net são processadas antes de serem enviadas para o *decoder*. O *bottleneck* é um bloco composto por duas camadas de convolução com 1024 *kernels*. A saída do *bottleneck* é então enviada para o *decoder* para gerar a máscara de segmentação.

Na Figura 3.13 é apresentada a arquitetura completa da U-Net, cada bloco representa uma etapa de processamento, o número na parte superior do retângulo azul indica o número de canais presentes no mapa de características, enquanto o número na parte inferior do retângulo, indica as dimensões espaciais. As setas azuis representam a operação de convolução com *kernels* de dimensões $3 \times 3 \times C_r^{(i)}$, em que $C_r^{(i)}$ é o número de canais do mapa de características resultante do bloco i , com função de ativação *ReLU*, as setas vermelhas representam a operação de *pooling* com *kernels* de dimensões $2 \times 2 \times C_a^{(i)}$ e *stride* de 2, em que $C_a^{(i)}$ é o número de canais do mapa de características do bloco i . As setas cinzas representam as conexões de atalho e as setas verdes representam as operações de convolução transposta com *kernels* de dimensões $2 \times 2 \times C_d^{(i)}$, em que $C_d^{(i)} = 2 \cdot C_r^{(5-i)}$, para $i = 1, 2, 3, 4$. A última camada de convolução utiliza um *kernel* de dimensões $1 \times 1 \times 1$ e função de ativação *sigmoid logística* para gerar a saída da rede, representado pela seta azul claro. Aplica-se um *threshold* para converter a saída da rede em uma máscara de segmentação, em que os pixels com valor “1” representam a região de interesse e os pixels com valor “0” representam o fundo.

Na U-Net original a técnica de *padding* utilizada é a “*valid*”, ou seja, não é adicionado nenhum preenchimento às bordas dos mapas de características, com isso em cada etapa de convolução as dimensões dos mapas são reduzidas, conforme descrito na Subseção 1.3.6. Os cortes feitos para a concatenação, sinalizados por linhas azuis pontilhadas, nos mapas de características do *encoder* (lado direito da Figura 3.13) são feitas do centro para fora, ou seja, a parte central do mapa de características é mantida para a concatenação, sendo retiradas as mesmas quantidades de linhas e colunas de todos os lados. O total de parâmetros a serem otimizados, ou seja, pesos e viéses é de 27.548,993.

Figura 3.13: Arquitetura completa da U-Net.



Fonte: Adaptado de Ronneberger, Fischer e Brox (2015).

3.2 Modelos Propostos

Para melhorar a capacidade de segmentação da U-Net, foram propostas duas arquiteturas derivadas do modelo original: a U-Net Multitarefa e a U-Net Fuzzy. Ambas compartilham uma mesma arquitetura base e diferem no tratamento aplicado ao *bottleneck* da rede.

3.2.1 Visão Geral das Modificações Propostas

As duas arquiteturas propostas preservam a estrutura geral da U-Net, composta por encoder, bottleneck e decoder. Em relação à arquitetura original, foram adotadas modificações na quantidade de *kernels*, no tipo de *padding* e na forma de processamento das características extraídas no bottleneck. Na primeira proposta, a informação da raça do animal é incorporada por meio de uma camada multitarefa. Na segunda, além dessa etapa, é incluído um módulo de inferência fuzzy para refinar as características antes da reconstrução da máscara de segmentação.

Modelo Base

A arquitetura base utilizada nos dois modelos é composta por quatro blocos no encoder e quatro blocos no decoder. No *encoder* cada bloco é composto por duas camadas de convolução com *kernels* de tamanho $3 \times 3 \times C$, *stride* de tamanho 1, *padding* “same” e função de ativação ReLU, seguidas por uma camada de *Max pooling* para reduzir a dimensão do mapa de características resultante, com *kernel* de tamanho $2 \times 2 \times C$, *stride* de tamanho 2, Subseções 1.21, 1.3.7,

1.3.8, respectivamente. O número de canais (C) dos *kernels* dobra de quantidade a cada bloco, começando com 32 canais no primeiro bloco e chegando a 128 canais no quarto bloco. Optou-se pela convolução *same* pois melhores resultados foram obtidos.

No *bottleneck*, tem-se duas camadas de convolução com *kernel* de tamanho $3 \times 3 \times 256$, *stride* de tamanho 1, *padding* “*same*” e função de ativação ReLU.

No *decoder* a rede também é dividida em blocos, cada bloco é composto por uma camada de convolução transposta com *kernel* de tamanho $2 \times 2 \times C$, *stride* de tamanho 2 e *padding* “*same*”, seguida por uma camada de concatenação para concatenar as características extraídas pelo *encoder* com as características extraídas pelo *decoder*, seguida por uma camada de convolução com *kernel* de tamanho $3 \times 3 \times C$, *stride* de tamanho 1, *padding* “*same*” e função de ativação ReLU. Como a dimensão da imagem não é reduzida em cada etapa de convolução do *encoder* não é necessário realizar a técnica de *copy and crop* vista na U-Net original, Seção 3.1, pois as características extraídas pelo *encoder* e pelo *decoder* possuem a mesma dimensão espacial, como é mostrado na Figura 3.14, o que facilita a concatenação dos mapas de características. O número de canais é reduzido pela metade a cada bloco, começando com 128 canais no primeiro bloco e chegando a 32 canais no quarto bloco.

A camada de saída é composta por uma camada de convolução com um único *kernel* de tamanho $1 \times 1 \times 1$, *stride* de tamanho 1 e função de ativação *Sigmoid* (1.11) para produzir a saída da rede. Como a saída é uma imagem em que cada pixel representa a probabilidade de pertencer ou não à região de interesse, um limiar, do inglês *threshold*, é aplicado para binarizar a saída da rede e produzir a máscara de segmentação.

U-Net Multitarefa

Esta modificação envolve a adição de uma nova camada de classificação à arquitetura original da U-Net, permitindo que a rede aprenda a realizar tanto a segmentação quanto a classificação simultaneamente. A camada de classificação é composta por unidades adicionais que processam as características extraídas pela U-Net para produzir uma saída de classificação no *bottleneck* (ZHU; ROHLING; SALCUDEAN, 2022). Um exemplo de aplicação é a segmentação de imagens de cancer de mama feitas por ultrassom, em que a U-Net é utilizada para segmentar a região de interesse (tumor) e a camada de classificação é utilizada para classificar o tipo do tumor (benigno (0) ou maligno (1)), a entrada da rede é (IMG, [0,1]), em que IMG representa a imagem de entrada e [0,1] representa os rótulos de classificação. A segunda entrada da rede geralmente está em *OneHotEncoder* que é uma técnica de codificação de rótulos categóricos em vetores binários, no caso do exemplo, benigno seria codificado para [1, 0] e [0, 1] para maligno (DATA SCIENCE ACADEMY, 2025).

O funcionamento da camada pode-se ser entendido da seguinte forma: Após as duas operações de convolução do *bottleneck* a rede gera um mapa de característica de dimensões (B, H, W, C) , em que B é o tamanho do lote (Subseção 1.2.2), H (altura) e W (largura) são as dimensões da imagem e C é o número de canais. A entrada da rede está no formato (IMG, $[x_0, \dots, x_n]$), para $x_k = 0$ e $x_j = 1$, $k \neq j$ e $k, j = 0, \dots, n$. Na camada multitarefa é definida

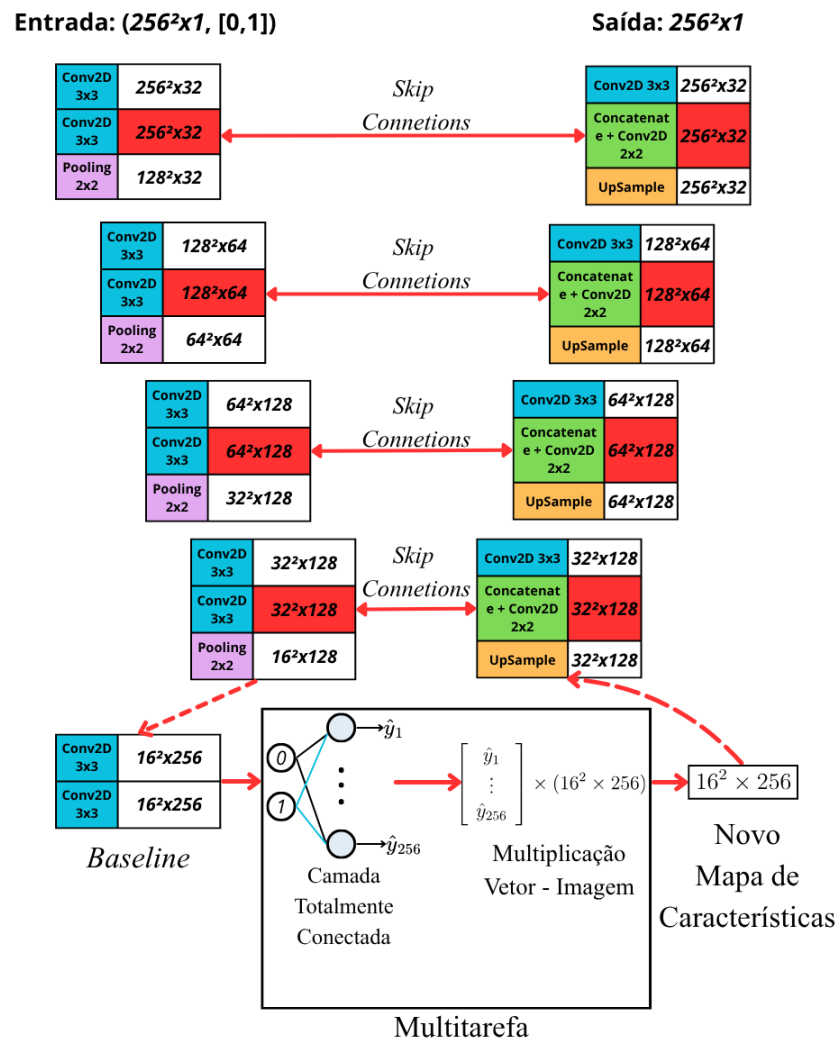
uma camada totalment conectada, ou seja, uma rede neural com uma única camada oculta, com matriz de pesos W de entradas reais, com dimensão $C \times n$. A saída da camada é então calculada pela seguinte operação matricial:

$$\tau(W_{C \times n} \odot x_{1 \times n}) = [\hat{y}_0, \hat{y}_1, \dots, \hat{y}_C]_{C \times 1},$$

em que “ $\odot$ ” representa a multiplicação matricial, $x = [x_0, x_1, \dots, x_n]_{1 \times n}$ e τ é a função de ativação utilizada. A saída da camada é um vetor coluna de tamanho C , em que cada elemento $\hat{y}_i \in \mathbb{R}$. Para finalizar a camada multitarefa, o mapa de caracterísiticas original é multiplicado pelo vetor de saída da camada totalmente conectada. Após esse processo, com todos os pixels calculados, o novo mapa de características é formado com a mesma dimensão do mapa de características de entrada e é enviada para o *decoder* da U-Net para obter a segmentação final ou, no caso deste trabalho, também é enviado para o módulo de inferência fuzzy, como é mostrado na Figura 3.15.

Na Figura 3.14 é mostrado a arquitetura da U-Net multitarefa utilizada e como as dimensões das imagens vão se modificando ao longo da rede. A cor azul representa as camadas de convolução com *kernels* de tamanho 3×3 , a cor roxa representa as camadas de *Max pooling*, a cor laranja representa as camadas de convolução transposta com *kernels* de tamanho 2×2 e a cor verde representa as camadas de concatenação e convoluções com *kernels* de tamanho 2×2 . As setas indicam o caminho dos dados ao longo da rede, mostrando como as características extraídas pelo *encoder* são concatenadas com as características extraídas pelo *decoder* (*Skip Connection*) para produzir a máscara de segmentação final. O Multitarefa após o *bottleneck* é a representação visual das equações vistas na Subseção 3.2.

Figura 3.14: Arquitetura da U-Net multitarefa utilizada e as dimensões das imagens ao longo da rede.



Fonte: Elaboração do Autor.

Na figura 3.14 a imagem é representada como $16^2 \times 256 = (16, 16, 256)$, sem o *lote* para facilitar a visualização. A camada multitarefa está recebendo uma imagem de dimensões $(16, 16, 256)$ junto com o vetor $[0, 1]$. A saída da camada totalmente conectada é um vetor de dimensões 256×1 que é multiplicado pelo mapa de características original.

U-Net Fuzzy

A U-Net Fuzzy compartilha a arquitetura base da U-Net Multitarefa, diferenciando-se pela inclusão de um módulo de inferência fuzzy (ANFIS) após a camada multitarefa da rede. Este módulo é responsável por aplicar regras fuzzy para refinar as previsões da U-Net (VERMA *et al.*, 2026). A adição de um módulo fuzzy adiciona incerteza e flexibilidade ao processo de segmentação, permitindo que a rede aprenda a lidar com casos ambíguos ou ruidosos, como por

exemplo um pixel que está próximo ao limite entre duas classes. O módulo fuzzy é adicionada após a camada multitarefa da U-Net permitindo que as regras fuzzy sejam aplicadas aos mapas de características extraídas pela camada de multitarefa antes de serem processadas pelo *decoder*. Intuitivamente o módulo segue as etapas vistas na Seção 2.3, com algumas modificações para o contexto da U-Net. As etapas do módulo de inferência fuzzy são: pré-processamento, fuzzificação, inferência e defuzzificação.

Na etapa de pré-processamento, o mapa de características (B, H, W, C) recebido da camada multitarefa é separado em pixels individuais de dimensões $(B, 1, 1, C)$, ou seja, cada pixel do mapa de características é tratado como um vetor de entrada para o módulo de inferência fuzzy. Para que o módulo não crie muitos parâmetros, é necessário que o vetor de entrada seja reduzido, para isso, ao iniciar a módulo fuzzy é definido o número de antecedentes (número de entradas) e o número de conjuntos fuzzy para cada antecedente, com isso, a camada de pré-processamento é responsável por reduzir o número de entradas do vetor de cada pixel para o número de antecedentes definidos. O pré-processamento é dado por:

$$\bar{x}_{1 \times a} = x_{1 \times C} \odot W_{C \times a} + b_{1 \times a} = [\bar{x}_0, \bar{x}_1, \dots, \bar{x}_{a-1}]_{1 \times a},$$

em que $W_{C \times a}$ é a matriz de pesos treináveis da camada de pré-processamento, $x_{1 \times C}$ é o vetor de entrada do pixel, “ a ” é o número de antecedentes definidos para o módulo fuzzy e $b_{1 \times a}$ é o vetor de vieses da camada de pré-processamento.

Na etapa de fuzzificação, cada valor do vetor $\bar{x}_{1 \times a}$ receberá um grau de pertinência para cada conjunto fuzzy definido previamente, ou seja, as dimensões de $\bar{x}_{1 \times a}$ são transformadas para $\bar{x}_{1 \times a \times F}$, em que F é o número de conjuntos fuzzy definidos para cada vetor. As funções de pertinência são gaussianas, equação (2.2). c é o centro da função de pertinência, inicializado com valores dentro do intervalo de $[-1, 1]$, e σ tendo valor padrão de 0.5. Cada vetor $\bar{x}_{1 \times a}$ têm um total de três c e um σ que são treináveis utilizando o método de *backpropagation*, Subseção 1.2.1.

A etapa de inferência é dividida em duas partes. A parte “SE” e a parte “ENTÃO”. Para a primeira parte, a quantidade de regras fuzzy são definidas como $R = F^a$, em que R é o número total de regras fuzzy. Observa-se que o número de regras cresce exponencialmente com o número de antecedentes, por isso é importante definir um número pequeno de antecedentes para o módulo fuzzy. Com isso cada valor do vetor $\bar{x}_{1 \times a \times F}$ é combinado para produzir os pesos w_i , também chamado de força de ativação de cada regra. Por exemplo, se o vetor $\bar{x} = [[0.2, 0.3, 0.5], [0.1, 0.1, 0.8], [0.7, 0.1, 0.2]]$, ou seja, 3 antecedentes e 3 conjuntos fuzzy, então o número de regras é $R = 3^3 = 27$ e as forças de ativação de cada regra são dadas por:

$$\begin{cases} w_1 = 0.2 \cdot 0.1 \cdot 0.7, \\ w_2 = 0.2 \cdot 0.1 \cdot 0.1, \\ \vdots \\ w_{27} = 0.5 \cdot 0.8 \cdot 0.2, \end{cases}$$

De forma geral, a força de ativação da regra r é dada por:

$$w_r = (\mu_{A_1}(x_{11}) \cdot \mu_{A_1}(x_{21}) \cdot \dots \cdot \mu_{A_1}(x_{a1})) = \prod_{i=0}^a \mu_{A_j}(x_{ij}),$$

para $j = 1, \dots, F$, em que A_j é o conjunto fuzzy e w_r é a força de ativação da regra r , com $r = 1, \dots, R$. Em seguida, as forças de ativação são normalizadas utilizando a seguinte fórmula:

$$\overline{w_r} = \frac{w_r}{\sum_{r=0}^R w_r + \epsilon},$$

em que ϵ é um valor pequeno adicionado para evitar a divisão por zero.

A parte “ENTÃO” é definida por uma função linear das entradas, baseado no método de Takagi-Sugeno, ou seja:

$$\begin{cases} (f_{regra_1}(x))_{1 \times C} = (A_{regra_1})_{C \times C} \cdot x_{1 \times C} + (b_{regra_1})_{1 \times C}, \\ \vdots \\ (f_{regra_R}(x))_{1 \times C} = (A_{regra_R})_{C \times C} \cdot x_{1 \times C} + (b_{regra_R})_{1 \times C}, \end{cases}$$

em que A_{regra_j} é a matriz de pesos treináveis da regra j , b_{regra_j} é o vetor de viéses treináveis da regra j e $x_{1 \times C}$ é o vetor de entrada do módulo de inferência fuzzy. Nesta etapa, o número de parâmetros treináveis é de $R \times (C^2 + C)$, ou seja, para cada regra são treinados uma matriz de pesos de dimensões $C \times C$ e um vetor de viéses de dimensões $1 \times C$. Na etapa de defuzzificação, para gerar a saída da inferência, cada regra é multiplicada pela força de ativação da regra:

$$y_r = w_r \cdot f_{regra_r}(x),$$

em que y_r é a saída da inferência e $\overline{w_r}$ é a força de ativação normalizada da regra r . A saída é então calculada pela soma ponderada das saídas de todas as regras, ou seja:

$$y_{1 \times C} = \sum_{r=1}^R y_r,$$

A saída da inferência é um novo vetor $(B, 1, 1, C)$. É por esse motivo que w_r é chamado de força de ativação, já que se w_r for baixo, ou seja, a regra r não for muito relevante para o pixel de entrada, a saída da regra y_r também será baixa, mesmo que a regra $f_{regra_r}(x)$ produza um valor alto. Esse processo é repetido para cada pixel do mapa de características recebido pelo módulo de inferência fuzzy, gerando um novo mapa de características de saída do módulo fuzzy.

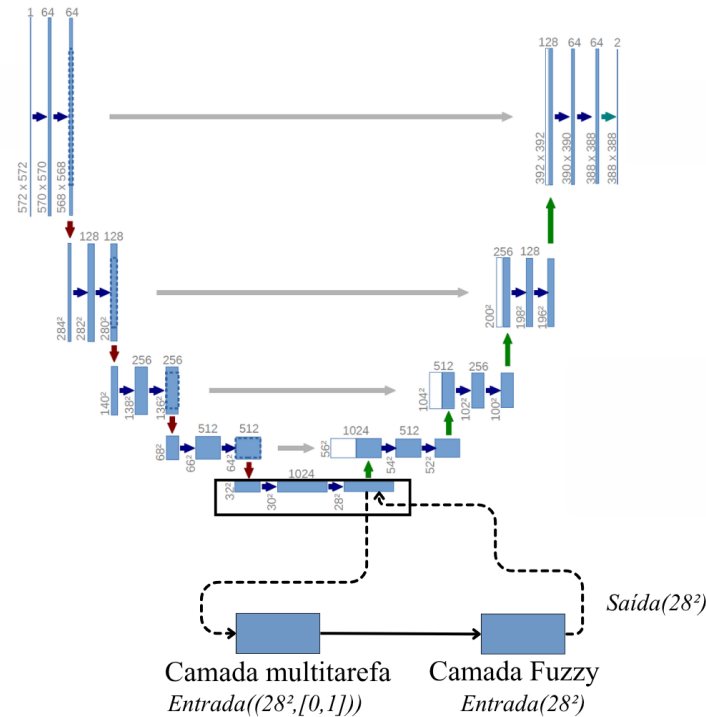
No início do treinamento, o mapa de características de saída do módulo fuzzy é somente ruído, para evitar que esse ruído extra atrapalhe o processo de treinamento da U-Net, a saída do módulo fuzzy é somada ao mapa de características original da camada multitarefa para obter um mapa de características final:

$$Y = Y_m + \alpha Y_f,$$

em que Y é o mapa de características final, Y_m é o mapa de características original da camada multitarefa, Y_f é o mapa de características de saída do módulo fuzzy e α é um hiperparâmetro que controla a influência do módulo fuzzy no mapa de características final. O valor de α é definido como um valor pequeno, por exemplo, 0.1, para que a influência do módulo fuzzy seja limitada no início do treinamento, permitindo que a U-Net aprenda a segmentar as imagens sem ser muito afetada pelo ruído gerado pelo módulo fuzzy. Conforme o treinamento avança, o valor de α pode ser aumentado gradualmente para permitir que o módulo fuzzy tenha mais influência no processo de segmentação, refinando as previsões da U-Net à medida que o modelo aprende a lidar com casos ambíguos ou ruidosos.

Após essa etapa o mapa de características é processado pelo *decoder* da U-Net para obter a segmentação final. Na Figura 3.15 é ilustrada a arquitetura da U-Net com a adição de uma camada Multitarefa e uma camada de inferência fuzzy.

Figura 3.15: Arquitetura da U-Net com a adição de uma camada Multitarefa e uma camada de inferência fuzzy.



Fonte: Adaptado de Ronneberger, Fischer e Brox (2015).

As setas mostram o fluxo do *bottleneck* pelas camadas adicionais, em que “28²” representa a dimensão do mapa de características.

3.3 Critérios de Avaliação

3.3.1 Coeficiente de Dice

Quando a região de interesse em uma imagem é pequena em comparação ao plano de fundo, a métrica da acurácia pode ser enganosa, pois a maioria dos pixels da imagem pertencem ao plano de fundo, o que pode levar a um resultado de alta acurácia mesmo que o modelo não esteja segmentando corretamente, ou seja, classificando cada pixel como pertencente ou não à região de interesse. Para lidar com esse problema, é comum utilizar métricas específicas para avaliar o resultado, como o coeficiente de Dice, que é uma medida de similaridade entre a máscara de segmentação predita e o *ground truth* (DICE, 1945). O coeficiente de Dice é calculado utilizando a seguinte fórmula:

$$Dice = \frac{2|A \cap B|}{|A| + |B|}, \quad (3.1)$$

em que A é o conjunto de pixels da máscara de segmentação predita e B é o conjunto de pixels do *ground truth*. O coeficiente de Dice varia entre 0 e 1, em que um valor de 1 indica que todas as regiões de interesse foram corretamente identificadas e um valor de 0 indica que não há sobreposição entre a máscara de segmentação predita e o *ground truth*.

Exemplo 3.2. Seja $A = \{1, 2, 3, 4, 5, 6\}$ e $B = \{1, 2, 3, 4, 9\}$. A interseção entre os conjuntos A e B é $A \cap B = \{1, 2, 3, 4\}$, o que significa que existem 4 elementos em comum entre os dois conjuntos. O número total de elementos em A é $|A| = 6$ e o número total de elementos em B é $|B| = 5$. Substituindo esses valores na fórmula do coeficiente de Dice, tem-se:

$$Dice = \frac{2|A \cap B|}{|A| + |B|} = \frac{2 \cdot 4}{6 + 5} = \frac{8}{11} \approx 0,727.$$

Portanto, o coeficiente de Dice para os conjuntos A e B é aproximadamente 0,727, indicando uma boa sobreposição entre os dois conjuntos, mas não uma correspondência perfeita.

A métrica Dice é bastante utilizada em conjunto com a arquitetura U-Net para avaliar a qualidade da segmentação gerada pela rede. O objetivo é maximizar o valor do coeficiente de Dice, ou seja, aumentar a sobreposição entre a imagem segmentada e a máscara de segmentação (MELO *et al.*, 2022).

3.3.2 Acurácia

A acurácia é uma métrica de avaliação utilizada para medir a proporção de pixels classificados corretamente pelo modelo em relação ao total de pixels na imagem. Para o contexto de segmentação de imagens, se C_T for os pixels da região de AOL traçada pelo especialista e C o conjunto dos pixels obtidos pelo método de predição. Os pixels definidos como verdadeiros positivos são $VP = C_T \cap C$, enquanto os pixels definidos como verdadeiros negativos são $VN = \{p|p \notin C_T \text{ e } p \notin C\}$. Os pixels falsos positivos são $FP = \{p|p \notin C_T \text{ e } p \in C\}$ enquanto os

pixels falsos negativos satisfazem $FN = \{p|p \in C_T \text{ e } p \notin C\}$. Com isso, a acurácia é calculada utilizando a fórmula (3.2),

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN}. \quad (3.2)$$

Para o contexto de segmentação de imagens, a acurácia nem sempre é tão precisa quanto o *Dice*. Caso a imagem de segmentação seja uma região pequena em comparação com o fundo da imagem, o modelo pode classificar a maioria dos pixels como fundo e ainda assim obter uma acurácia alta, mesmo que o coeficiente de Dice seja próximo de zero (MELO *et al.*, 2022).

3.3.3 Função de Custo

A função de custo indica o quão bem o modelo está se ajustando aos dados. Para avaliar os modelos, é utilizada a função *Binary Cross-Entropy*, que mede a diferença entre as distribuições de probabilidade das máscaras de segmentação preditas pelo modelo e as máscaras de segmentação reais. Quanto menor for o valor da *Binary Cross-Entropy*, melhor é o desempenho do modelo na segmentação das imagens.

3.3.4 Erro Médio Absoluto

O Erro Absoluto Médio, do inglês *Mean Absolute Error* (MAE), é uma métrica de avaliação utilizada para medir a média dos erros absolutos entre as medidas preditas pelo modelo e as medidas reais (AOL, EGL e EGG) (MELO *et al.*, 2022). Este é calculado utilizando a seguinte fórmula:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3.3)$$

em que y_i são as medidas reais e $\hat{y}_i$ são as medidas preditas pelo modelo e n é o número de amostras.

3.3.5 Coeficiente de Determinação e Reta de Regressão Linear

O coeficiente de determinação (R^2) é uma métrica de avaliação utilizada para medir a proporção da variância dos dados que é explicada pelo modelo (CANDIDO, 2024). Este varia entre 0 e 1, em que 1 indica que o modelo explica toda a variância dos dados e 0 indica que o modelo não explica nenhuma variância dos dados. O R^2 é calculado utilizando a seguinte fórmula:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3.4)$$

em que y_i são as medidas reais, $\hat{y}_i$ são as medidas preditas pelo modelo e $\bar{y}$ é a média das medidas reais. O R^2 é uma métrica importante para avaliar a qualidade das previsões do modelo. Como o objetivo é prever as medidas numéricas correspondentes à AOL, EGL e EGG a partir das

imagens de ultrassom. A reta de regressão linear é uma linha que melhor se ajusta aos dados, e o R^2 indica também o quão bem a reta se ajusta aos dados.

Com isso, neste capítulo foi apresentado os modelos propostos, as métricas de avaliação utilizadas e a metodologia empregada para a análise dos resultados. No próximo capítulo, são apresentados os resultados obtidos a partir da aplicação dos modelos propostos para o problema de segmentação de imagens de ultrassom bovino e uma análise detalhada dos mesmos.

Capítulo 4

Aplicação da U-Net e U-Net Fuzzy em Ultrassonografia Bovina

O processo para desenvolver uma rede neural envolve vários desafios, desde a obtenção de um conjunto de dados adequado, passando pelo pré-processamento das imagens, configuração dos modelos e avaliação dos resultados. Neste capítulo, são descritos os procedimentos adotados para cada uma dessas etapas, bem como os resultados obtidos e as análises realizadas.

4.1 Base de Dados e Planejamento Experimental

Para que os modelos propostos possam ser avaliados, é necessário um conjunto de dados que permita a comparação entre as arquiteturas. Esta Seção apresenta o banco de dados utilizado, o processo de pré-processamento das imagens, a configuração dos modelos comparados e as métricas adotadas para a avaliação do desempenho dos modelos.

4.1.1 Contexto do Problema e Medidas Avaliadas

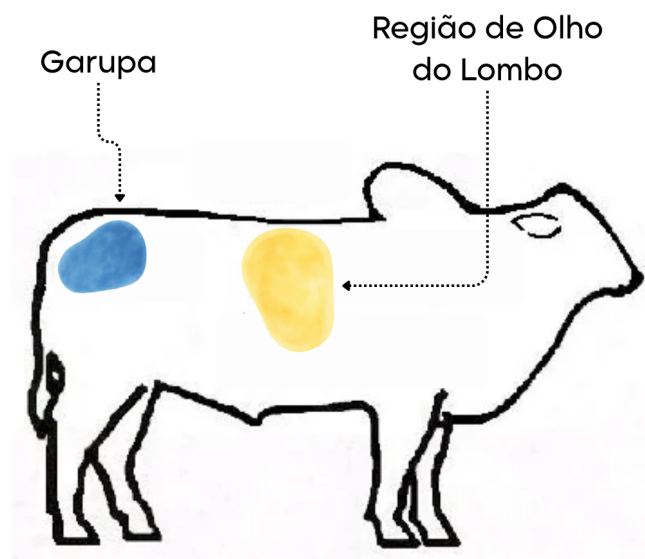
A raça Nelore é originária da Índia, onde seus ancestrais, os bovinos da raças Ongole, eram criados há milênios por sua rusticidade, resistência a climas tropicais e capacidade de adaptação a ambientes variados, esses animais foram trazidos ao Brasil no final do século XIX, em um momento de expansão da pecuária nacional (ZANCANER, 2026). A raça Caracu, por sua vez, tem origem na região da Península Ibérica, chegando no Brasil logo após o descobrimento, os animais da raça Caracu são extremamente funcionais, com alta adaptabilidade e rusticidade, além de serem resistentes a doenças e parasitas (ABCCARACU, 2022). O Brasil é um dos maiores criadores mundiais de bovinos da raça Nelore, com uma população estimada em mais de 230 milhões de cabeças, enquanto a raça Caracu é menos numerosa, mas ainda assim desempenha um papel importante na pecuária brasileira (LAMAS, 2023).

Uma das principais formas de avaliar o desenvolvimento e a qualidade fenotípica dos animais é por meio da análise de imagens de ultrassonografia em que é possível obter informações da carcaça do animal. Entre os recursos e metodologias existentes para avaliação de carcaças e de

características associadas à qualidade da carne, a ultrassonografia se destaca. A avaliação é de extrema importância pois existe uma variabilidade nas características que estão relacionadas com a qualidade de desossa, que por sua vez influenciam a comercialização e os resultados econômicos dos produtores (FELICIO, 2010).

Dependendo da região do corpo do animal onde a imagem de ultrassom é obtida, é possível extrair diferentes informações, neste trabalho serão utilizadas imagens de ultrassom da região do músculo *Longissimus Dorsi* (Contra-filé), que localiza-se entre a 12^a e 13^a costela e da Garupa (WILSON, 1998). Na figura Figura 4.1 é mostrada a localização dessas regiões no corpo do animal.

Figura 4.1: Região do músculo *Longissimus Dorsi* (em amarelo) e da garupa (em azul).



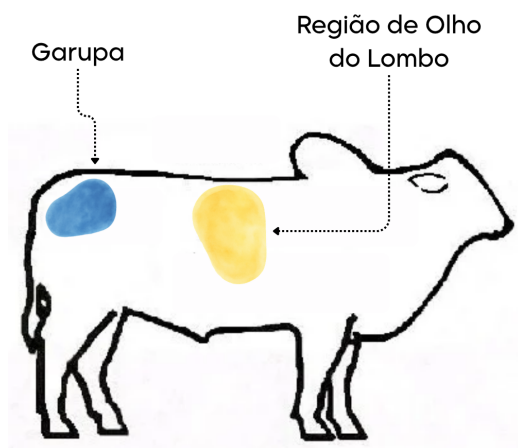
Fonte: Adaptado de Wilson (1998).

Na região de olho do lombo é possível obter informações de dois indicadores para a avaliação da carcaça, a Área de Olho de Lombo (AOL) e a Espessura de Gordura subcutânea no Lombo (EGL). Na Garupa é possível obter a informação da Espessura de Gordura subcutânea na Garupa (EGG).

As imagens de ultrassom utilizadas foram obtidas por meio do aparelho de ultrassom Aloka SSD-500 (Aloka Co. Ltd., 1990), com todos os animais sendo machos por volta de um ano de idade, localizados no Centro Avançado de Pesquisa e Desenvolvimento de Bovinos de Corte, Instituto de Zootecnia, SAA-SA, Sertãozinho - SP. Na Figura 4.2(a) é mostrada a imagem do especialista obtendo a imagem de ultrassom da região do olho de lombo, na Figura 4.2(b) é mostrada a imagem de ultrassom, na Figura 4.2(c) mostra a imagem de ultrassom da garupa e na Figura 4.2(d) é mostrada a imagem do especialista obtendo a imagem de ultrassom da garupa.

esperando elas mandarem as imagens

Figura 4.2: Aquisição da imagem de ultrassom da região do olho de lombo.



(a) Especialista obtendo a imagem de ultrassom da região do olho de lombo.



(b) Imagem de ultrassom da região do olho de lombo.



(c) Imagem de ultrassom da garupa.



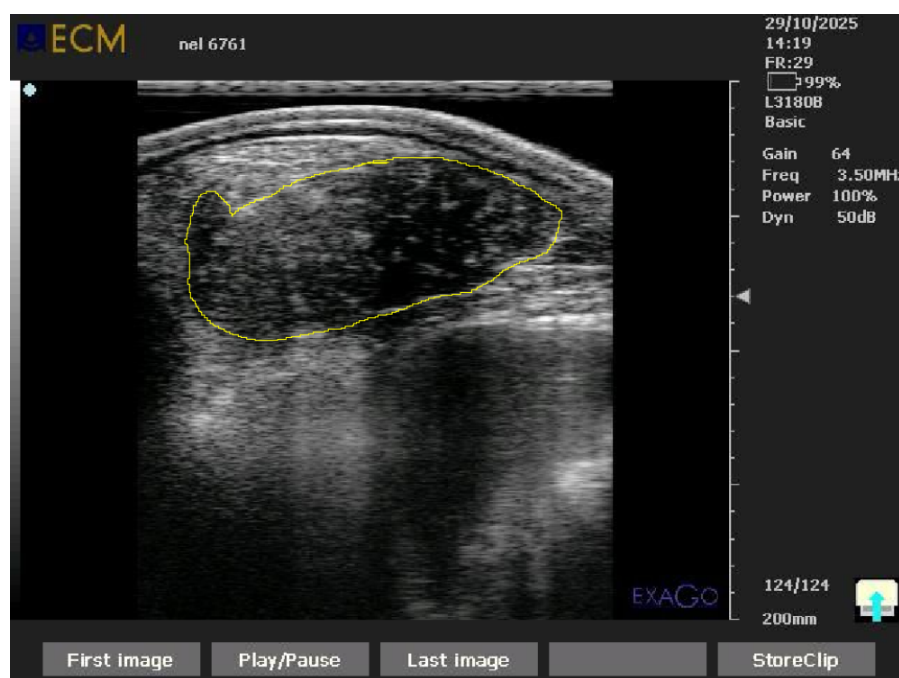
(d) Especialista obtendo a imagem de ultrassom da garupa.

Fonte: Elaborado pelo Autor.

A área de olho de lombo é um indicador que representa a área da seção transversal do músculo *Longissimus Dorsi*, medida em centímetro quadrados. Uma maior área é preferível pelos produtores. Do ponto de vista produtivo, pode-se afirmar que animais com valores de AOL superiores a 75 cm^2 , ao abate, apresentam elevados rendimentos de cortes cárneos na indústria frigorífica (SUGISAWA; MATOS; SUGISAWA, 2013). Na Figura 4.3 é mostrada a demarcação manual da área feita utilizando o aplicativo de manipulação e edição de imagens *ImageJ* em que a região interna a demarção amarela é a AOL (NATIONAL INSTITUTES OF HEALTH, 2026).

trocar a imagem

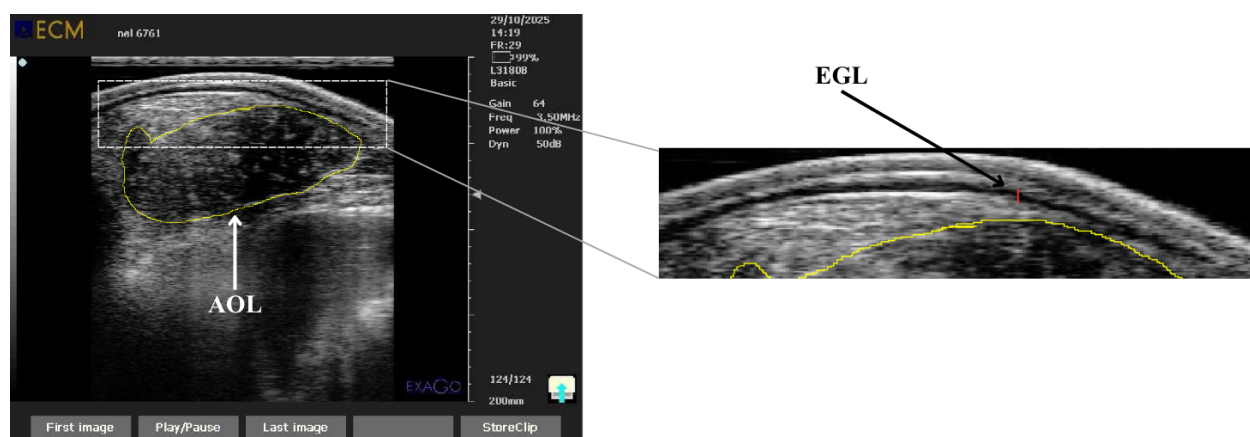
Figura 4.3: Demarcação manual da Área de Olho de Lombo (AOL) utilizando o aplicativo *ImageJ*.



Fonte: Elaborado pelo Autor.

A espessura de gordura subcutânea no lombo (EGL) é um indicador que representa a espessura da camada de gordura sobre o músculo *Longissimus Dorsi*, sendo sua espessura (EGL) medida, em milímetros, a 3/4 da distância medial do músculo. Esta gordura de cobertura é extremamente importante para a proteção da carcaça contra a rápida e intensa queda de temperatura nas câmaras frias, que pode provocar o endurecimento (perda em maciez de até 5 vezes) e o escurecimento da carne em carcaças pobres em acabamento (SUGUISAWA; MATOS; SUGUISAWA, 2013). A EGL apresenta características antagônicas à AOL, ou seja, animais com maior deposição de gordura implicará em uma menor proporção de AOL na carcaça e menor peso ao abate (SUGUISAWA; MATOS; SUGUISAWA, 2013). Na Figura 4.4 é mostrada a demarcação manual feita no aplicativo *ImageJ* em que a linha vermelha representa a demarcação da EGL.

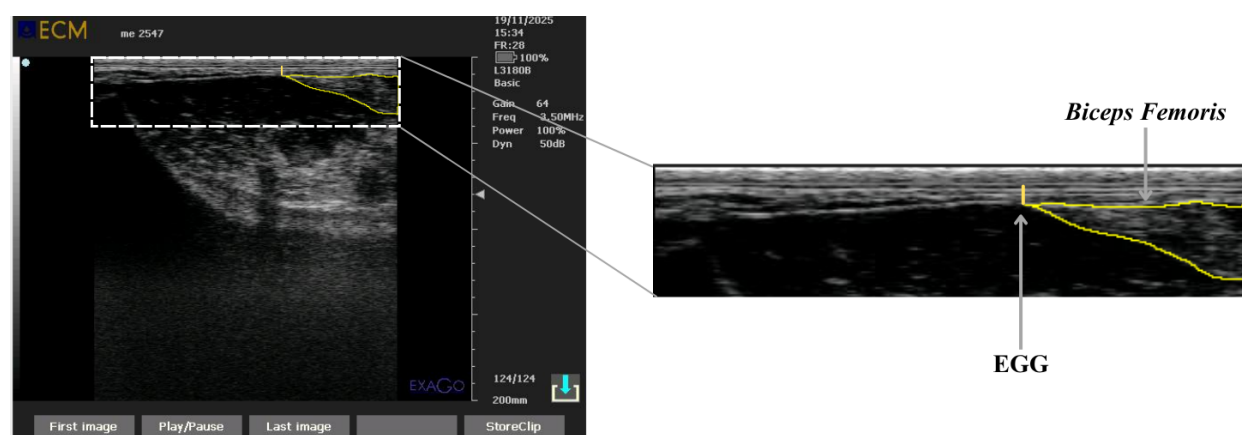
Figura 4.4: Demarcação manual da Espessura de Gordura subcutânea no Lombo (EGL) utilizando o aplicativo *ImageJ*.



Fonte: Elaborado pelo Autor.

A espessura de gordura subcutânea na garupa (EGG) é um indicador que representa a espessura da camada de gordura sobre a região da garupa, sendo medida em milímetros. A EGG é uma medida complementar à EGL, indicada principalmente para situações em que os animais têm a deposição de gordura subcutânea comprometida, principalmente por nutrição inadequada, como pode ocorrer, por exemplo, com animais em sistemas extensivos de pastejo (SUGUISAWA; MATOS; SUGUISAWA, 2013). O indicador é obtido no encontro do músculo *Biceps Femoris*, comumente chamado de ponta da picanha, com a camada de gordura subcutânea (WILSON, 1998). Na Figura 4.5 é mostrada a demarcação manual feita no aplicativo *ImageJ*.

Figura 4.5: Demarcação manual da Espessura de Gordura subcutânea na Garupa.



Fonte: Elaborado pelo Autor.

A linha vertical amarela representa a demarcação da EGG.

4.1.2 Conjunto de Dados

O total de imagens enviadas pelo Centro Avançado de Pesquisa e Desenvolvimento de Bovinos de Corte com a delimitação do contorno da região feita por uma especialista foram 134 imagens de ultrassom (134 imagens da região de olho do lombo e 134 imagens da região da garupa), sendo 108 da raça Nelore e 27 da raça Caracu. Além das imagens enviadas, foram enviadas também as medidas numéricas correspondentes à delimitação da AOL, EGL e EGG. As medidas foram enviadas em um arquivo do Excel, em que a primeira coluna continha o número de identificação do animal, a segunda coluna continha o indicador da AOL, a terceira coluna continha o indicador da EGL e a quarta coluna continha o indicador da EGG. Essas medidas foram utilizadas para calcular o erro absoluto médio (MAE) entre as medidas preditas pelo modelo e as medidas reais.

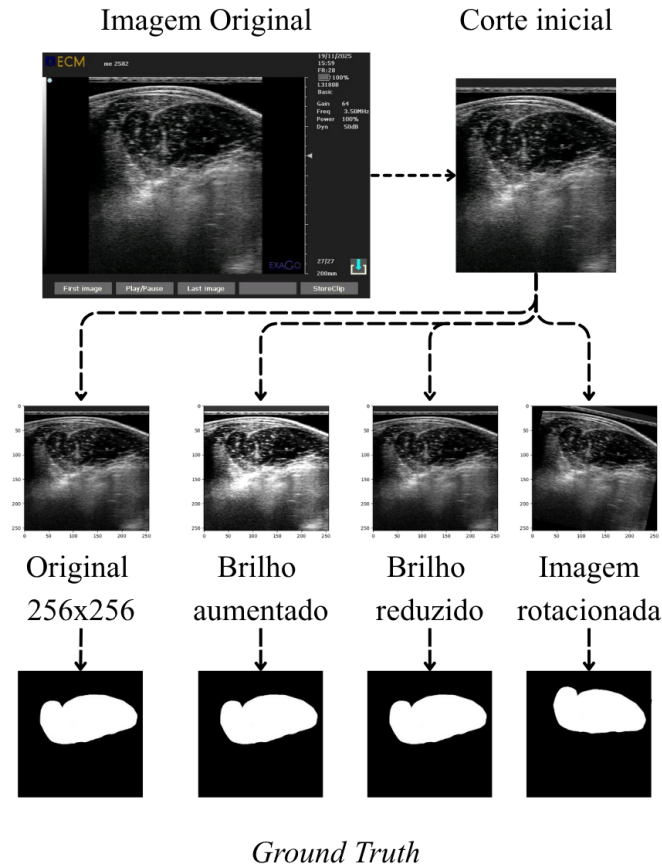
4.1.3 Pré-processamento e Máscaras de Segmentação

As imagens foram divididas em conjuntos de treinamento (70%), validação (20%) e teste (10%). A biblioteca do Python, *label studio* (HUMANSIGNAL, 2020), foi utilizada para criar o *ground truth* para as regiões correspondentes à AOL, EGL e EGG com base nas delimitações das regiões feitas pela especialista, sendo estas utilizadas como “rótulos”, ou seja, cada imagem do banco de dados tinha um *ground truth* correspondente. Devido a pequena quantidade de imagens para o treinamento de um modelo de aprendizado profundo, foi necessário aplicar um processo de aumento de dados, do inglês *data augmentation*, para ampliar o conjunto de imagens utilizado no treinamento. Além disso, foi realizado um recorte nas imagens para remover informações irrelevantes, que poderiam introduzir vieses no treinamento e comprometer o desempenho dos modelos. O processo de aumento de dados incluiu as seguintes transformações:

- Rotação: As imagens foram rotacionadas em ângulos aleatórios dentro do intervalo de $[-15^\circ, 15^\circ]$ para criar variações na orientação das imagens.
- Aumento e redução de brilho: As imagens foram aumentadas e reduzidas em brilho para simular diferentes condições de iluminação.

Todas as modificações feitas foram realizadas observando as características das imagens de ultrassom presentes no banco de dados. Como cada imagem foi modificada tres vezes, o processo de aumento de dados resultou em um total de 372 imagens para o treinamento do modelo (adicionando também as imagens originais). Após o aumento de dados, as imagens foram normalizadas, com os valores dos pixels ajustados para o intervalo $[0, 1]$, e redimensionadas para um tamanho fixo de 256×256 pixels. As imagens após o corte inicial tinham dimensões de 333×403 , a escolha de 256×256 foi feita para que a imagem não fosse muito expandida (aumento de pixels) e que não perdesse muita resolução (diminuição de pixels). Outros testes foram feitos para resoluções diferentes que resultaram em um desempenho inferior. Na Figura 4.6 é mostrado o processo de pré-processamento de uma imagem.

Figura 4.6: Exemplo do processo de pré-processamento das imagens de ultrassom.



Fonte: Elaborado pelo Autor.

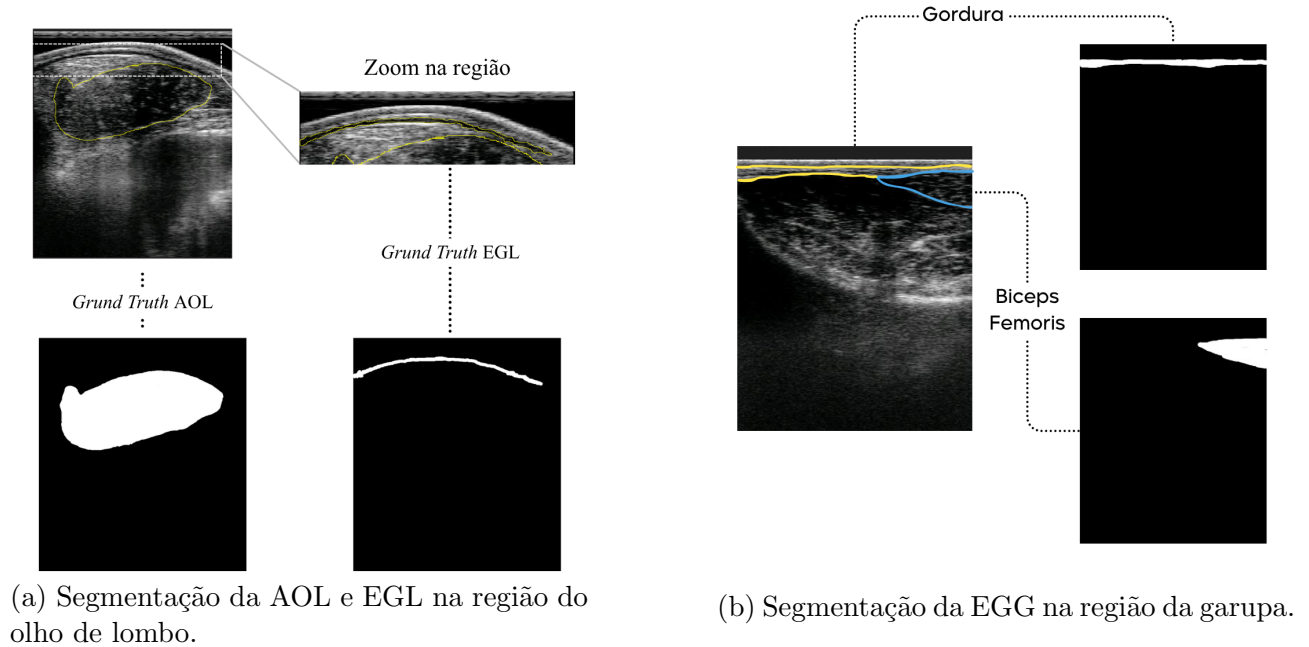
Na Figura 4.6 uma imagem é pré-processada e o *ground truth* correspondente à medida de AOL da imagem original é adicionado a nova imagem criada. Observa-se que a imagem rotacionada mantém consistência com o *ground truth*, ou seja, o *ground truth* acompanha corretamente a rotação da imagem.

Cada medida extraída nas imagens de ultrassom (AOL, EGL e EGG) são números reais. Uma ideia inicial foi desenvolver uma CNN (1.3.2) que recebia como entrada as imagens de ultrassom e retornava como saída os valores numéricos correspondentes à AOL, EGL e EGG, ou seja, uma rede de regressão. No entanto, devido à quantidade baixa de imagens disponíveis para o treinamento do modelo, foi necessário realizar uma adaptação na abordagem inicial. Em vez de treinar uma CNN para prever os valores numéricos das medidas, optou-se, após análise literária, por criar *ground truth* para cada indicador (AOL, EGL e EGG) a partir das imagens de ultrassom (MELO *et al.*, 2022). Com isso, os modelos recebem como entradas durante o treinamento, uma imagem de ultrassom junto com seu respectivo *ground truth* e retornam como saída uma máscara de segmentação..

Para construir o *ground truth* da EGL, foi necessário demarcar toda a camada de gordura a partir da qual esse indicador é obtido. De modo análogo, para o EGG, foi preciso demarcar tanto a região de gordura quanto a região do músculo *Biceps Femoris*, gerando dois *ground*

truths distintos: um correspondente à gordura e outro ao músculo, já que o indicador é obtido a partir da espessura da camada de gordura precisamente localizada no início do músculo. Na Figura 4.7 é mostrado um exemplo das máscaras de segmentação criadas para as imagens de ultrassom.

Figura 4.7: Exemplo das máscaras de segmentação criadas para as imagens de ultrassom.



Fonte: Elaborado pelo Autor.

Na Figura 4.7(a) é mostrada a delimitação da região feita pela especialista e o respectivo *ground truth*. Na Figura 4.7(b) é mostrada a delimitação da região feita pela especialista, com cada uma tendo seu próprio *ground truth*.

Cada imagem de animal presente no banco de dados vinha com um número de indentificação. Animais com o primeiro dígito começando em 6, são animais da raça Nelore, enquanto animais com o primeiro dígito começando em 2, são animais da raça Caracu. Essa indentificação serviu para terminar o desenvolvimento do banco de dados e desenvolver os modelos como abordados na seção 3.2.

4.2 Configuração dos Modelos

Esta seção descreve os procedimentos adotados para a comparação experimental entre os modelos propostos.

4.2.1 Modelos Comparados

Foram comparadas duas arquiteturas para a segmentação das imagens de ultrassom: a U-Net multitarefa e a U-Net Fuzzy. A U-Net multitarefa incorpora, no *baseline* da rede, a informação categórica da raça do animal, enquanto a U-Net Fuzzy mantém essa estrutura e adiciona uma

camada de inferência fuzzy baseada em ANFIS, conforme descrito no Capítulo 3. Em ambas as arquiteturas, a entrada do modelo durante o treinamento é composta pela imagem de ultrassom, o *ground truth* e pela codificação da raça do animal no formato *One-Hot Encoding* (Nelore ou Caracu).

A comparação foi realizada separadamente para cada um dos indicadores avaliados. Para a AOL e a EGL, foi treinado um modelo de cada arquitetura. Para a EGG, devido à necessidade de identificar simultaneamente a camada de gordura e o músculo *Biceps Femoris*, foram treinados dois modelos por arquitetura, um para cada estrutura anatômica. Assim, no total, foram avaliados oito modelos.

Tabela 4.1: Quantidade de modelos desenvolvidos para cada medida.

Tarefa de segmentação	U-Net multitarefa	U-Net Fuzzy
AOL	1	1
EGL	1	1
EGG - camada de gordura	1	1
EGG - músculo <i>Biceps Femoris</i>	1	1
Total	4	4

Fonte: Elaborado pelo Autor.

Após o treinamento, a saída de cada modelo é uma máscara de segmentação e a partir desta máscara é possível extrair os indicadores correspondentes à AOL, EGL e EGG. Para a AOL, a medida é obtida a partir da contagem do número de pixels classificados como pertencentes a região de interesse (pixels brancos) e convertida para centímetros quadrados. Para EGL, os pixels pertencentes à região de interesse são contados e convertidos em milímetros, com isso é possível obter a largura da camada de gordura, a 3/4 desta largura é obtida a altura da camada de gordura, que é a medida da EGL. Para a EGG, como duas máscaras são criadas por dois modelos diferentes, quando uma imagem é enviada e processada pelos modelos, um retorna a máscara de segmentação da camada de gordura presente na Garupa e o outro retorna a máscara de segmentação do músculo *Biceps Femoris*. A partir dessas duas máscaras é possível obter a medida da EGG, primeiramente salvando a localização da coluna que se inicia o músculo *Biceps Femoris* (da esquerda para a direita) e calculando a espessura da gordura presente na mesma coluna, ou seja, a espessura da camada de gordura na garupa que está no começo do músculo *Biceps Femoris*.

4.2.2 Configuração de Treinamento

Todos os modelos foram treinados a partir das imagens previamente recortadas, normalizadas no intervalo $[0, 1]$ e redimensionadas para 256×256 pixels. As imagens destinadas ao treinamento foram submetidas ao processo de aumento de dados descrito anteriormente, com o objetivo de ampliar a variabilidade do conjunto amostral e reduzir o risco de *overfitting*.

A configuração de treinamento foi mantida, em sua maior parte, idêntica para as duas arquiteturas, a fim de tornar a comparação entre os modelos mais consistente. Em ambos os casos, foi utilizado o otimizador Adam, Seção 1.2.6, com taxa de aprendizado inicial de 0,001, tamanho de lote igual a 4, número máximo de 100 épocas e função de custo *Binary Cross-Entropy* Seção 1.2.8. Além disso, foram empregadas camadas de *Dropout* com taxa de 0,2, Seção 1.9 e o critério de *EarlyStopping*, Seção 1.10, definido em 15 épocas, ou seja, com base no desempenho do coeficiente Dice,

explicar melhor o que é convergência

equação (3.1), no conjunto de validação, ou seja, caso o coeficiente não melhore em 15 épocas, o treinamento é interrompido e são salvos os melhores pesos do modelo.

Para a U-Net Fuzzy, além dessas configurações comuns, foram definidos 3 conjuntos fuzzy para a variável de entrada do módulo de fuzzy e 9 regras, considerando as combinações possíveis entre os conjuntos. Para reduzir o efeito da aleatoriedade inerente ao processo de treinamento, cada modelo foi treinado quatro vezes, sendo os resultados apresentados na próxima seção uma média dos valores obtidos.

Tabela 4.2: Resumo da configuração de treinamento adotada nos experimentos.

Parâmetro	Valor
Tamanho da imagem de entrada	256×256 pixels
Normalização	intervalo $[0, 1]$
Otimizador	Adam
Taxa de aprendizado inicial	0,001
Tamanho do lote	4
Número máximo de épocas	100
Função de custo	<i>Binary Cross-Entropy</i>
<i>Dropout</i>	0,2
Critério de parada	<i>EarlyStopping</i>
Número de execuções por modelo	4
Conjuntos fuzzy	3
Regras fuzzy	9

Fonte: Elaborado pelo Autor.

4.2.3 Métricas de Avaliação

O desempenho dos modelos foi analisado sob duas perspectivas complementares: a qualidade da máscara de segmentação gerada e a precisão dos indicadores extraídos a partir das máscaras preditas.

No contexto da segmentação, foi adotado o coeficiente de Dice como principal métrica de avaliação, por quantificar diretamente a sobreposição entre a máscara predita pelo modelo e o *ground truth*. A acurácia foi utilizada como métrica complementar, permitindo observar a pro-

porção global de pixels classificados corretamente. Também foram analisados o comportamento da função de custo ao longo das épocas e o número de épocas necessárias até a convergência, como indicadores da estabilidade do processo de treinamento.

No contexto da mensuração dos indicadores, foram utilizadas a reta de regressão linear, o coeficiente de determinação (R^2) e o erro absoluto médio (*Mean Absolute Error* - MAE). O coeficiente R^2 foi empregado para avaliar o grau de associação entre as medidas previstas pelos modelos e as medidas fornecidas pela especialista, enquanto o MAE foi utilizado para quantificar, em unidade física, a magnitude média do erro entre os valores estimados e os valores de referência. Dessa forma, a combinação dessas métricas permitiu avaliar não apenas a capacidade dos modelos em segmentar corretamente as regiões de interesse, mas também sua utilidade prática na obtenção das medidas AOL, EGL e EGG.

4.2.4 Ambiente Computacional

Os experimentos foram conduzidos em um notebook Asus TUF *Gaming* F15, equipado com processador Intel Core i7-12700H, 8 GB de memória RAM, unidade SSD de 512 GB e placa de vídeo NVIDIA GeForce RTX 3050 Mobile com 4 GB de memória VRAM. O sistema operacional utilizado foi o Linux Mint 22.3 “Zena” (LINUX MINT TEAM, 2026).

A implementação dos modelos foi realizada na linguagem Python, versão 3.11.15 (PYTHON SOFTWARE FOUNDATION, 1990), com uso das bibliotecas TensorFlow 2.21.0, com suporte a GPU, e Keras 3.13.2 (GOOGLE LLC, 2015; KERAS TEAM, 2015). O desenvolvimento foi realizado no ambiente Visual Studio Code (MICROSOFT, 2015). O treinamento das redes neurais foi executado com aceleração por GPU, o que contribuiu para reduzir o tempo computacional necessário para a realização dos experimentos.

4.3 Resultados e Análise

4.3.1 AOL

Nesta subseção são apresentados os resultados obtidos para a Área de Olho de Lombo (AOL), considerando tanto a qualidade da segmentação quanto a precisão das medidas numéricas extraídas a partir das máscaras previstas.

U-Net multitarefa

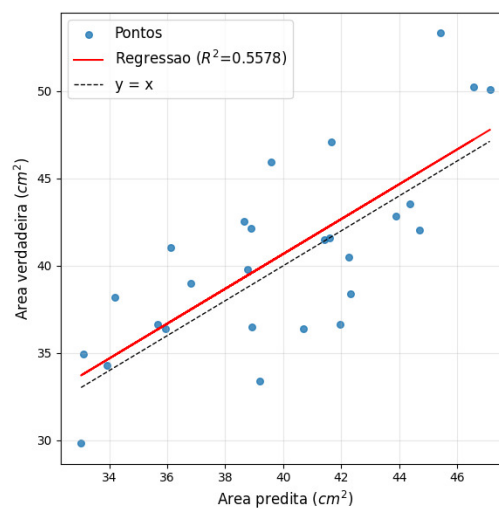
Para a AOL, o modelo U-Net multitarefa apresentou bom desempenho na tarefa de segmentação. No conjunto de validação, obteve coeficiente de Dice médio de 92,73% e acurácia média de 94,75%, indicando elevada sobreposição entre as máscaras previstas e as máscaras de referência. Em média, a convergência do treinamento ocorreu após 80 épocas.

Em relação às medidas numéricas, o modelo alcançou erro absoluto médio (MAE) de 2,988 cm² e coeficiente de determinação R^2 igual a 0,5578. No conjunto de teste, o erro máximo

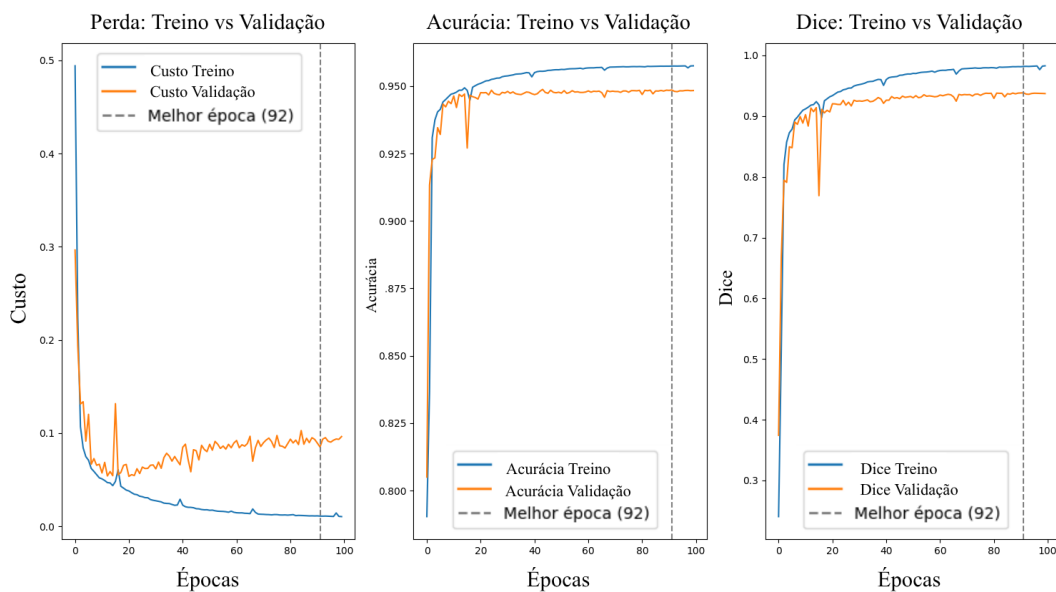
observado foi de 7,905 cm², o erro mínimo foi de 0,005 cm², a média predita foi de 39,880 cm² e a média real foi de 40,559 cm².

A Figura 4.8(a) mostra a relação entre as medidas preditas e as medidas reais da AOL. Observa-se que os pontos acompanham a reta identidade, embora com dispersão compatível com o valor moderado de R^2 . Na Figura 4.8(b), a função de custo apresenta redução até valores próximos de 0,09, enquanto as curvas de acurácia e Dice permanecem relativamente estáveis entre treinamento e validação. Embora haja indícios de *overfitting* a partir de aproximadamente 20 épocas, o modelo manteve capacidade de generalização satisfatória. A linha tracejada indica a época em que foram selecionados os melhores pesos por meio do critério de *EarlyStopping*.

Figura 4.8: Reta de regressão linear e R^2 entre as medidas preditas e as medidas reais da AOL e gráficos de custo, *Dice* e acurácia ao longo das épocas.



(a) R^2 entre as medidas preditas e as medidas reais da AOL.



(b) Gráficos de custo, *Dice* e acurácia ao longo das épocas para o modelo de U-Net multitarefa.

Fonte: Elaborado pelo Autor.

A linha tracejada na Figura 4.8(b) representa em qual época o modelo selecionou os melhores pesos por meio do critério de *EarlyStopping*.

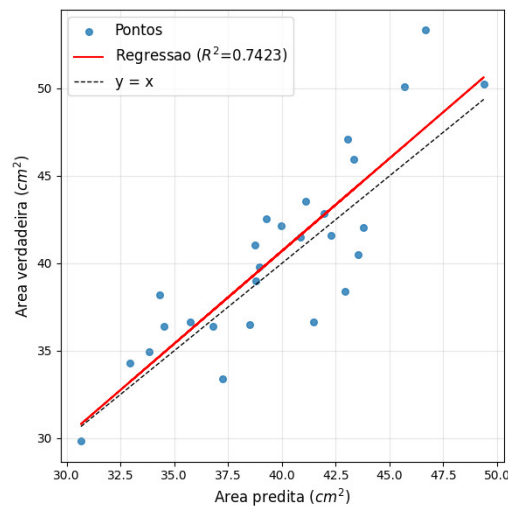
U-Net Fuzzy

Para a AOL, o modelo U-Net Fuzzy apresentou desempenho superior ao da U-Net multitarefa na maior parte das métricas analisadas. No conjunto de validação, o modelo alcançou coeficiente de Dice médio de 93,29% e acurácia média de 95,15%, indicando melhor sobreposição entre as máscaras preditas e as máscaras de referência. A convergência ocorreu, em média, após 89 épocas, valor superior ao observado para o modelo sem inferência fuzzy.

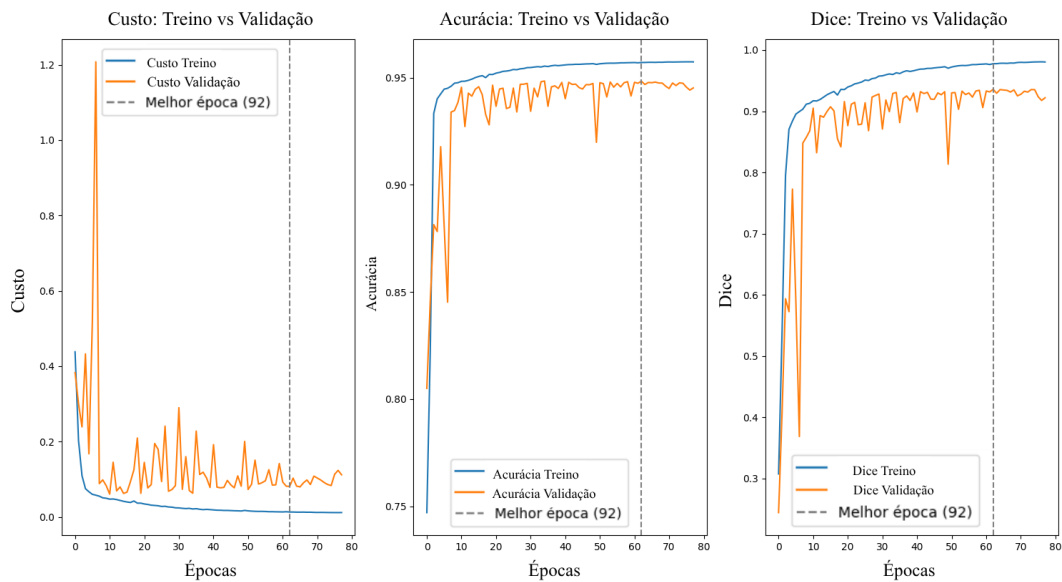
No que se refere às medidas numéricas, o modelo apresentou MAE de 2,312 cm² e R^2 de 0,7423, obtendo melhor associação entre as medidas preditas e as medidas reais da AOL. No conjunto de teste, o erro máximo foi de 6,644 cm², o erro mínimo foi de 0,225 cm², a média predita foi de 39,865 cm² com a média real sendo 40,559 cm².

Na Figura 4.9(a) é mostrado menor dispersão das medidas preditas em relação à reta identidade, o que é coerente com o aumento do valor de R^2 . Na Figura 4.9(b), observa-se que as curvas de custo, acurácia e Dice apresentam maior oscilação ao longo do treinamento. Esse comportamento pode estar associado ao aumento de complexidade introduzido pela camada de inferência fuzzy, especialmente nas etapas iniciais do processo de otimização.

Figura 4.9: R^2 entre as medidas previstas e as medidas reais da AOL e gráficos de custo, *Dice* e acurácia ao longo das épocas para o modelo de U-Net Fuzzy da AOL.



(a) R^2 entre as medidas previstas e as medidas reais da AOL.



(b) Gráficos de custo, *Dice* e acurácia ao longo das épocas para o modelo de U-Net Fuzzy da AOL.

Fonte: Elaborado pelo Autor.

Comparação entre os modelos

Na Tabela 4.3 são resumidos os resultados obtidos para a AOL. De modo geral, o modelo U-Net Fuzzy superou o modelo U-Net multitarefa nas principais métricas, apresentando maior coeficiente de Dice, maior acurácia, menor erro absoluto médio e maior coeficiente de determinação. Esses resultados indicam que a integração do ANFIS contribuiu positivamente tanto para a segmentação da região de interesse quanto para a extração dos indicadores numéricos da AOL.

Por outro lado, o modelo fuzzy exigiu maior número de épocas até a convergência e apresentou maior instabilidade nas curvas de treinamento.

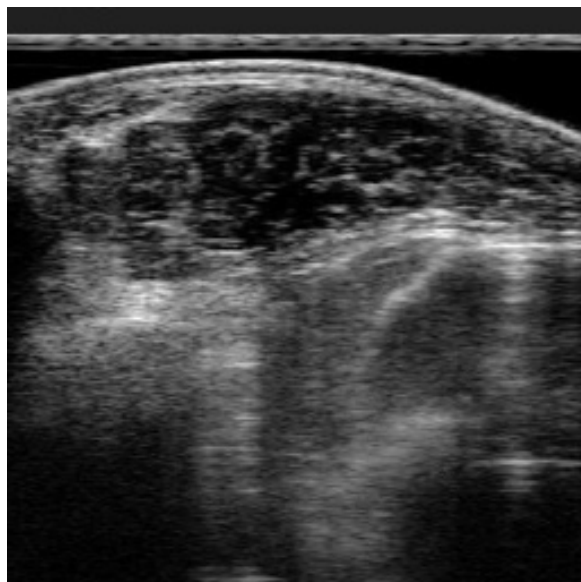
Tabela 4.3: Resumo dos resultados obtidos para a AOL, comparando o modelo de U-Net multitarefa com o modelo de U-Net Fuzzy.

Métrica	U-Net Multitarefa	U-Net Fuzzy
Coefficiente de Dice	92,73%	93,29%
Acurácia	94,75%	95,15%
Número de Épocas	80	89
R^2	0,5578	0,7423
MAE	2,988 cm ²	2,312 cm ²
Erro Máximo	7,905 cm ²	6,644 cm ²
Erro Mínimo	0,005 cm ²	0,225 cm ²
Média Predita	39,880 cm ²	39,865 cm ²
Média Real	40,559 cm ²	40,559 cm ²

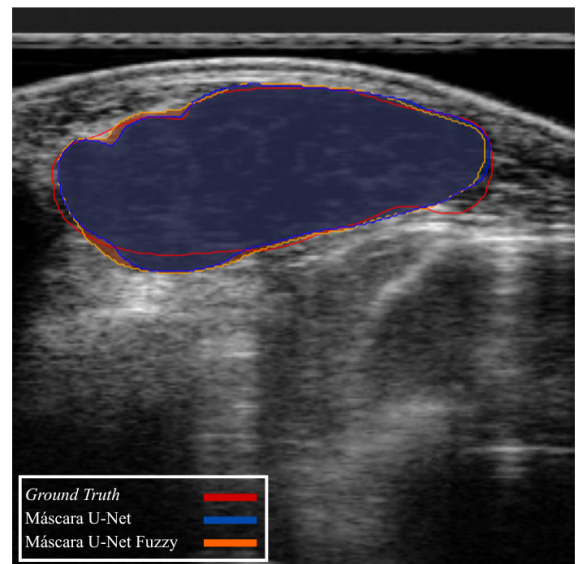
Fonte: Elaborado pelo Autor.

Na Figura 4.10 é apresentada uma comparação visual entre o *ground truth* e as máscaras de segmentação geradas pelos modelos para a mesma imagem de ultrassom. Observa-se que ambas as máscaras apresentam boa sobreposição com a referência, com diferenças sutis, o que é coerente com os valores elevados de coeficiente de Dice obtidos por ambos os modelos. As máscaras de segmentação foram modificadas para serem sobrepostas à imagem original, a fim de facilitar a visualização e comparação entre os modelos.

Figura 4.10: Comparação entre as máscaras de segmentação.



(a) Imagem de ultrassom original.



(b) *ground truth* e máscaras de segmentação geradas pelos modelos.

Fonte: Elaborado pelo Autor.

Na Figura 4.10(a) é apresentado a imagem de ultrassom original. Já Na Figura 4.10(b)

é mostrado a o *ground truth* em vermelho, a máscara de segmentação gerada pelo modelo de U-Net multitarefa em azul e a máscara de segmentação gerada pelo modelo de U-Net Fuzzy em laranja, todas sobrepostas sobre a imagem original. Observa-se que ambas as máscaras apresentam boa sobreposição com o *ground truth*, com diferenças bem sutis, o que é coerente com os valores elevados de coeficiente de Dice obtidos por ambos os modelos.

4.3.2 EGL

Nesta subseção são apresentados os resultados obtidos para a Espessura de Gordura subcutânea no Lombo (EGL). Em comparação com a AOL, trata-se de uma tarefa mais desafiadora, pois a camada de gordura apresenta menor espessura e menor contraste em relação ao fundo da imagem.

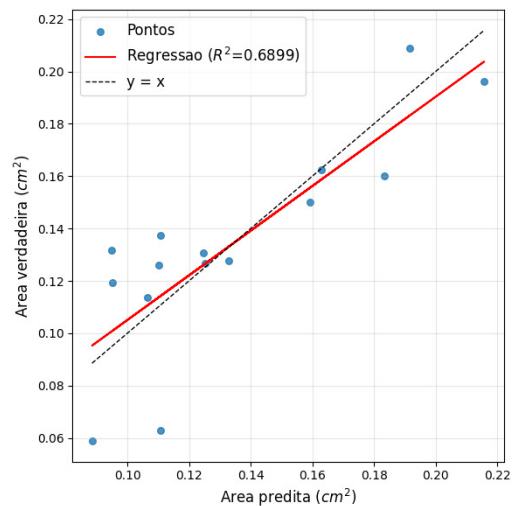
U-Net multitarefa

Para a EGL, o modelo U-Net multitarefa apresentou desempenho intermediário na tarefa de segmentação. No conjunto de validação, obteve coeficiente de Dice médio de 60,78% e acurácia média de 99,99%. Apesar da acurácia elevada, o coeficiente de Dice foi consideravelmente inferior ao observado para a AOL, o que indica maior dificuldade na delimitação da camada de gordura. Esse comportamento pode ser explicado pelo fato de a região segmentada ser estreita e ocupar uma pequena fração da imagem, tornando a tarefa mais sensível a pequenas variações de contorno. Em média, a convergência ocorreu após 88 épocas.

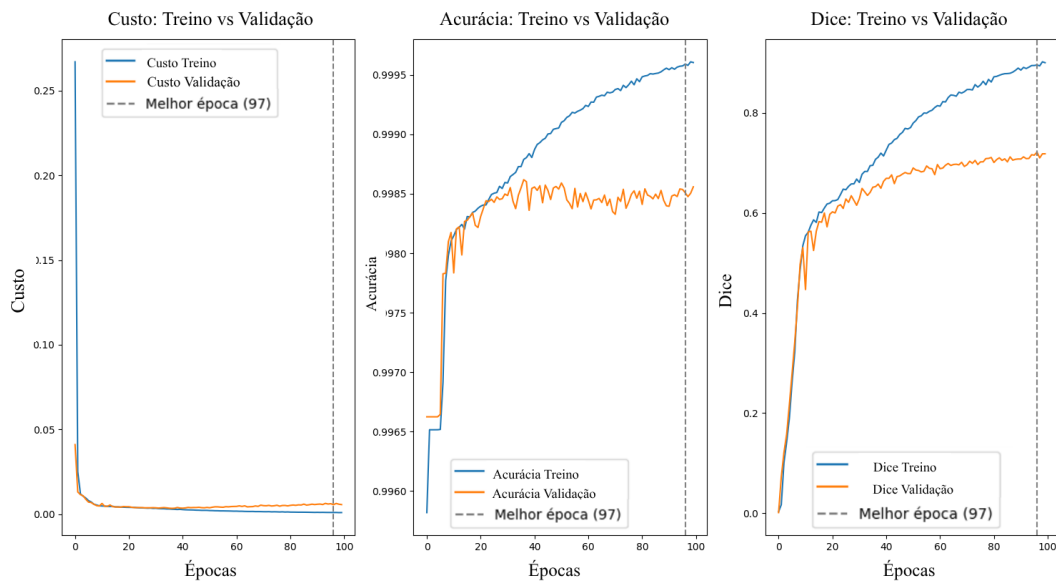
No que se refere às medidas numéricas, o modelo apresentou MAE de 0,018 mm e coeficiente de determinação R^2 de 0,6899. No conjunto de teste, o erro máximo observado foi de 0,48 mm, o erro mínimo foi de 0,01 mm, e as médias predita e real foram ambas de 0,134 mm.

Na Figura 4.11(a) é mostrada a relação entre as medidas preditas e as medidas reais da EGL. Já na Figura 4.11(b) é mostrado o comportamento das curvas de custo, Dice e acurácia ao longo do treinamento. Observa-se que a função de custo converge rapidamente para valores muito baixos, o que não implica necessariamente boa segmentação da camada de gordura. Isso ocorre porque a maior parte dos pixels pertence ao fundo da imagem, o que favorece soluções triviais e explica, em parte, a discrepância entre a acurácia elevada e o coeficiente de Dice mais baixo.

Figura 4.11: Reta de regressão linear e R^2 entre as medidas previstas e as medidas reais da EGL e gráficos de custo, *Dice* e acurácia ao longo das épocas.



(a) R^2 entre as medidas previstas e as medidas reais da EGL.



(b) Gráficos de custo, *Dice* e acurácia ao longo das épocas para o modelo de U-Net multitarefa da EGL.

Fonte: Elaborado pelo Autor.

U-Net Fuzzy

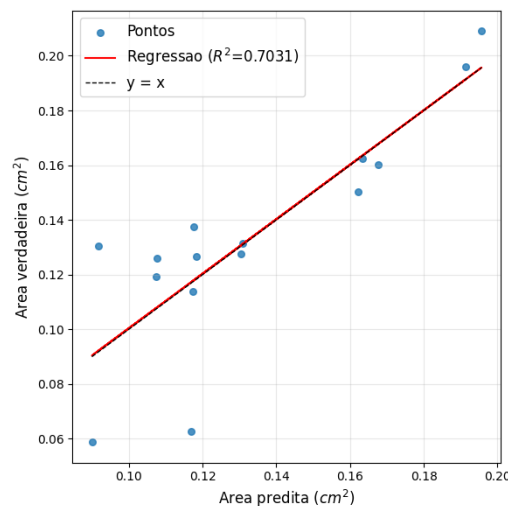
Para a EGL, o modelo U-Net Fuzzy apresentou desempenho semelhante ao da U-Net multitarefa na tarefa de segmentação, com pequenas diferenças entre as métricas analisadas. O modelo alcançou coeficiente de Dice médio de 69,39% e acurácia média de 99,50%. A convergência ocorreu, em média, após 94 épocas.

Em relação às medidas numéricas, o modelo apresentou MAE de 0,015 mm e coeficiente de determinação R^2 de 0,7031, indicando leve melhora na associação entre as medidas previstas e

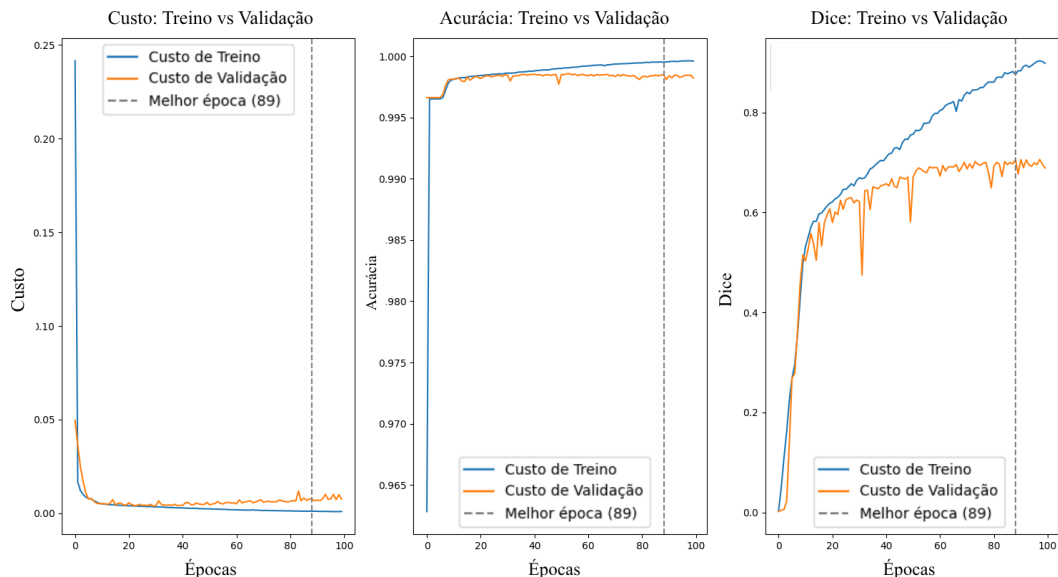
as medidas reais. No conjunto de teste, o erro máximo foi de 0,54 mm, o erro mínimo foi de 0,01 mm, e as médias predita e real foram de 0,134 mm.

Na Figura 4.12(a) é mostrada a relação entre as medidas preditas e as medidas reais da EGL para o modelo fuzzy. Na Figura 4.12(b), observa-se que as curvas de custo, acurácia e Dice apresentam maior oscilação ao longo do treinamento, padrão semelhante ao observado para a AOL. Esse comportamento pode estar associado à maior complexidade introduzida pela camada de inferência fuzzy.

Figura 4.12: Reta de regressão linear e R^2 entre as medidas preditas e as medidas reais da EGL e gráficos de custo, *Dice* e acurácia ao longo das épocas.



(a) R^2 entre as medidas preditas e as medidas reais da EGL.



(b) Gráficos de custo, *Dice* e acurácia ao longo das épocas para o modelo de U-Net Fuzzy da EGL.

Fonte: Elaborado pelo Autor.

Comparação entre os modelos

Na Tabela 4.4 são resumidos os resultados obtidos para a EGL. De forma geral, as diferenças entre os dois modelos foram pequenas. O modelo U-Net multitarefa apresentou ligeira vantagem nas métricas de segmentação e no número de épocas até a convergência, enquanto o modelo U-Net Fuzzy apresentou resultados ligeiramente melhores para MAE e R^2 .

Tabela 4.4: Resumo dos resultados obtidos para a EGL, comparando o modelo de U-Net multitarefa com o modelo de U-Net Fuzzy.

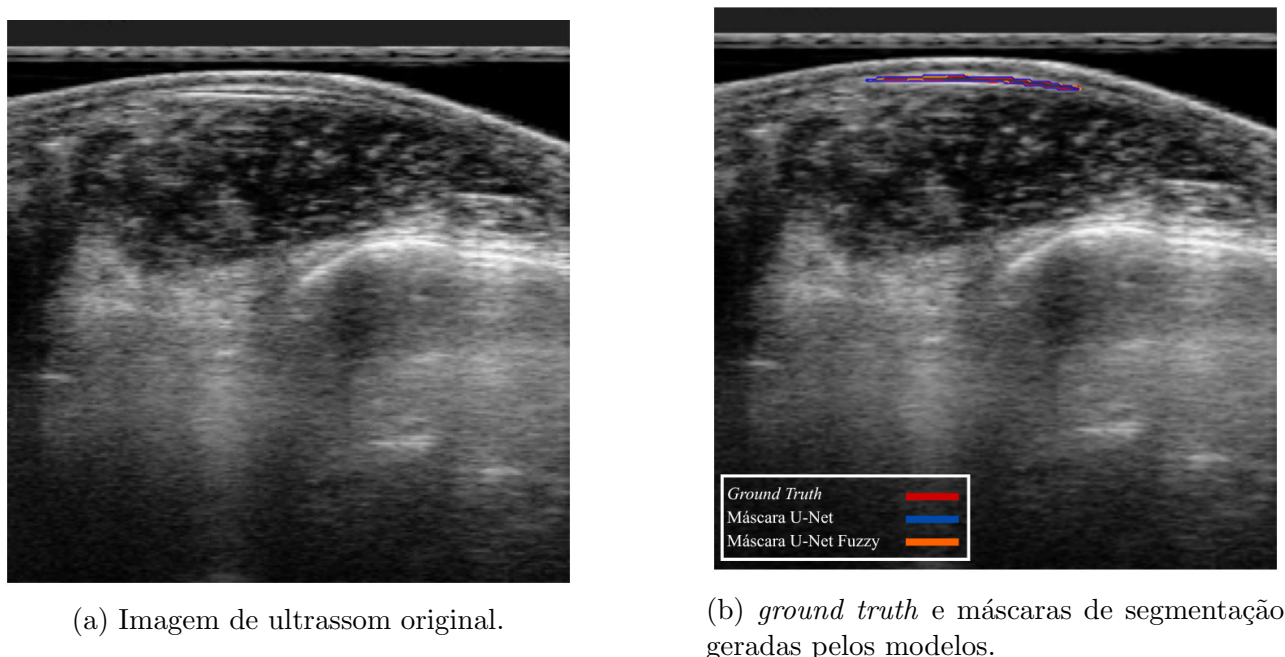
Métrica	U-Net Multitarefa	U-Net Fuzzy
Coefficiente de Dice	70,25%	69,39%
Acurácia	99,85%	99,50%
Número de Épocas	88	94
R^2	0,6899	0,7031
MAE	0,018 mm	0,015 mm
Erro Máximo	0,48 mm	0,54 mm
Erro Mínimo	0,01 mm	0,01 mm
Média Predita	0,134 mm	0,134 mm
Média Real	0,134 mm	0,134 mm

Fonte: Elaborado pelo Autor.

Esse resultado sugere que, para a EGL, a adição do módulo fuzzy não produziu uma vantagem tão clara quanto a observada na AOL. A própria natureza da tarefa, marcada por forte desbalanceamento entre a região de interesse e o fundo da imagem, parece limitar o ganho obtido por ambas as arquiteturas.

Na Figura 4.13 é apresentada uma comparação visual entre o *ground truth* e as máscaras de segmentação geradas pelos modelos. Observa-se que ambas as máscaras apresentam boa sobreposição visual com a referência, com diferenças discretas. O coeficiente de Dice relativamente menor para os dois modelos decorre do fato de a região segmentada ser pequena, de modo que pequenas diferenças no contorno resultam em variações relativamente grandes nessa métrica.

Figura 4.13: Comparação entre as máscaras de segmentação.



Fonte: Elaborado pelo Autor.

Na Figura 4.13(a) é apresentado a imagem de ultrassom original. Já Na Figura 4.13(b) é mostrado a o *ground truth* em vermelho, a máscara de segmentação gerada pelo modelo U-Net multitarefa em azul e a máscara de segmentação gerada pelo modelo U-Net Fuzzy em laranja, todas sobrepostas sobre a imagem original.

4.3.3 EGG

Nesta subseção são apresentados os resultados obtidos para a Espessura de Gordura subcutânea na Garupa (EGG). Diferentemente dos indicadores anteriores, a obtenção da EGG depende de duas segmentações distintas: uma para a camada de gordura e outra para o músculo *Biceps Femoris*. Assim, eventuais erros em qualquer uma dessas etapas podem se propagar para a medida final.

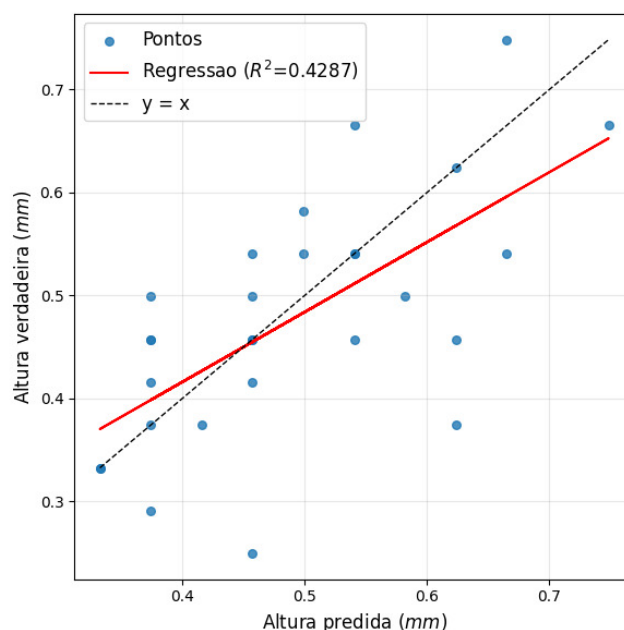
U-Net multitarefa

Para a EGG, o modelo U-Net multitarefa apresentou coeficiente de Dice médio de 82,74% para a segmentação do músculo *Biceps Femoris* e 79,84% para a segmentação da região de gordura. As acurácias médias correspondentes foram de 98,75% e 96,87%, respectivamente. A convergência ocorreu, em média, após 31 épocas para o músculo *Biceps Femoris* e 61 épocas para a região de gordura.

Em relação às medidas numéricas, o modelo apresentou MAE de 0,7 mm e coeficiente de determinação R^2 de 0,4287. No conjunto de teste, o erro máximo observado foi de 2,5 mm, o erro mínimo foi de 0,00 mm, a média predita foi de 4,915 mm e a média real foi de 4,781 mm.

Na Figura 4.14 é mostrada a relação entre as medidas preditas e as medidas reais da EGG para o modelo U-Net multitarefa. Observa-se alta dispersão em relação à reta identidade, o que sugere maior sensibilidade da medida a pequenas variações na localização do início do músculo *Biceps Femoris*. Em outras palavras, mesmo pequenas diferenças na segmentação podem produzir mudanças relativamente grandes no valor final da EGG.

Figura 4.14: Reta de regressão linear e R^2 entre as medidas preditas e as medidas reais da EGG para o modelo de U-Net multitarefa.



Fonte: Elaborado pelo Autor.

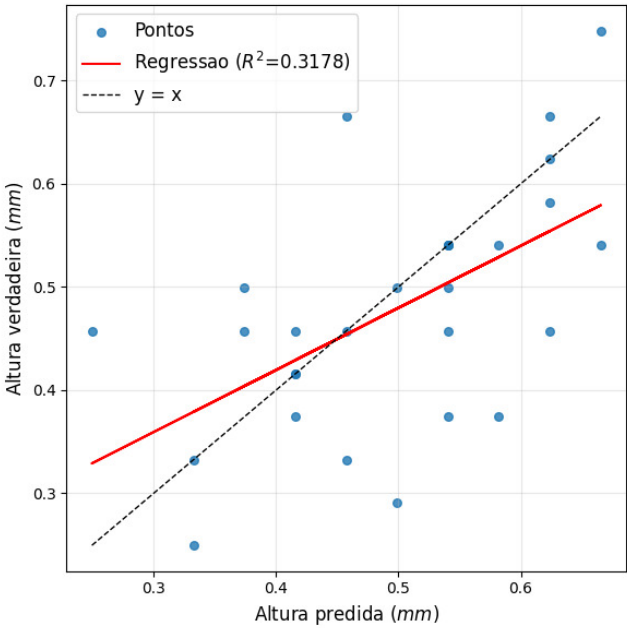
U-Net Fuzzy

O modelo U-Net Fuzzy apresentou coeficiente de Dice médio de 84,60% para a segmentação do músculo *Biceps Femoris* e 79,99% para a segmentação da região de gordura. As acurácias médias correspondentes foram de 98,60% e 96,88%, respectivamente. A convergência ocorreu, em média, após 45 épocas para o músculo e 76 épocas para a região de gordura.

Para a medida numérica da EGG, o modelo apresentou MAE de 0,8 mm e coeficiente de determinação R^2 de 0,3178. No conjunto de teste, o erro máximo observado foi de 2,1 mm, o erro mínimo foi de 0,00 mm, a média predita foi de 4,974 mm e a média real foi de 4,781 mm.

A Figura 4.15 mostra a relação entre as medidas preditas e as medidas reais da EGG para o modelo U-Net Fuzzy. Assim como no modelo multitarefa, observa-se dispersão considerável em relação à reta identidade, o que reforça a dificuldade da tarefa e a sensibilidade da medida à localização precisa do início do músculo *Biceps Femoris*.

Figura 4.15: Reta de regressão linear e R^2 entre as medidas previstas e as medidas reais da EGG para o modelo de U-Net Fuzzy.



Fonte: Elaborado pelo Autor.

Comparação entre os modelos

Na Tabela 4.5 são resumidos os resultados obtidos para a EGG. Em termos de segmentação, os dois modelos apresentaram desempenho semelhante, com leve vantagem da U-Net Fuzzy nos valores de Dice. Entretanto, ao se analisar a medida final da EGG, o modelo U-Net multitarefa apresentou melhor desempenho, com menor MAE e maior coeficiente de determinação.

Tabela 4.5: Resumo dos resultados obtidos para a EGG, comparando o modelo de U-Net multitarefa com o modelo de U-Net Fuzzy.

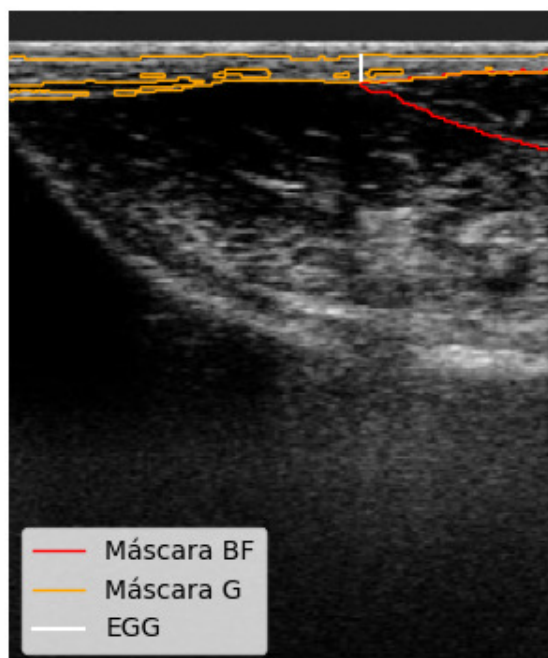
Métrica	U-Net Multitarefa	U-Net Fuzzy
Coeficiente de Dice	82,74% e 79,84%	84,60% e 79,99%
Acurácia	98,75% e 96,87%	98,60% e 96,88%
Número de Épocas	31 e 61	45 e 76
R^2	0,4287	0,3178
MAE	0,7 mm	0,8 mm
Erro Máximo	2,5 mm	2,1 mm
Erro Mínimo	0,00 mm	0,00 mm
Média Predita	4,915 mm	4,974 mm
Média Real	4,781 mm	4,781 mm

Fonte: Elaborado pelo Autor.

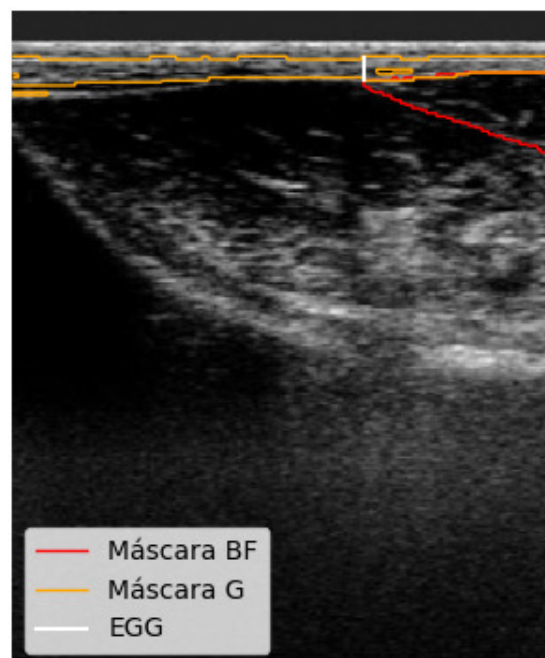
Na Figura 4.16 são apresentadas uma comparação visual entre o modelo U-Net multitarefa,

Figura 4.16(a), e o modelo U-Net Fuzzy, Figura 4.16(b), para a mesma imagem de ultrassom. Observa-se que ambas as máscaras apresentam boa segmentação das regiões, embora haja diferenças sutis na delimitação do início do músculo *Biceps Femoris*, o que pode explicar a maior dispersão observada na medida da EGG em ambos os modelos. As máscaras de segmentação

Figura 4.16: Comparação entre as máscaras de segmentação para a EGG.



(a) Máscaras de segmentação geradas pelo modelo U-Net multitarefa.



(b) Máscaras de segmentação geradas pelo modelo U-Net Fuzzy.

Fonte: Elaborado pelo Autor.

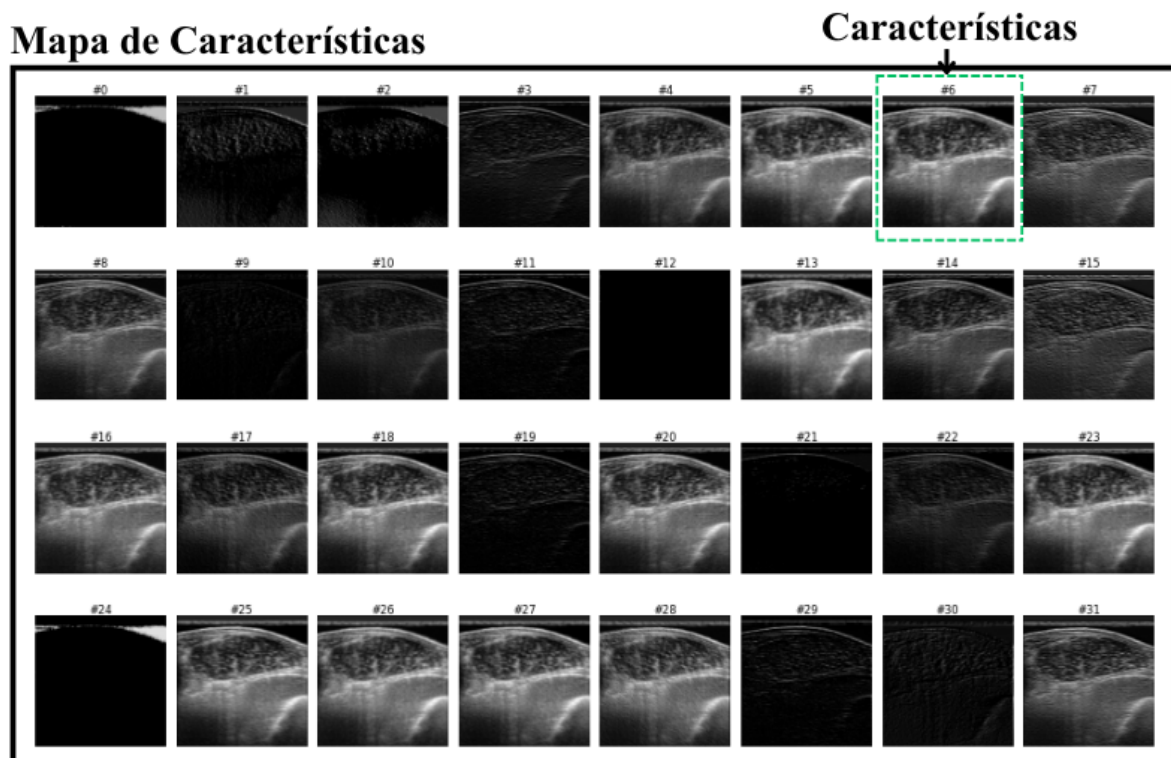
estão em formato de delimitação, ou seja, o interior da máscara não está sendo mostrado, isso foi escolhido para facilitar a visualização da região segmentada. A cor vermelha representa a máscara de segmentação do músculo *Biceps Femoris*, a cor laranja representa a máscara de segmentação da região de gordura e a cor branca representa o local em que foi obtido o valor do indicador da EGG. Observa-se que ambas as máscaras apresentam boa segmentação das regiões, embora haja diferenças sutis na delimitação do início do músculo *Biceps Femoris*, o que pode explicar a maior dispersão observada na medida da EGG em ambos os modelos.

Esse resultado sugere que, para a EGG, pequenas melhorias na segmentação isolada das estruturas anatômicas não se traduziram necessariamente em maior precisão da medida numérica final. Como a EGG depende da combinação de duas segmentações e da localização exata do início do músculo *Biceps Femoris*, a propagação de pequenos erros compromete de forma mais acentuada a medida final. Nesse contexto, os resultados indicam vantagem da U-Net multitarefa para a obtenção da medida numérica da EGG.

4.4 Análise Geral dos Resultados

De forma geral, os resultados obtidos para os três indicadores analisados (AOL, EGL e EGG) indicam que a integração do módulo de inferência fuzzy na arquitetura de U-Net proporcionou melhorias na qualidade da segmentação, especialmente para a AOL. Como desenvolvido anteriormente, a arquitetura U-Net multitarefa/U-Net Fuzzy é formada por camadas de convolução, camadas de pooling e camadas de convolução transposta, além de conexões de salto entre as camadas correspondentes. Quando a imagem é processada pela primeira camada de convolução, são extraídas características e a imagem resultante passa a ser conhecida como mapa de características. Para entender o motivo do nome, é necessário visualizar o que acontece em cada etapa de processamento das arquiteturas. Na Figura 4.17 é mostrado o processo de extração de características e geração do mapa de características na primeira etapa de convolução dos modelos, para o indicador de AOL. Esta etapa recebe uma imagem ($256 \times 256 \times 1$) e transforma em um mapa de características ($256 \times 256 \times 32$), ou seja, a imagem é processada por 32 *kernels* de convolução, resultando em 32 características, cada um com as mesmas dimensões da imagem original. Alguns mapas de características presentes nesta seção não apresentam todas as características extraídas, isso é feito para facilitar a visualização, pois o mapa de características pode conter centenas de características, o que dificultaria a visualização. Na Figura 4.17 é mostrada

Figura 4.17: Processo de extração de características e geração do mapa de características na primeira etapa de convolução da U-Net.



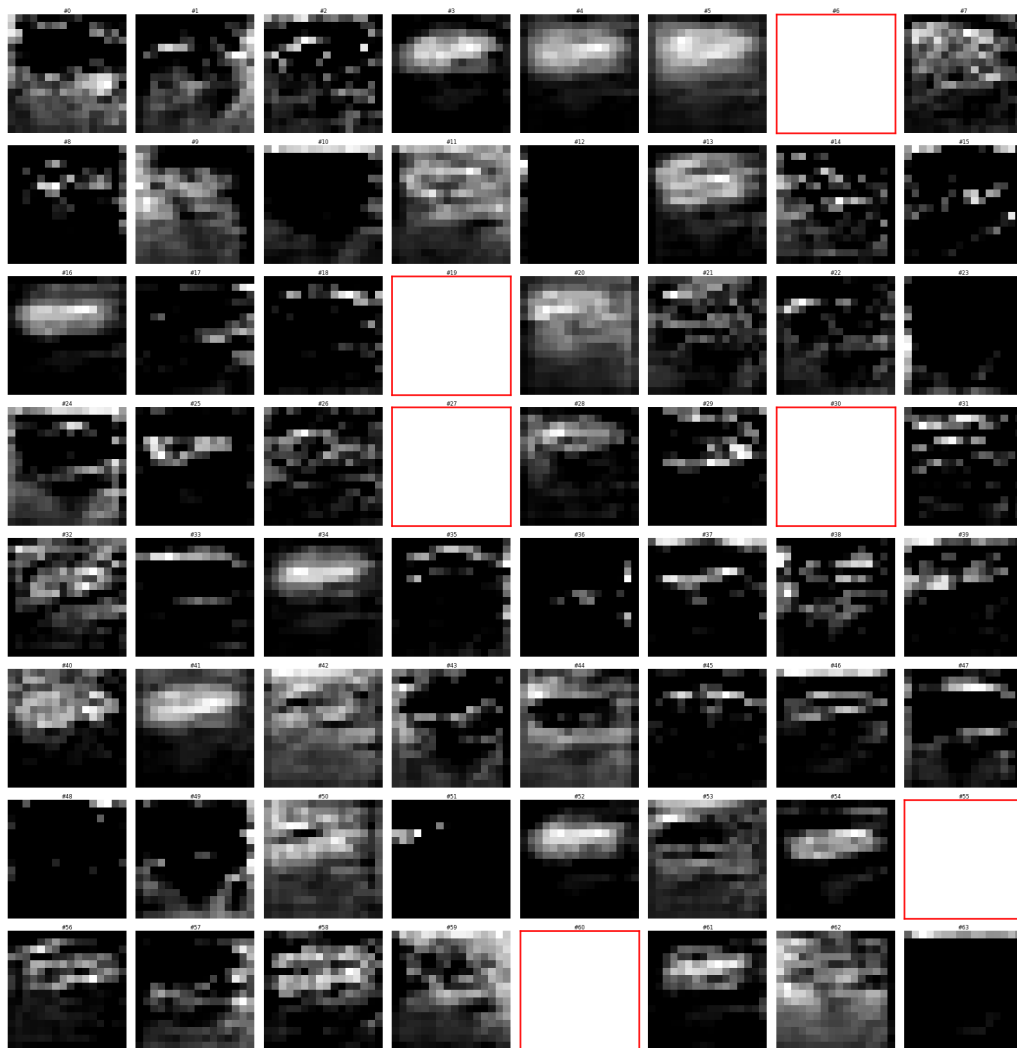
Fonte: Elaborado pelo Autor.

como é gerado o mapa de características na primeira etapa de convolução, após a época 60, isso implica que a rede já passou por um processo de otimização dos pesos e as características ex-

traídas não são aleatórias, o que acontece em épocas iniciais, mas sim características relevantes para a rede produzir a máscara de segmentação.

Em cada etapa de convolução nem todas as características são relevantes para a rede criar a máscara de segmentação da imagem, observa-se por exemplo, que na última etapa de convolução do *baseline* (antes de ser enviado para a camada multitarefa ou para o módulo Fuzzy), presente na Figura 4.18, algumas características não foram ativadas pela rede, ou seja, apresentam valores próximos de zero. Isso não significa que esta característica é irrelevante, mas sim que para a imagem de entrada, a rede não encontrou informações relevantes para ativar esta característica. As características que não foram ativadas estão brancas e com a borda vermelha para facilitar a visualização.

Figura 4.18: Mapa de características na última etapa de convolução do *baseline* da U-Net.



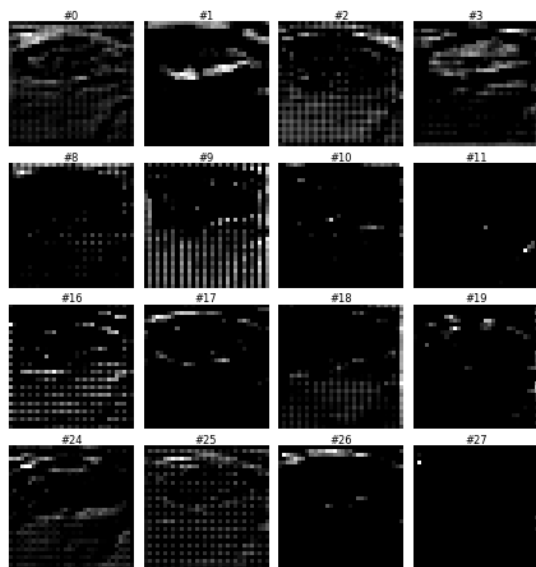
Fonte: Elaborado pelo Autor.

Observa-se que em algumas subfiguras da Figura 4.18 a rede extraiu características relevantes para segmentar a região de AOL. Nesta etapa de convolução o mapa de características tem dimensões de $(16 \times 16 \times 256)$, ou seja, o mapa de características anterior foi processado por 256 filtros de convolução, resultando em 256 características, cada uma com dimensões de 16×16 .

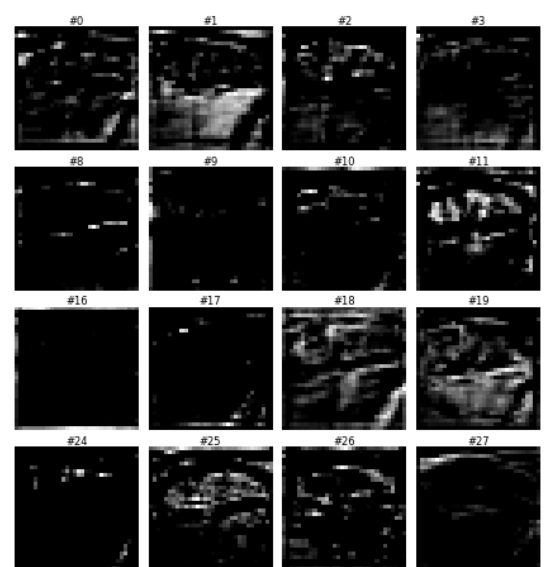
Para ser possível visualizar essas características na Figura 4.18 foram selecionadas apenas 64 características.

É no início do *decoder* que acontece a principal diferença entre os modelos, ou seja, a adição do módulo de inferência fuzzy. A U-Net Fuzzy apresenta o módulo de inferência fuzzy que recebe o mapa de características da última etapa de convolução do *baseline*, após a camada de multitarefa, e processa este mapa para gerar um novo mapa de características, que é enviado para o *decoder*. Na Figura 4.19 é possível visualizar as diferenças entre os mapas de características gerados, com o mapa na Figura 4.19(a) passando pelo módulo de inferência fuzzy e o mapa na Figura 4.19(b) não. Estes mapas foram gerados no primeiro bloco do *decoder*. Na Fi-

Figura 4.19: Mapa de características gerado com o módulo de inferência fuzzy e mapa de características gerado sem o módulo de inferência fuzzy.



(a) Mapa de características gerado pelo módulo de inferência fuzzy.



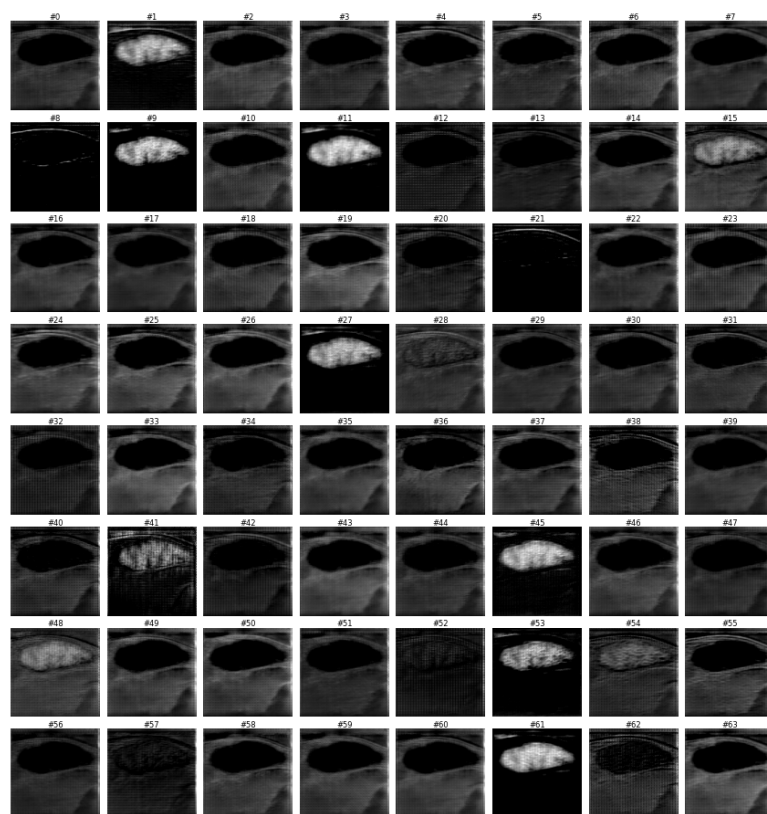
(b) Mapa de características sem o módulo de inferência fuzzy.

Fonte: Elaborado pelo Autor.

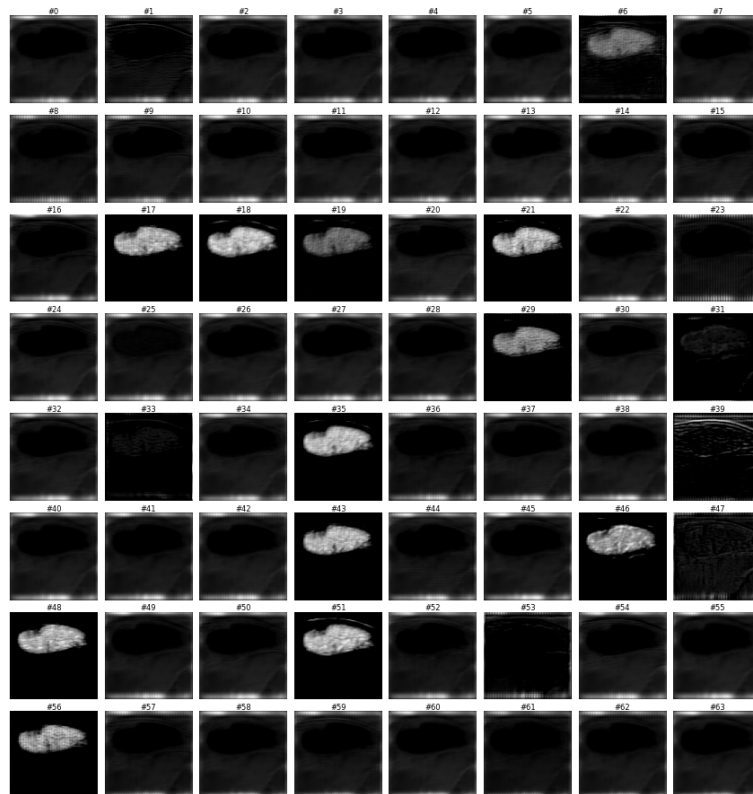
gura 4.19(a) é possível notar um padrão quadriculado em algumas características, isso acontece devido a convolução transposta, Seção 1.3.9, e como foi escolhido plotar os mapas de características, com cada pixel passando por uma normalização para o intervalo $[0, 1]$ ressaltando ainda mais o padrão quadriculado.

Como os modelos diferem na saída do *baseline*, o *decoder* de cada arquitetura passa a reconstruir a imagem de maneira distinta. Isso pode ser observado na Figura 4.20(a), em que são apresentados os mapas de características gerados no terceiro bloco do *decoder* da U-Net Fuzzy e da U-Net multitarefa (Figura 3.13). No caso da U-Net Fuzzy, o mapa de características evidencia com maior nitidez a separação entre a região correspondente à AOL e as demais regiões da imagem de ultrassom. Já no modelo U-Net multitarefa, o mapa de características apresenta um padrão mais homogêneo com menor distinção entre a região de AOL e o restante da imagem, como é possível ser observado na Figura 4.20(b).

Figura 4.20: Mapa de características gerado no terceiro bloco do *decoder* da U-Net Fuzzy e da U-Net multitarefa.



(a) Mapa de características do *decoder* da U-Net Fuzzy.



(b) Mapa de características do *decoder* da U-Net multitarefa.

Fonte: Elaborado pelo Autor.

Como é possível perceber nos resultados apresentados para a AOL, isso se traduz em uma máscara de segmentação mais precisa da região de interesse, o que impacta positivamente na extração da medida numérica.

4.5 Aplicativo Computacional

Com o objetivo de facilitar a utilização dos modelos desenvolvidos, foi criado um aplicativo computacional com interface gráfica, utilizando a biblioteca *Tkinter* do Python. O aplicativo permite que o usuário selecione uma imagem de ultrassom, escolha o modelo desejado (U-Net multitarefa ou U-Net Fuzzy) e visualize a máscara de segmentação gerada, bem como os valores numéricos dos indicadores (AOL, EGL e EGG). O aplicativo é dividido em três etapas principais.

- A primeira etapa é destinada à seleção do indicador que será medido (AOL, EGL ou EGG) e o modelo que será utilizado para a segmentação (U-Net multitarefa ou U-Net Fuzzy). Na Figura 4.21 é mostrado a tela inicial do aplicativo, em que é selecionado o indicador AOL e o modelo U-Net Fuzzy.

Figura 4.21: Tela inicial do aplicativo para seleção do indicador e modelo.

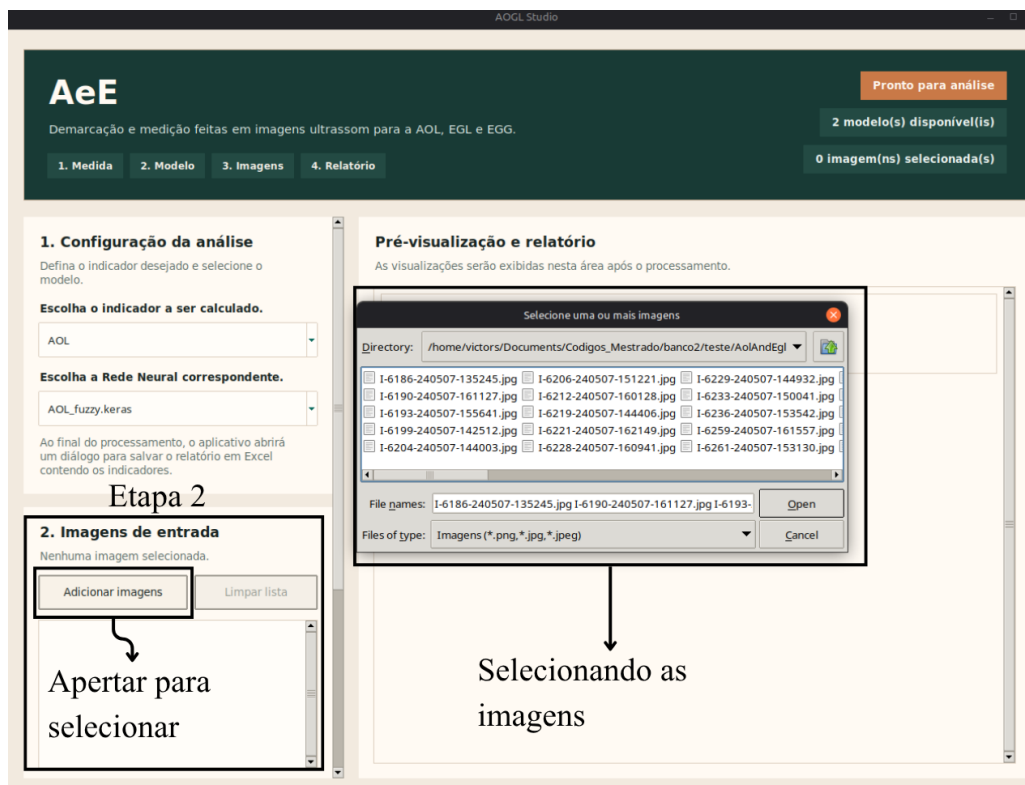


Fonte: Elaborado pelo Autor.

- A segunda etapa é responsável pela seleção da(s) imagem(ns) de ultrassom a ser(em) processada(s). O usuário pode selecionar uma ou mais imagens, e o aplicativo irá processar cada uma delas utilizando o modelo selecionado na etapa anterior. Na Figura 4.22 é

mostrado a tela de seleção das imagens, em que foram selecionadas três imagens de ultrassom para serem processadas.

Figura 4.22: Tela de seleção das imagens de ultrassom a serem processadas.



Fonte: Elaborado pelo Autor.

Caso o indicador seja o AOL, todas as imagens selecionadas precisam ser do músculo *longuissimos dorsi*, para que seja possível medir corretamente. A imagem selecionada não precisa passar por nenhum processo de pré-processamento, ou seja, o aplicativo é capaz de processar a imagem original, sem a necessidade de redimensionamento ou normalização prévia. O aplicativo realiza automaticamente o pré-processamento necessário para que a imagem seja compatível com os modelos de segmentação.

- A terceira etapa é a visualização dos resultados. Após o processamento das imagens, o aplicativo exibe a máscara de segmentação gerada para cada imagem, sobreposta à imagem original, e apresenta os valores numéricos dos indicadores correspondentes, gerando também um relatório em formato de Excel, contendo o número do animal e a medida do indicador. Na Figura 4.23 é mostrado a tela de visualização dos resultados, em que são exibidas as máscaras de segmentação para as três imagens selecionadas e os valores numéricos do indicador AOL.

Figura 4.23: Tela de visualização dos resultados, com as máscaras de segmentação sobrepostas à imagem original e os valores numéricos do indicador.

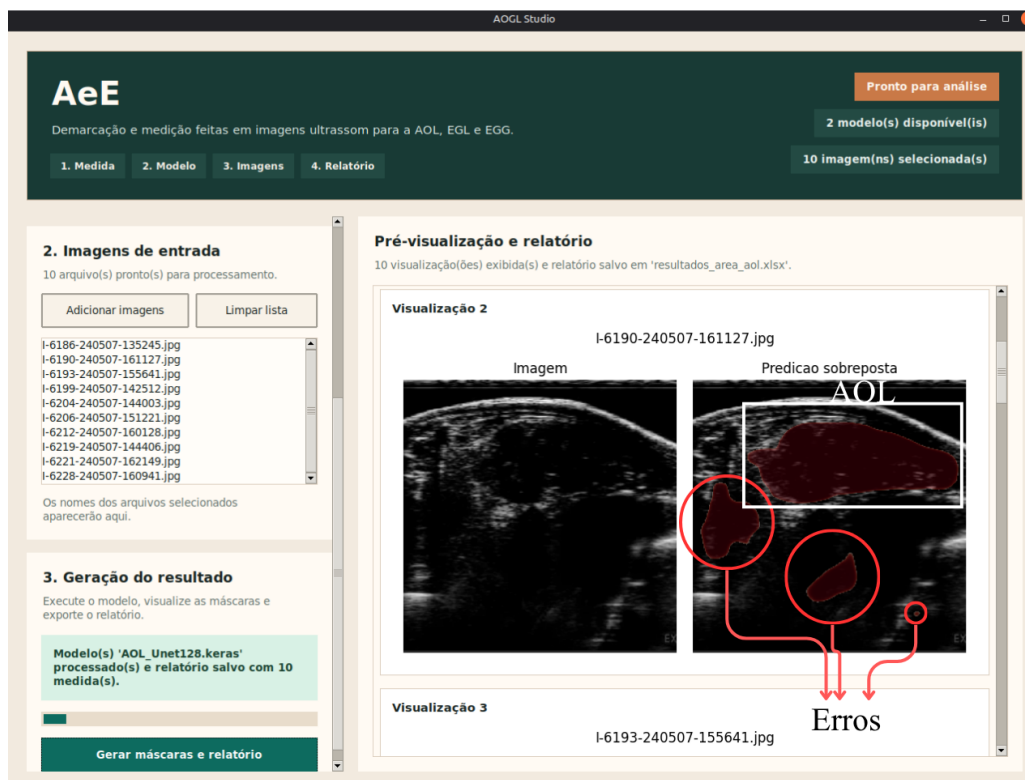


Fonte: Elaborado pelo Autor.

Antes que se possa visualizar os resultados, é necessário salvar o relatório gerado, com o aplicativo solicitando o local onde o arquivo Excel deve ser salvo.

O aplicativo contém algumas funções que reduzem o erro da rede, como por exemplo, para a AOL, caso a imagem de entrada seja ruim e a rede “alucine”, gerando regiões que não correspondem à região de interesse, o aplicativo contém funções que eliminam regiões menores que um determinado número de pixels, o que ajuda a reduzir o impacto de segmentações errôneas. Na Figura 4.24 é mostrado um exemplo de uma imagem de ultrassom com baixa qualidade, em que a rede gerou uma segmentação errônea, e o aplicativo calculou apenas a região maior, eliminando as regiões menores.

Figura 4.24: Exemplo de uma imagem de ultrassom com baixa qualidade, em que a rede gerou uma segmentação errônea, e o aplicativo calculou apenas a região maior, eliminando as regiões menores.



Fonte: Elaborado pelo Autor.

Posto isso, neste capítulo foi abordado o contexto do problema, a manipulação feita no banco de dados, os resultados das redes neurais utilizadas e o desenvolvimento do aplicativo. No próximo capítulo, são apresentadas as considerações finais do trabalho, incluindo as principais conclusões, limitações e sugestões para trabalhos futuros.

Capítulo 5

Considerações Finais

Uma das principais dificuldades encontradas durante o desenvolvimento do trabalho foi a qualidade das imagens de ultrassom. Para que uma imagem seja considerada de boa qualidade, é necessário seguir um protocolo de aquisição e análise rigoroso, como explicado por (MERCA-DANTE *et al.*, 2010). Cada imagem deve conter em posições específicas pontos anatômicos bem definidos, o que não foi possível devido à quantidade limitada de imagens disponíveis, por conta disso, os modelos treinados podem propagar erros que seriam facilmente evitáveis com imagens de melhor qualidade. A presença de artefatos, ruídos e variações na qualidade das imagens pode comprometer a precisão dos modelos de segmentação e, consequentemente, a acurácia das medidas numéricas derivadas dessas segmentações. Outro desafio foi o desenvolvimento da U-Net Fuzzy, com poucos artigos sendo encontrados na literatura sobre o tema, o que dificultou a implementação e a obtenção de resultados satisfatórios. A falta de uma base teórica sólida e de exemplos práticos na literatura exigiu um esforço adicional para adaptar os conceitos de sistemas neuro-fuzzy à arquitetura da U-Net, o que resultou em um modelo com desempenho inferior ao esperado. Vale comentar que os modelos estão intimamente ligados ao aparelho de ultrassom utilizado, já que o formato da imagem pode variar significativamente entre diferentes equipamentos, o que pode limitar a generalização dos resultados para outras situações. Uma possível solução para esse problema seria a utilização de técnicas que retirem informações específicas do aparelho, deixando apenas as informações relevantes para a segmentação.

Por conta de como as regras no módulo fuzzy são implementadas, a quantidade de parâmetros aumenta de forma significativa com o número de regras, o que limita a quantidade de regras que podem ser utilizadas. No caso da U-Net Fuzzy, 9 regras foram utilizadas, o que resultou em um modelo com 8.060.478 parâmetros, enquanto a U-Net multitarefa possui 2.729.249 parâmetros. Ainda sim, as redes desenvolvidas são relativamente pequenas, essa característica representa uma vantagem em termos de eficiência computacional, pois favorece a implementação em ambientes com recursos limitados e abre a possibilidade de adaptação do aplicativo desenvolvido para dispositivos móveis.

Como trabalhos futuros, podem ser estudados e implementados modelos que utilizem o algoritmo de atenção, desenvolvido pela Google (VASWANI *et al.*, 2017), em conjunto com a U-Net, como também a utilização de técnicas de transferência de aprendizado (*fine tuning*),

que podem ajudar a melhorar o desempenho dos modelos, especialmente quando a quantidade de dados disponíveis é limitada.

Referências Bibliográficas

- ABCCARACU. *História da raça caracu*. 2022. Acessado: 2026-04-03. Disponível em: <https://www.abccaracu.com.br/historia-da-raca-caracu/>.
- ABIEC. *Beef Report — Perfil da Pecuária no Brasil*. 2021. Acessado: 2026-04-3. Disponível em: <https://abiec.com.br/publicacoes/beef-report-2025-perfil-da-pecuaria-no-brasil/>.
- Aloka Co. Ltd. *Aloka Echo Camera Model SSD-500: Operator's Manual*. Japan, 1990. Manual No. MN1-0346, Effective software version 5.0, issued August 27, 1990.
- BEAR, M. F.; CONNORS, B. W.; PARADISO, M. A. *Neurociências: desvendando o sistema nervoso*. 4. ed. Porto Alegre: Artmed, 2017. ISBN 9788582713091.
- BENGIO, Y. Practical recommendations for gradient-based training of deep architectures. *Cornell University*, Version 2, 2012.
- CANDIDO, G. *R Quadrado: coeficiente de determinação*. 2024. Acesso em: 23 abr. 2026. Disponível em: <https://medium.com/@lg0702/r-quadrado-3318259ff2e2>.
- CHOWDHURY, M. *Padding and Strides in CNN*. 2024. Acessado: 2026-04-14. Disponível em: <https://medium.com/@minhazc.engg/padding-and-strides-in-cnn-58dc56493887>.
- DATA SCIENCE ACADEMY. *Comparativo Técnico – Label Encoding vs One-Hot Encoding em Machine Learning*. 2025. Acessado: 2026-04-13. Disponível em: <https://blog.dsacademy.com.br/comparativo-tecnico-label-encoding-vs-one-hot-encoding-em-machine-learning/>.
- DEEP LEARNING BOOK. *Capítulo 10: As Principais Arquiteturas de Redes Neurais*. 2025. Acessado: 2026-03-26. Disponível em: <https://www.deeplearningbook.com.br/>.
- DEEP LEARNING BOOK. *Capítulo 19: Overfitting e Regularização: Parte 1*. 2025. Acessado: 2026-01-26. Disponível em: <https://www.deeplearningbook.com.br/overfitting-e-regularizacao-parte-1/>.
- DEEP LEARNING BOOK. *Capítulo 22 – Regularização L1*. 2025. Acessado: 2026-03-26. Disponível em: <https://www.deeplearningbook.com.br/>.
- DICE, L. R. Measures of the amount of ecologic association between species. *Ecology*, v. 26, n. 3, p. 297–302, 1945.
- ELHARROUSS, O.; MAHMOOD, Y.; BECHQITO, Y.; SERHANI, M. A.; BADIDI, E.; RIFFI, J.; TAIRI, H. Loss functions in deep learning: A comprehensive review. *arXiv preprint arXiv:2504.04242*, 2025. Disponível em: <https://arxiv.org/abs/2504.04242>.
- FELICIO, P. E. Classificação, tipificação e qualidade da carne bovina. *VI Congresso Brasileiro de Ciência e Tecnologia de Carnes*, São Pedro, SP, Brasil, p. 9, 2010.

GARCAO, L. M. C. Características socioeconômicas da pecuária brasileira: características socioeconômicas de los bovinos brasileños. *Revista Mirante*, Anápolis, GO, Brasil, jun. 2015. ISSN 1981-4089.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016. <http://www.deeplearningbook.org>.

GOOGLE LLC. *TensorFlow*. 2015. Acesso em: 11 abr. 2026. Disponível em: <https://www.tensorflow.org/>.

GUARALDO, M. C. *Brasil é o quarto maior produtor de grãos e o maior exportador de carne bovina do mundo, diz estudo*. 2021. Acessado: 2026-04-3. Disponível em: <https://www.embrapa.br/busca-de-noticias/-/noticia/62619259/>.

HECHT-NIELSEN, R. Theory of the backpropagation neural network. *Academic Press*, 1992.

HUMANSIGNAL. *Label Studio: Open Source Data Labeling Platform*. 2020. Acesso em: 11 abr. 2026. Disponível em: <https://labelstud.io/>.

JAFELICE, R. S. M.; BARROS, L. C.; BASSANEZI, R. C. *Teoria dos Conjuntos Fuzzy com Aplicações*. 3. ed. São Paulo -SP, Brasil: SBMAC, 2023. ISBN 978-65-86399-14-5.

JANG, J.-S. R. Anfis: Adaptive-network-based fuzzy inference system. *IEEE Transactions on Systems, Man, and Cybernetics*, v. 23, n. 3, p. 665–685, 1993.

KERAS TEAM. *Keras: Deep Learning for Humans*. 2015. Acesso em: 11 abr. 2026. Disponível em: <https://keras.io/>.

KIRICHEV, M. M.; SLAVOV, T. S.; MOMCHEVA, G. D. Fuzzy U-net neural network architecture optimization for image segmentation. *IOP Conference Series: Materials Science and Engineering*, IOP Publishing, 2021.

LAMAS, F. M. *Artigo - A evolução da agricultura no Brasil*. 2023. Acessado: 2026-04-3. Disponível em: https://www.embrapa.br/web/portal/tema-integracao-lavoura-pecuaria-floresta-ilpf/busca-de-noticias/-/noticia/81665485/artigo---a-evolucao-da-agricultura-do-brasil?p_auth=HeYmQbdG.

LECUN, Y. *The MNIST Database of Handwritten Digits*. 1998. Acessado: 2026-02-22. Disponível em: <http://yann.lecun.com/exdb/mnist/>.

LETTVIN, J. Y.; MATURANA, H. R.; MCCULLOCH, W. S.; PITTS, W. H. What the frog's eye tells the frog's brain. *Proceedings of the Institute of Radio Engineers*, v. 47, n. 11, p. 1940–1951, 1959.

LINUX MINT TEAM. *Linux Mint*. 2026. Acesso em: 23 abr. 2026. Disponível em: <https://linuxmint.com/>.

MCCULLOCH, W.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 1943.

MELO, M. J.; GONÇALVES, D. N.; GOMES, M. N. B.; FARIA, G.; SILVA, J. A.; RAMOS, A. P. M.; OSCO, L. P.; FURUYA, M. T. G.; JUNIOR, J. M.; GONÇALVES, W. N. Automatic segmentation of cattle rib-eye area in ultrasound images using the UNet++ deep neural network. *Computers and Electronics in Agriculture*, Elsevier, v. 195, p. 106818, 2022.

MERCADANTE, M. E. Z.; SILVA, S. d. Luz e; BUENO, M. S.; TAROUÇO, J. U.; YOKOO, M. J. I. Repetibilidade da mensuração de imagens das características de carcaça obtidas por ultrassonografia em fêmeas nelore. *Revista Brasileira de Zootecnia*, v. 39, n. 4, p. 752–757, 2010. ISSN 1806-9290.

MICROSOFT. *Visual Studio Code*. 2015. Acesso em: 23 abr. 2026. Disponível em: <https://code.visualstudio.com/>.

MÜLLER, V. C. Review of margaret boden, “mind as machine: A history of cognitive science”. *Minds and Machines*, Springer, v. 18, n. 1, p. 121–125, 2008.

NATIONAL INSTITUTES OF HEALTH. *ImageJ: Image Processing and Analysis in Java*. 2026. Acessado em: 2026-04-04. Disponível em: <https://imagej.net/ij/>.

O’SHEA, K.; NASH, R. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015. Disponível em: <https://arxiv.org/abs/1511.08458>.

PURWONO, P.; NATALIANI, Y.; PURNOMO, H. D.; TIMOTIUS, I. K. Dfu-fuzzyliteunet: A lightweight U-net with fuzzy sigmoid and lite transformer for diabetic foot ulcer segmentation. *Biomedical Signal Processing and Control*, Elsevier, v. 108, p. 107902, 2025.

PYTHON SOFTWARE FOUNDATION. *Python*. 1990. Acesso em: 23 abr. 2026. Disponível em: <https://www.python.org/>.

PYTHON SOFTWARE FOUNDATION. *tkinter — Python interface to Tcl/Tk*. 1991. Acesso em: 23 abr. 2026. Disponível em: <https://docs.python.org/3/library/tkinter.html>.

RONNEBERGER, O.; FISCHER, P.; BROX, T. U-net: Convolutional networks for biomedical image segmentation. In: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Munich, Germany: Springer, 2015. p. 234–241.

ROSENBLATT, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 1958.

RUDER, S. An overview of gradient descent optimization algorithms. *Insight Centre for Data Analytics, NUI Galway Aylien Ltd., Dublin*, 2016.

SENA, V.; JAFELICE, R. S. M.; SANTOS, C. A. R. dos. Reconstrução geométrica de uma função através de uma rede neural. In: *Anais da XXIV Semana da Matemática e XIV Semana da Estatística (SEMAT & SEMEST)*. Uberlândia, MG, Brasil: UFU, 2024. Disponível em: https://drive.google.com/file/d/1OmN2Gq6i3nHR7ZtEVitc5iCVH0ZFUY_8/view.

SENA, V.; JAFELICE, R. S. M.; SANTOS, C. A. R. dos. Reconstrução geométrica de uma função através de uma rede neural. In: *Caderno de Resumos do I Workshop de Matemática da UFU*. Uberlândia, MG, Brasil: UFU, 2025. Disponível em: <https://eventos.ufu.br/ime/wm>.

SOVIERZOSKI, M. A. Convolução de sinais: definição, propriedades e ferramentas. *Revista Ilha Digital*, v. 2, p. 81–95, 2010. ISSN 2177-2649. Disponível em: <https://ilhadigital.florianopolis.ifsc.edu.br/index.php/ilhadigital>.

STANFIELD, C. L. *Fisiologia Humana*. São Paulo: Pearson Education do Brasil, 2014. ISBN 978-85-430-1426-5.

SUGUISAWA, L.; MATOS, B. da Conceição de; SUGUISAWA, J. M. Uso da ultrassonografia na avaliação de características de carcaça e de qualidade da carne. In: ROSA, A. do N.; MARTINS, E. N.; MENEZES, G. R. de O.; SILVA, L. O. C. da (Ed.). *Melhoramento Genético Aplicado em Gado de Corte*. Brasília, DF, Brasil: Embrapa, 2013. p. 98–107.

SUTSKEVER, I. On the importance of initialization and momentum in deep learning. *International Conference on Machine Learning*, 2013.

SZELISKI, R. *Computer Vision: Algorithms and Applications*. Cham, Suíça: Springer, 2010. ISBN 978-3030343712.

TileStats. *U-Net Clearly Explained — Image Segmentation with AI*. 2025. Vídeo publicado em 23 jun. 2025. Acesso em: 05 jun. 2026. Disponível em: <https://www.youtube.com/watch?v=oxcgx75k6yU&t=1324s>.

TREVISAN, V. *Como funcionam as Redes Neurais Convolucionais (CNNs)*. 2021. Acessado: 2025-12-16. Disponível em: <https://medium.com/data-hackers/como-funcionam-as-redes-neurais-convolucionais-cnns-71978185c1>.

UNITED STATES DEPARTMENT OF AGRICULTURE. *Livestock and Poultry: World Markets and Trade*. Washington, DC, USA, 2026. Acesso em: 23 abr. 2026. Disponível em: <https://www.fas.usda.gov/data/livestock-and-poultry-world-markets-and-trade>.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, L.; POLOSUKHIN, I. Attention is all you need. In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. Curran Associates, Inc., 2017. p. 5998–6008. Disponível em: <https://arxiv.org/abs/1706.03762>.

VERMA, H.; IM, K.; GUPTA, A.; TANVEER, M. A novel framework using intuitionistic fuzzy logic with u-net and u-net++ architecture: A case study of mri brain image segmentation. *arXiv preprint arXiv:2603.18042*, 2026. Disponível em: <https://arxiv.org/abs/2603.18042>.

WERBOS, P. J. Backpropagation through time: What it does and how to do it? *PROCEEDINGS OF THE IEEE*, 1990.

WILSON, D. E. *Scanning into the Future: Real-Time Ultrasound in Beef Cattle*. 1998. ASB 1998-DEW-417. Disponível em: <https://admin.webplus.com.br/public/upload/downloads/090220121626346252000AHZV.pdf>.

ZADEH, L. A. Fuzzy sets. *Information and Control*, v. 8, n. 3, p. 338–353, 1965.

ZANCANER, R. *História da raça nelore*. 2026. Acessado: 2026-04-03. Disponível em: <https://zancaner.com.br/site/conteudo/1-historia-da-raca-nelore.html>.

ZHU, H.; ROHLING, R.; SALCUDEAN, S. Multi-task unet: Jointly boosting saliency prediction and disease classification on chest x-ray images. *arXiv preprint arXiv:2202.07118*, 2022. Disponível em: <https://arxiv.org/abs/2202.07118>.